



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



DESARROLLO DE UN SISTEMA WEB BASADO EN EMBEDDINGS PARA UNA FARMACIA

ALBERT COMAS PACHECO

Director/a

CRISTINA GÓMEZ SEOANE (Departamento de Ingeniería de Servicios y Sistemas de Información)

Titulación

Grado en Ingeniería Informática (Ingeniería del Software)

Memoria del trabajo de fin de grado

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

25/06/2025

Agradecimientos

A Cristina Gómez, por la ayuda durante esta etapa,
A mi familia, por su gran interés desde el inicio,
A mis amigos, por hacer más ameno estos años,
A Laura, por estar el día a día soportándome,
A mis padres, Yolanda y Jose, gracias.

Resumen

Este proyecto desarrolla un sistema web para la gestión de productos farmacéuticos, destacando el uso de modelos de embeddings complementados con IA para búsquedas semánticas, recomendaciones y gestión de recetas. El propósito del proyecto es mejorar la accesibilidad y eficiencia en la compra y gestión de productos farmacéuticos.

Los usuarios pueden gestionar recetas electrónicas y privadas, encargar productos con y sin receta, y recibir notificaciones sobre su medicación. Los farmacéuticos pueden gestionar los encargos, mientras que los administradores supervisan los usuarios y los productos. El sistema ha sido diseñado con aportes de profesionales del sector y el análisis de farmacias online.

Resum

Aquest projecte desenvolupa un sistema web per a la gestió de productes farmacèutics, destacant l'ús de models d'*embeddings* complementats amb IA per a cerques semàntiques, recomanacions i gestió de receptes. L'objectiu del projecte és millorar l'accessibilitat i l'eficiència en la compra i gestió de productes farmacèutics.

Els usuaris poden gestionar receptes electròniques i privades, encarregar productes amb i sense recepta, i rebre notifikacions sobre la seva medicació. Els farmacèutics poden gestionar els encàrrecs, mentre que els administradors supervisen els usuaris i els productes. El sistema ha estat dissenyat amb aportacions de professionals del sector i l'anàlisi de farmàcies en línia.

Abstract

This project develops a web system for managing pharmaceutical products, highlighting the use of embedding models complemented by AI for semantic searches, recommendations and prescription management. The project's goal is to improve accessibility and efficiency in purchasing and managing pharmaceutical products.

Users can manage electronic and private prescriptions, order products with or without a prescription, and receive notifications about their medication. Pharmacists can manage orders, while administrators oversee users and products. The system has been designed with input from industry professionals and the analysis of online pharmacies.

Índice de contenidos

1. Introducción.....	11
1.1 Contexto.....	11
1.2 Glosario de términos.....	12
1.3 Descripción del problema.....	13
1.4 Stakeholders.....	13
1.5 Motivación.....	14
1.6 Flujo de trabajo en una farmacia.....	14
2. Justificación.....	16
2.1 Mercado actual.....	16
3. Alcance.....	18
3.1 Objetivos.....	18
3.2 Requisitos.....	18
3.2.1 Requisitos funcionales.....	19
3.2.2 Requisitos no funcionales.....	21
3.3 Obstáculos y riesgos.....	23
4. Metodología.....	24
4.1 Metodología de trabajo.....	24
4.2 Herramientas de seguimiento.....	25
5. Descripción de las tareas.....	27
5.1 Tareas.....	28
5.1.1 Gestión del proyecto.....	28
5.1.2 Desarrollo.....	29
5.1.2.1 Inception.....	29
5.1.2.2 Sprint 1.....	29
5.1.2.3 Sprint 2.....	30
5.1.2.4 Sprint 3.....	31
5.1.2.5 Sprint 4.....	31
5.1.3 Finalización.....	32
5.2 Recursos.....	33
5.2.1 Recursos humanos.....	33
5.2.2 Recursos materiales.....	33
6. Estimaciones y diagrama de Gantt.....	35
6.1 Estimaciones.....	35
6.2 Diagrama de Gantt.....	37
7. Gestión de riesgos.....	38
8. Gestión económica.....	39
8.1 Identificación de los costes.....	39
8.1.1 Costes humanos.....	39

8.1.2 Costes materiales.....	41
8.1.2.1 Hardware.....	41
8.1.2.2 Software.....	42
8.1.2.3 Indirectos.....	42
8.2 Estimación de los costes.....	43
8.2.1 Contingencias.....	43
8.2.1 Imprevistos.....	43
8.2.1 Coste total.....	44
8.3 Control de gestión.....	44
9. Especificación.....	46
9.1 Diagrama de casos de uso.....	46
9.2 Descripción de casos de uso.....	50
9.3 Modelo conceptual.....	59
9.3.1 Esquema conceptual de los datos.....	59
9.3.2 Restricciones de integridad.....	60
9.3.3 Descripción de las clases.....	61
9.4 Modelo de comportamiento.....	63
10. Diseño.....	74
10.1 Arquitectura física.....	74
10.1.1 Front-end.....	74
10.1.2 Back-end.....	75
10.2 Arquitectura lógica.....	76
10.2.1 Front-end.....	77
10.2.2 Back-end.....	79
10.3 Diseño del front-end.....	81
10.3.1 Diseño externo.....	81
10.3.2 Diseño interno.....	91
10.4 Diseño del back-end.....	95
10.4.1 Métodos públicos.....	95
10.4.2 Diagrama de clases.....	97
10.4.3 Diagrama de secuencia.....	99
10.4.4 Esquema de la base de datos.....	100
11. Implementación.....	104
11.1 Implementación del front-end.....	104
11.1.1 Sprint 1.....	105
11.1.2 Sprint 2.....	108
11.1.3 Sprint 3.....	110
11.1.4 Sprint 4.....	112
11.2 Implementación del back-end.....	114
11.2.1 Sprint 1.....	114
11.2.2 Sprint 2.....	117

11.2.3 Sprint 3.....	121
11.2.4 Sprint 4.....	122
12. Validación.....	124
12.1 Validación de los requisitos funcionales.....	124
12.2 Validación de los requisitos no funcionales.....	127
13. Identificación de leyes y regulaciones.....	132
14. Resultados de la gestión del proyecto.....	134
14.1 Resultados de la metodología.....	134
14.2 Resultados de la planificación.....	135
14.3 Resultados económicos.....	138
15. Informe de sostenibilidad.....	139
15.1 Sostenibilidad económica.....	139
15.2 Sostenibilidad social.....	139
15.3 Sostenibilidad ambiental.....	140
15.4 Reflexión personal.....	140
16. Conclusiones.....	141
16.1 Conclusiones del proyecto.....	141
16.2 Justificación de las competencias.....	142
16.3 Integración del conocimiento.....	143
16.4 Valoración personal.....	144
16.5 Acciones futuras.....	145
Bibliografía.....	146
Anexo 1: Descripción de casos de uso.....	151
Anexo 2: Modelo de comportamiento.....	161
Anexo 3: Diseño externo.....	169

Índice de figuras

Figura 1: Compra de medicamentos. [2].....	11
Figura 2: Scrum Process. [17].....	24
Figura 3: Flujo de trabajo GitFlow. [20].....	26
Figura 4: Diagrama de Gantt. Elaborado con Lucidchart.....	37
Figura 5: Diagrama de casos de uso de gestión de cuentas. Elaborado con Lucidchart.....	46
Figura 6: Diagrama de casos de uso de búsqueda y recomendaciones. Elaborado con Lucidchart.....	47
Figura 7: Diagrama de casos de uso de gestión de recetas. Elaborado con Lucidchart.....	48
Figura 8: Diagrama de casos de uso de notificaciones personalizadas. Elaborado con Lucidchart.....	48
Figura 9: Diagrama de casos de uso de gestión de cuentas. Elaborado con Lucidchart.....	48
Figura 10: Diagrama de casos de uso de gestión de cuentas. Elaborado con Lucidchart.....	49
Figura 11: Diagrama de casos de uso de gestión de cuentas. Elaborado con Lucidchart.....	49
Figura 12: Esquema conceptual de los datos. Elaborado con Lucidchart.....	59
Figura 13: Diagrama de modificar perfil. Elaborado con Lucidchart.....	63
Figura 14: Diagrama de recuperar contraseña. Elaborado con Lucidchart.....	64
Figura 15: Diagrama de búsqueda semántica. Elaborado con Lucidchart.....	65
Figura 16: Diagrama de consultar recomendaciones. Elaborado con Lucidchart.....	66
Figura 17: Diagrama de valorar búsqueda. Elaborado con Lucidchart.....	66
Figura 18: Diagrama de cargar receta. Elaborado con Lucidchart.....	67
Figura 19: Diagrama de recordatorios. Elaborado con Lucidchart.....	68
Figura 20: Diagrama de consultar duda. Elaborado con Lucidchart.....	69
Figura 21: Diagrama de solucionar duda. Elaborado con Lucidchart.....	70
Figura 22: Diagrama de cambiar estado consulta. Elaborado con Lucidchart.....	70
Figura 23: Diagrama de añadir producto de receta a carrito. Elaborado con Lucidchart.....	71
Figura 24: Diagrama de encargar productos. Elaborado con Lucidchart.....	71
Figura 25: Diagrama de informar usuario sobre encargo. Elaborado con Lucidchart.....	72
Figura 26: Diagrama de editar producto. Elaborado con Lucidchart.....	73
Figura 27: Arquitectura física del sistema. Elaborado con Lucidchart.....	74
Figura 28: Arquitectura lógica del sistema. Elaborado con Lucidchart.....	76
Figura 29: Arquitectura lógica del front-end. Elaborado con Lucidchart.....	77
Figura 30: Patrón Observador. [42].....	78
Figura 31: Patrón Fachada. [44].....	78
Figura 32: Patrón Singleton. [45].....	78
Figura 33: Arquitectura lógica del back-end. Elaborado con Lucidchart.....	79
Figura 34: Patrón Estado. [47].....	80
Figura 35: Diseño inicio de usuarios. Elaborado con Wix.....	81
Figura 36: Diseño catálogo de productos. Elaborado con Wix.....	82

Figura 37: Diseño de detalle de producto. Elaborado con Wix.....	83
Figura 38: Diseño de recomendaciones. Elaboración con Wix.....	83
Figura 39: Diseño de gestión de recetas público. Elaborado con Wix.....	84
Figura 40: Diseño de gestión de recetas. Elaborado con Wix.....	85
Figura 41: Diseño de carrito. Elaborado con Wix.....	85
Figura 42: Diseño de encargos público. Elaborado con Wix.....	86
Figura 43: Diseño de encargos. Elaborado con Wix.....	87
Figura 44: Diseño de contacto. Elaborado con Wix.....	87
Figura 45: Diseño inicio de farmacéuticos. Elaborado con Wix.....	88
Figura 46: Diseño de asesoramiento a usuarios. Elaborado con Wix.....	89
Figura 47: Diseño inicio de administrador. Elaborado con Wix.....	89
Figura 48: Diseño de gestión de productos. Elaborado con Wix.....	90
Figura 49: Diseño interno del front-end. Elaborado con Lucidchart.....	91
Figura 50: Diagrama de secuencia de navegación inicial. Elaborado con Lucidchart.....	93
Figura 51: Diagrama de secuencia de producto concreto. Elaborado con Lucidchart.....	94
Figura 52: Diagrama de clases del back-end. Elaborado con Lucidchart.....	98
Figura 53: Diagrama de secuencia de registro de usuario. Elaborado con Lucidchart.....	99
Figura 54: Carpetas del front-end. Elaboración propia.....	104
Figura 55: Código de actualizar el token. Elaboración propia.....	105
Figura 56: Código de comprobar rol y autenticación de usuario. Elaboración propia.....	106
Figura 57: Código de obtener productos iniciales. Elaboración propia.....	107
Figura 58: Código de filtros para búsquedas. Elaboración propia.....	107
Figura 59: Código de búsqueda semántica en el front-end. Elaboración propia.....	109
Figura 60: Código de manejo de subir receta. Elaboración propia.....	110
Figura 61: Código de manejo de cambio de estado. Elaboración propia.....	111
Figura 62: Código de envío de consulta. Elaboración propia.....	112
Figura 63: Código de añadir un recordatorio. Elaboración propia.....	113
Figura 64: Carpetas del back-end. Elaboración propia.....	114
Figura 65: Código de crear usuario. Elaboración propia.....	115
Figura 66: Código autenticación con Google. Elaboración propia.....	115
Figura 67: Código de envío de correo con Resend. Elaboración propia.....	116
Figura 68: Código de generación de embeddings. Elaboración propia.....	117
Figura 69: Código de subir una imagen a S3. Elaboración propia.....	118
Figura 70: Código de fine-tuning productos. Elaboración propia.....	118
Figura 71: Código de búsqueda vectorial. Elaboración propia.....	119
Figura 72: Código de proceso de subida de receta. Elaboración propia.....	120
Figura 73: Código de recuperación de recomendaciones. Elaboración propia.....	121
Figura 74: Código para actualizar el estado del encargo. Elaboración propia.....	122
Figura 75: Código de configuración de BackgroundScheduler. Elaboración propia.....	123
Figura 76: Tests unitarios del back-end. Elaboración propia.....	124

Figura 77: Cobertura de los tests del back-end. Elaboración propia.....	125
Figura 78: Fixture de los tests del back-end. Elaboración propia.....	125
Figura 79: Test de obtener recomendaciones. Elaboración propia.....	126
Figura 80: Mock de historial de búsquedas. Elaboración propia.....	126
Figura 81: Resultados de la primera pregunta de la encuesta. Elaborado con Google Forms.....	127
Figura 82: Resultados de la segunda pregunta de la encuesta. Elaborado con Google Forms.....	128
Figura 83: Resultados de la tercera pregunta de la encuesta. Elaborado con Google Forms.....	128
Figura 84: Resultados de la cuarta pregunta de la encuesta. Elaborado con Google Forms.....	129
Figura 85: Tiempo de las búsquedas. Elaborado con herramienta de desarrollo de Chrome.....	130
Figura 86: Página de inicio responsiva. Elaboración propia.....	130
Figura 87: Página de contacto responsiva. Elaboración propia.....	131
Figura 88: Diagrama de registro. Elaborado con Lucidchart.....	161
Figura 89: Diagrama de inicio de sesión. Elaborado con Lucidchart.....	161
Figura 90: Diagrama de cerrar sesión. Elaborado con Lucidchart.....	162
Figura 91: Diagrama de eliminar cuenta. Elaborado con Lucidchart.....	162
Figura 92: Diagrama de consultar usuario. Elaborado con Lucidchart.....	163
Figura 93: Diagrama de consultar catálogo. Elaborado con Lucidchart.....	164
Figura 94: Diagrama de consultar historial. Elaborado con Lucidchart.....	164
Figura 95: Diagrama de consultar FAQ. Elaborado con Lucidchart.....	165
Figura 96: Diagrama de consultar recetas. Elaborado con Lucidchart.....	165
Figura 97: Diagrama de eliminar receta. Elaborado con Lucidchart.....	166
Figura 98: Diagrama de consultar encargos. Elaborado con Lucidchart.....	167
Figura 99: Diagrama de añadir producto. Elaborado con Lucidchart.....	167
Figura 100: Diagrama de eliminar producto. Elaborado con Lucidchart.....	168
Figura 101: Diseño de inicio de usuarios II. Elaboración propia.....	169
Figura 102: Diseño de inicio de usuarios III. Elaboración propia.....	169
Figura 103: Diseño de inicio de sesión. Elaboración propia.....	170
Figura 104: Diseño de registro. Elaboración propia.....	170
Figura 105: Diseño de recuperar contraseña. Elaboración propia.....	171
Figura 106: Diseño de perfil de usuario. Elaboración propia.....	171
Figura 107: Diseño de historial de búsqueda. Elaboración propia.....	172
Figura 108: Diseño de recordatorios. Elaboración propia.....	172
Figura 109: Diseño de FAQ. Elaboración propia.....	173
Figura 110: Diseño de encargos para farmacéuticos. Elaboración propia.....	173
Figura 111: Diseño de consulta de usuarios. Elaboración propia.....	174

Figura 112: Diseño de gestión de farmacéuticos. Elaboración propia.....	175
Figura 113: Diseño de añadir producto. Elaboración propia.....	175
Figura 114: Diseño de editar producto. Elaboración propia.....	176
Figura 115: Diseño de estadísticas. Elaboración propia.....	176

Índice de tablas

Tabla 1: Comparativa de las propuestas de mercado. Elaboración propia.....	17
Tabla 2: Planificación de cada etapa. Elaboración propia.....	27
Tabla 3: Recursos humanos del proyecto. Elaboración propia.....	33
Tabla 4: Recursos materiales del proyecto. Elaboración propia.....	33
Tabla 5: Estimación de horas, dependencias y recursos. Elaboración propia.....	35
Tabla 6: Impacto y probabilidad de riesgos. Elaboración propia.....	38
Tabla 7: Sueldos por rol y hora. Elaboración propia.....	39
Tabla 8: Costes humanos por cada tarea. Elaboración propia.....	40
Tabla 9: Costes y amortización del hardware. Elaboración propia.....	42
Tabla 10: Costes y amortización del software. Elaboración propia.....	42
Tabla 11: Costes y amortización del software. Elaboración propia.....	43
Tabla 12: Costes materiales. Elaboración propia.....	43
Tabla 13: Costes con contingencias. Elaboración propia.....	43
Tabla 14: Costes de imprevistos. Elaboración propia.....	44
Tabla 15: Presupuesto final. Elaboración propia.....	44
Tabla 16: Diferencia entre horas estimadas y reales. Elaboración propia.....	135
Tabla 17: Resumen de las desviaciones presupuestarias de costes humanos. Elaboración propia.....	138

1. Introducción

El proyecto *Desarrollo de un sistema web basado en embeddings para una farmacia* es mi trabajo de fin de grado para la Facultad de Informática de Barcelona, concretamente para el grado en Ingeniería Informática y la especialidad de Ingeniería del Software.

1.1 Contexto

El comercio electrónico en el sector de la salud y farmacia en España ha crecido mucho en los últimos años, con un aumento del 40% en cinco años [1]. Esto muestra cómo cada vez más personas optan por soluciones digitales y buscan maneras más cómodas para comprar medicamentos y productos de farmacia desde casa.

Como se ve en la Figura 1, en 2023 un 93% todavía compraba medicamentos en las farmacias físicas, mientras que sólo un 13% hacía compras online. Comparado con otros países como Alemania, donde el 39% compra por internet, o Brasil, con un 30%, en España parece que todavía estamos un poco más rezagados en este tema. Pero con el tiempo y el crecimiento del comercio online, seguramente esta tendencia siga creciendo en los próximos años.

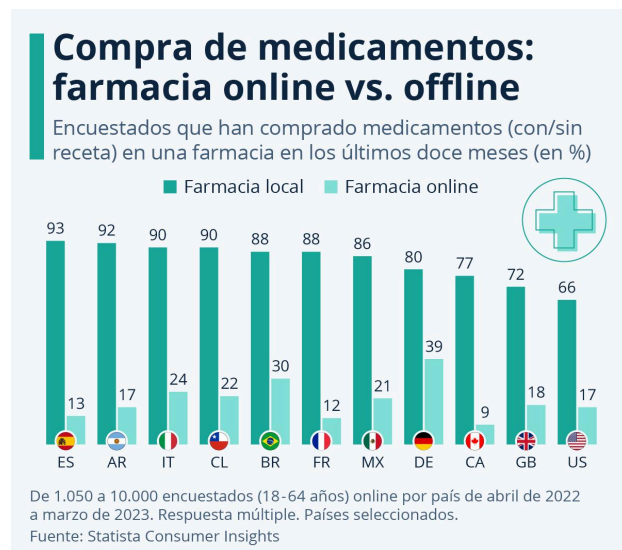


Figura 1: Compra de medicamentos. [2]

Este escenario nos muestra que el mercado digital en la farmacia en España todavía tiene mucho espacio para crecer y para innovar. Factores como la confianza en las compras por internet, las regulaciones y lo fácil que sea acceder a los productos van a jugar un papel clave en cómo evoluciona. Todo esto significa que aún hay muchas oportunidades para avanzar y hacer que este sector siga alineándose con las tendencias globales de digitalización en salud.

1.2 Glosario de términos

- a) **Embedding:** Un embedding es una representación de datos en forma de vectores de números reales que captura sus significados y relaciones, facilitando su procesamiento por modelos de aprendizaje automático. [3]
- b) **Búsqueda semántica:** La búsqueda semántica es una tecnología que interpreta el significado y la intención detrás de las consultas de los usuarios (y no sólo las palabras clave), para ofrecer resultados más relevantes. [4]
- c) **Front-end:** El front-end es la parte de una aplicación que interactúa directamente con los usuarios, incluyendo elementos visuales como botones, menús y gráficos. [5]
- d) **Back-end:** El back-end se refiere a la infraestructura que soporta la aplicación, gestionando la lógica del negocio, bases de datos y servidores que procesan las solicitudes del usuario. [5]
- e) **FAQ:** "Frequently Asked Questions" (Preguntas Frecuentes), es una sección en sitios web que proporciona respuestas a las consultas más comunes de los usuarios, facilitando la resolución autónoma de dudas sobre productos o servicios.
- f) **Receta privada:** Una receta privada es una prescripción médica emitida por un profesional de la salud en el ámbito privado, fuera del sistema público de salud, que permite la dispensación de medicamentos en farmacias.
- g) **Receta electrónica:** La receta electrónica de la Seguridad Social es un sistema digital que permite a los médicos prescribir medicamentos y tratamientos de manera electrónica, facilitando su dispensación en farmacias.
- h) **Carrito:** En este proyecto, el término "carrito" se utiliza para referirse a la lista de productos que se desean encargar.

1.3 Descripción del problema

El sector farmacéutico online en España enfrenta desafíos que dificultan su integración en el mercado digital. Aunque cada vez más plataformas permiten comprar o consultar medicamentos, todavía hay problemas como la validación de recetas electrónicas, la gestión de medicamentos y la digitalización de las farmacias [6]. Además, la falta de un estándar en los catálogos en línea y la gran cantidad de nombres comerciales diferentes confunden a los usuarios, complicando la búsqueda de medicamentos [7]. Aunque existen sistemas digitales para comprar fármacos, todavía no logran clasificar y recuperar la información de manera efectiva usando por ejemplo, herramientas que entiendan las similitudes semánticas entre productos.

Este proyecto busca abordar esta problemática mediante un sistema web basado en *embeddings* que optimice la búsqueda de medicamentos, la gestión de información y otros procesos importantes. La solución mejorará la experiencia tanto de los usuarios como de los farmacéuticos, aumentando la eficiencia en las farmacias. Se diseñará una herramienta adaptada a las necesidades del sector, priorizando accesibilidad y precisión. La intención es que tanto farmacias como consumidores puedan encontrar más fácilmente los medicamentos, hacer compras en línea más rápidas y colaborar en la digitalización del mundo farmacéutico.

1.4 Stakeholders

Los *stakeholders* son todas aquellas partes interesadas, beneficiadas e implicadas en el proyecto. A continuación, se listan los diferentes *stakeholders* del proyecto:

- **Clientes:** Beneficiarios directos del sistema, quienes pueden usar el sistema web final. Está formado por usuarios (pacientes), farmacéuticos y personal administrativo de la farmacia
- **Desarrollador:** La persona encargada de desarrollar todo el software, documentar el proyecto y asegurar su correcto funcionamiento. En este proyecto sólo hay un desarrollador: Albert Comas.
- **Testers de UX:** Grupo de personas cercanas al desarrollador, que proporcionan feedback directo sobre la interfaz, usabilidad y accesibilidad del sistema.
- **Proveedores de productos:** Los encargados de mantener la disponibilidad de productos en la farmacia.
- **Reguladores y autoridades sanitarias:** Aseguran que el sistema cumpla con las normativas sanitarias y de privacidad de datos.

1.5 Motivación

El interés por este proyecto surge tanto de mi formación en ingeniería del software como de mi experiencia cercana con el sector farmacéutico. Mi madre, que trabaja en una farmacia, me ha contado lo difícil que resulta para profesionales y clientes comprar medicamentos en línea, sobre todo por la falta de herramientas digitales que funcionen de verdad.

Estos problemas, sumados al crecimiento del comercio electrónico, muestran claramente la necesidad de soluciones tecnológicas que hagan más fácil y eficiente la gestión en las farmacias. Con este proyecto, busco desarrollar un sistema que no solo optimice la búsqueda de medicamentos mediante técnicas avanzadas, sino que también ayude a organizar mejor la información y a digitalizar los procesos en las farmacias. Esto beneficiará tanto a los usuarios como a los profesionales del sector.

1.6 Flujo de trabajo en una farmacia

Este apartado tiene el objetivo de describir el flujo de trabajo en una farmacia física para que se entiendan mejor todos los procesos que se van a llevar a cabo en este proyecto.

1. Recepción del paciente o cliente

Cuando un usuario llega a la farmacia, puede hacerlo por diversas razones:

- Comprar un medicamento con receta o sin receta.
- Adquirir productos de parafarmacia.
- Solicitar asesoramiento sobre medicamentos, síntomas o tratamientos.

2. Verificación de la receta (si aplica)

Si el cliente presenta una receta médica, el farmacéutico debe comprobar su validez:

- Receta electrónica: Se accede al sistema de salud con la Tarjeta Sanitaria Individual (TSI).
- Receta en papel (de la Seguridad Social): Se comprueba la identidad del cliente con la Tarjeta Sanitaria Individual (TSI).
- Receta privada: Se verifica que tenga todos los datos obligatorios (nombre del paciente, prescripción médica, fecha, etc.).

3. Dispensación del medicamento o producto

- Si el medicamento requiere receta, el farmacéutico procede a la dispensación sólo si la receta y la TSI son válidas.
- Si no requiere receta, el cliente puede adquirirlo libremente.
- En algunos casos, el farmacéutico puede recomendar alternativas o aclarar dudas sobre la posología.

4. Cobro y entrega del producto

- Si el medicamento es financiado por el sistema de salud, el cliente solo paga la parte que le corresponde.
- En otros casos, el cliente paga el importe total del producto.
- Se entrega el medicamento con la información necesaria sobre su uso.

5. Registro y actualización del stock

- Se actualiza el inventario de productos vendidos.
- En caso de falta de stock, se gestiona un pedido al distribuidor.

La digitalización de este flujo optimiza la dispensación de medicamentos y mejora la experiencia del usuario. Los embeddings con inteligencia artificial permiten búsquedas más intuitivas, mientras que la gestión automatizada agiliza pedidos y validaciones. Además, el asesoramiento online mantiene la calidad del servicio farmacéutico.

2. Justificación

Usar embeddings basados en inteligencia artificial puede hacer que los procesos en un sistema web de farmacia sean mucho más eficientes. A diferencia de los métodos tradicionales, que suelen depender de coincidencias exactas o reglas fijas, los embeddings convierten la información en representaciones numéricas dentro de un espacio semántico. Esto ayuda a entender relaciones complejas entre diferentes datos, mejorando la precisión y haciendo que los servicios sean más fáciles de usar [8].

La inteligencia artificial mejora estos embeddings usando modelos de aprendizaje automático entrenados con datos del sector farmacéutico y patrones de comportamiento de los usuarios, lo que permite que el sistema siga mejorando con el tiempo. Esto incluye desde la búsqueda avanzada de productos hasta recomendaciones personalizadas. Además, los embeddings permiten adaptar la plataforma automáticamente a nuevos productos o cambios en el catálogo, sin tener que hacer ajustes manuales, lo que hace que el sistema sea más escalable y eficiente.

2.1 Mercado actual

El estudio de mercado es fundamental para comprender el estado actual del comercio farmacéutico online, identificar sus limitaciones y detectar oportunidades de mejora. Analizar las plataformas existentes permite justificar la necesidad del proyecto y destacar sus ventajas competitivas.

Según estadísticas realizadas por Semrush [9], las 3 principales plataformas farmacéuticas con más visitas en España son las siguientes:

- **Atida:** Es una plataforma online de farmacia y parafarmacia que ofrece productos de salud, bienestar y asesoramiento farmacéutico. Cuenta con un amplio catálogo de medicamentos sin receta, suplementos y artículos de cuidado personal [10].
- **Promofarma:** Es un *marketplace*¹ que permite comprar productos de salud, belleza y bienestar de múltiples farmacias. Tiene una sección dedicada a ofertas y descuentos, lo que atrae a muchos clientes [11].
- **Dosfarma:** Es una farmacia online líder en España que proporciona la opción de comprar de forma segura medicamentos sin receta y productos de parafarmacia. Su catálogo incluye más de 40.000 referencias, abarcando desde productos sanitarios hasta artículos de cuidado personal y bienestar [12].

¹ Un marketplace es una plataforma digital donde vendedores y compradores intercambian bienes o servicios.

Después de ver las diferentes opciones que hay en el mercado, en la Tabla 1 se analizará las funcionalidades clave de cada propuesta comparándolo con lo que ofrece este proyecto. El color verde indica que cumple la funcionalidad correctamente, el color rojo indica que no la cumple y el color naranja que la cumple parcialmente.

Funcionalidad	Atida	Promofarma	Dosfarma	Este proyecto
Gestión de usuarios	✓	✓	✓	✓
Gestión de farmacéuticos	—	✗	—	✓
Búsqueda de productos por nombre	✓	✓	✓	✓
Búsqueda semántica de productos	✗	✗	✗	✓
Filtrar productos detalladamente	—	—	✓	✓
Recomendaciones personalizadas	✗	✗	✗	✓
Gestión de recetas	✗	✗	✗	✓
Gestión de encargos	✓	✓	✓	✓
Consultar historial de búsquedas	✗	✗	✗	✓
Notificaciones personalizadas	✗	✗	✗	✓
Valoraciones de recomendaciones y búsquedas	✗	✗	✗	✓
Asesoramiento farmacéutico	✓	✓	✗	✓

Tabla 1: Comparativa de las propuestas de mercado. Elaboración propia.

La comparación de las principales farmacias online revela varias carencias en las plataformas existentes. Este proyecto soluciona estas carencias al incorporar inteligencia artificial mediante modelos de *embeddings*, lo que permite búsquedas más precisas y recomendaciones personalizadas. Además, la integración con recetas electrónicas y la gestión de farmacéuticos refuerza su utilidad en el sector. Estas mejoras no solo optimizan la experiencia del usuario, sino que también facilitan la digitalización de las farmacias, permitiendo un acceso más eficiente y personalizado a los medicamentos.

3. Alcance

Una vez contextualizado el problema, es fundamental delimitar el alcance del proyecto. Es importante establecer claramente los objetivos y requisitos para acotar el desarrollo del sistema, así como los posibles obstáculos y riesgos del proyecto. Cualquier funcionalidad que no esté presente en los objetivos o en los requisitos, está fuera del alcance de este proyecto.

3.1 Objetivos

El objetivo principal de este proyecto es desarrollar un sistema web basado en *embeddings* que mejore la experiencia de búsqueda y recomendación de productos, además de la gestión de recetas en farmacias online. Los sub-objetivos del proyecto son:

- **Implementar un sistema de búsqueda semántica** que utilice modelos de *embeddings* para mejorar la búsqueda de productos, permitiendo encontrar medicamentos incluso si el usuario no pone el nombre exacto.
- **Permitir la gestión de recetas electrónicas y privadas** de manera sencilla y segura.
- **Proporcionar recomendaciones personalizadas** basado en el historial de búsqueda del usuario y modelos de *embeddings* para facilitar el acceso a productos relevantes.
- **Permitir la gestión de encargos** para usuarios y farmacéuticos. No se incluye la opción de envío a domicilio ya que legalmente se ha de presentar en persona la Tarjeta Sanitaria para que el farmacéutico entregue productos con receta (véase el apartado de leyes y regulaciones).
- **Incorporar un sistema de asesoramiento farmacéutico** para que los usuarios puedan resolver dudas sobre medicamentos y tratamientos en tiempo real.
- **Desarrollar una plataforma intuitiva y accesible** que garantice una navegación fluida y eficiente para clientes, farmacéuticos y administradores.

3.2 Requisitos

Los requisitos sirven como base para el diseño, desarrollo y evaluación del proyecto. Muestran una descripción detallada de las características que debe tener el sistema para satisfacer las necesidades del usuario final [13].

3.2.1 Requisitos funcionales

Los requisitos funcionales definen qué debe hacer el sistema para cumplir con los objetivos del proyecto. Especifican las funcionalidades que el sistema debe ofrecer [13]. A continuación, se detallan los requisitos funcionales del sistema:

Permitir a los usuarios, farmacéuticos y administradores gestionar su cuenta.

- Gestión común (para usuarios, farmacéuticos y administradores):
 - Iniciar sesión y cerrar sesión.
 - Modificar su información de perfil.
 - Recuperar contraseña.
- Gestión específica usuarios:
 - Registrarse en el sistema con sus datos personales.
 - Eliminar su cuenta.

Permitir a los farmacéuticos consultar a los usuarios del sistema.

Permitir a los usuarios consultar el catálogo de productos.

Permitir a los usuarios buscar productos mediante lenguaje natural.

- Realizar búsquedas semánticas utilizando consultas en lenguaje natural.
 - Búsquedas semánticas de productos del catálogo de productos.
- Realizar búsquedas semánticas de productos en función de diferentes filtros:
 - Filtrar por laboratorio.
 - Filtrar por categoría.
 - Filtrar por prescripción. (Con receta / Sin receta).
 - Filtrar por precio.
- Recibir resultados a partir de las búsquedas realizadas basados en modelos de embeddings.
- Valorar la búsqueda mediante una encuesta de satisfacción (Sí/No).

Permitir a los usuarios consultar recomendaciones personalizadas.

- Consultar productos recomendados en función de sus búsquedas previas y modelos de embeddings.
- Valorar las recomendaciones mediante una encuesta de satisfacción (Sí/No).

Permitir a los usuarios gestionar recetas privadas y recetas electrónicas.

- Cargar recetas en formato PDF.
- Consultar recetas asociadas al usuario.
- Eliminar una receta asociada al usuario.

Permitir a los usuarios añadir productos al carrito y consultarlo.

- Añadir los productos que se obtienen de la receta a la lista de encargos (carrito). El usuario puede elegir cuáles quiere añadir y cuáles no.
- Añadir productos sin receta del catálogo a la lista de encargos (carrito).
- Consultar el carrito.

Permitir a los usuarios encargar productos con receta y sin receta.

- Encargar los productos que están en la lista de encargos.
- Consultar sus encargos.

Permitir a los farmacéuticos gestionar los encargos.

- Consultar encargos realizados por los usuarios.
- Cambiar el estado del encargo: pendiente, listo para recoger, entregado.
- Informar por correo al usuario cuando los productos estén listos.

Permitir a los usuarios consultar sus encargos.

- Consultar sus encargos realizados y los estados de los encargos.

Permitir a los usuarios acceder al historial de búsquedas de productos.

- Consultar un registro de las búsquedas de productos realizadas.

Permitir a los usuarios resolver dudas frecuentes mediante un sistema de FAQ.

- Consultar las preguntas y respuestas más frecuentes.

Permitir a los usuarios tener asesoramiento farmacéutico.

- Consultar las dudas a los farmacéuticos en cualquier momento.

Permitir a los farmacéuticos asesorar a los usuarios.

- Responder consultas de los usuarios sobre sus dudas.

Permitir a los usuarios recibir notificaciones personalizadas.

- Enviar recordatorios por correo al usuario para renovar medicamentos de uso recurrente.
- Informar por correo a los usuarios sobre ofertas o promociones relevantes.

Permitir al administrador gestionar el catálogo de productos.

- Añadir nuevos productos al sistema con sus propiedades.
- Editar información de productos existentes.
- Eliminar productos obsoletos.

Permitir al administrador gestionar el sistema a través de un panel de control.

- Gestionar los usuarios registrados

- Eliminar usuarios.
- Consultar usuarios del sistema.
- Gestionar a los farmacéuticos registrados
 - Añadir farmacéutico
 - Eliminar farmacéutico
 - Consultar farmacéuticos del sistema.

Para definir los requisitos funcionales de este sistema, me basé en diversas fuentes de información con el objetivo de diseñar una plataforma que cubriera las necesidades tanto de los usuarios como de los farmacéuticos.

Una de mis principales referencias fue la experiencia de mi madre, que trabaja en una farmacia. Hablé con ella y sus compañeras para comprender mejor el funcionamiento del día a día en la farmacia, especialmente en lo que respecta a la gestión de recetas, encargos y atención a los clientes.

Además, analicé el funcionamiento de la página web de la farmacia donde trabaja mi madre, así como otras farmacias online reconocidas. Esto me permitió entender qué funcionalidades suelen ofrecer este tipo de plataformas y qué mejoras podrían implementarse.

3.2.2 Requisitos no funcionales

Los requisitos no funcionales describen las características del sistema que no están relacionadas directamente con sus funcionalidades, pero son esenciales para garantizar su calidad, usabilidad y rendimiento [13].

Para realizar la descripción de los requisitos no funcionales, usaremos la clasificación de Volere [14].

Tipo de requisito de Volere	Requisito de apariencia.
Descripción	El sistema web tiene que ser visualmente atractivo.
Justificación del requisito	Una apariencia atractiva mejora la experiencia del usuario, aumentando la confianza en el sistema y fomentando su uso recurrente.
Criterio de aceptación	Realizar una encuesta a 15 usuarios finales. La encuesta consistirá en calificar la claridad, diseño y coherencia visual del sistema web en una escala del 1 al 5. Al menos el 80% de los usuarios deberán puntuar con un 4 o 5.

Tipo de requisito de Volere	Requisito de facilidad de uso.
Descripción	El sistema web ha de ser fácil de usar.
Justificación del requisito	Una interfaz intuitiva asegura que los usuarios de diversos niveles de experiencia puedan interactuar con el sistema sin problemas, reduciendo la curva de aprendizaje.
Criterio de aceptación	Realizar una encuesta a 15 usuarios finales. La encuesta consistirá en calificar la facilidad de uso del sistema web en una escala del 1 al 5. Al menos el 80% de los usuarios deberán puntuar con un 4 o 5.

Tipo de requisito de Volere	Requisito de rendimiento al hacer una búsqueda.
Descripción	Las búsquedas deben ser rápidas.
Justificación del requisito	Un tiempo de respuesta rápido es crucial para mantener la satisfacción del usuario.
Criterio de aceptación	El 90% de las búsquedas se completan en menos de 500 milisegundos.

Tipo de requisito de Volere	Requisito de adaptabilidad.
Descripción	El sistema web debe ser responsivo, permitiendo una correcta visualización y funcionamiento tanto en pantallas de ordenadores como en dispositivos móviles.
Justificación del requisito	Los usuarios pueden acceder al sistema desde diversos dispositivos. Garantizar una experiencia de usuario óptima en cualquier pantalla mejora la accesibilidad y usabilidad del sistema.
Criterio de aceptación	Debe pasar pruebas en múltiples dispositivos, asegurando que todos los elementos sean accesibles y legibles sin necesidad de desplazamientos horizontales o zoom manual.

Tipo de requisito de Volere	Requisito de privacidad.
Descripción	El sistema debe proteger la información personal y médica.

Justificación del requisito	Los datos personales y médicos son muy sensibles, y cualquier vulnerabilidad podría poner en riesgo la confianza de los usuarios y exponer a la farmacia a sanciones legales.
Criterio de aceptación	Todos los datos sensibles estarán cifrados, se aplicarán políticas de acceso restringido y se cumplirá el RGPD [15].

3.3 Obstáculos y riesgos

- **Fecha de entrega:** El proyecto tiene una fecha de entrega determinada, así que cualquier imprevisto que surja se ha de solucionar rápidamente. Una buena planificación será clave para cumplir todos los objetivos del proyecto.
- **Bugs²:** Es muy probable que se detecten bugs en el sistema, y más si no se está muy familiarizado con la tecnología que se va a usar. Es un aspecto muy importante a tener en cuenta a la hora de planificar las tareas.
- **API³ de productos:** La API que se usa para obtener los productos para la farmacia es gratis y funciona correctamente. Puede pasar que en cualquier momento del desarrollo esta API deje de funcionar por un fallo o deje de ser gratis.
- **Cumplimiento de la normativa:** La venta y gestión de medicamentos está regulada, por lo que el sistema debe ajustarse a las leyes vigentes. Al manejar información de salud, se deben cumplir normativas para garantizar la privacidad de los usuarios.

² Los bugs son errores o fallos en un software que causan un comportamiento inesperado o incorrecto.

³ Una API es una interfaz que permite que dos aplicaciones se comuniquen e intercambien información entre sí.

4. Metodología

Elegir la metodología de trabajo adecuada es fundamental para el éxito del proyecto. Esta decisión determinará nuestra forma de organizar tareas y optimizar recursos.

4.1 Metodología de trabajo

La metodología de trabajo que se va a usar en este proyecto es **Scrum**. Scrum es una metodología ágil que organiza el trabajo en ciclos cortos llamados *sprints*, que suelen durar entre una y cuatro semanas. Durante cada *sprint*, el equipo se enfoca en completar un conjunto específico de tareas seleccionadas del *Product Backlog*, permitiendo entregas incrementales y frecuentes de valor [16].

Dentro del marco metodológico Scrum, el equipo de desarrollo participa en reuniones diarias conocidas como *Daily Scrum*. No obstante, dado que este proyecto es llevado a cabo por una sola persona, estas sesiones no se consideran necesarias. Por otro lado, al finalizar cada *sprint*, se contemplan eventos como la *Sprint Review* y la *Sprint Retrospective*, donde se revisan los progresos y se presentan los resultados obtenidos. En este trabajo, dichas funciones serán asumidas por las reuniones periódicas con la tutora, las cuales permitirán supervisar y asegurar el correcto desarrollo del proyecto.

En la Figura 2 se puede observar el proceso de Scrum.

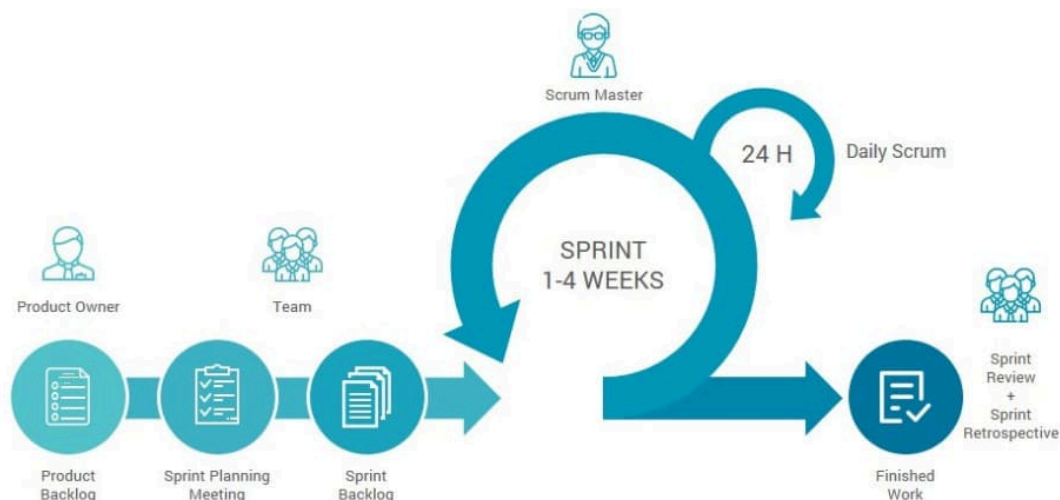


Figura 2: Scrum Process. [17]

La metodología *Scrum* se adecua muy bien a este proyecto debido a su enfoque en entregas incrementales y su capacidad para adaptarse a cambios frecuentes. La estructura de *sprints* cortos permite la retroalimentación continua, asegurando que el desarrollo del sistema web se alinee constantemente con las necesidades de los usuarios finales.

Los roles principales de *Scrum* son [16]:

- **Product Owner:** Define y prioriza las funcionalidades del producto, asegurando que el equipo trabaje en lo más valioso para el negocio. Para este proyecto yo cogeré el rol de *Product Owner*.
- **Scrum Master:** Facilita el uso de *Scrum*, asegura el correcto desarrollo y fomenta un entorno de trabajo eficiente. La directora del proyecto asume este rol.
- **Equipo de desarrollo:** Grupo multifuncional encargado de diseñar, desarrollar y entregar incrementos del producto en cada *sprint*. Para este proyecto yo seré el único desarrollador.

4.2 Herramientas de seguimiento

JIRA: Es una herramienta de gestión ágil utilizada para organizar el desarrollo en épicas, historias de usuario y tareas. Se usará para estructurar el trabajo en un tablero *Kanban*, donde las tareas avanzan por estados: *To Do*, *In Progress*, *Review* y *Done*. Esto facilita el seguimiento del proyecto y permite priorizar funcionalidades clave [18].

Github: Es una plataforma de control de versiones basada en *Git*, utilizada para almacenar y gestionar el código del proyecto. Se aplicará el flujo de trabajo *GitFlow* para organizar el desarrollo de manera estructurada. La rama *main* o *master* contendrá la versión estable del proyecto, mientras que *develop* servirá como base para integrar nuevas funcionalidades. Las nuevas características se desarrollarán en ramas *feature*, que una vez completadas, se fusionarán en *develop*. Para preparar una versión final, se creará una rama *release*, donde se realizarán pruebas antes de integrarla en *main*. Este sistema facilita la colaboración, evita conflictos y mejora la gestión del código a lo largo del proyecto [19]. En la Figura 3 se muestra el flujo de trabajo de *GitFlow*.



Figura 3: Flujo de trabajo GitFlow [20]

La validación de los resultados se llevará a cabo a través de reuniones periódicas de validación con los *stakeholders*, donde se revisará el avance del proyecto y se comprobará que las funcionalidades cumplen con los requisitos establecidos. Además, se realizarán tests automatizados y unitarios para asegurar que el sistema funciona correctamente y no presenta errores. Este enfoque permitirá detectar posibles fallos a tiempo, mejorar la calidad del código y garantizar que el desarrollo esté alineado con los objetivos del proyecto.

5. Descripción de las tareas

La planificación temporal es fundamental para garantizar el cumplimiento de los objetivos dentro del tiempo establecido. Permite organizar las tareas, priorizarlas y distribuir recursos de manera eficiente, evitando posibles retrasos.

El proyecto comenzó oficialmente el 19 de febrero de 2025, coincidiendo con el inicio del curso de gestión de proyectos. La defensa final está prevista para el 25 de junio del mismo año, marcando el cierre del trabajo. Este plazo comprende un total de 18 semanas, durante las cuales se ha planificado una dedicación aproximada de seis horas diarias de lunes a viernes. Con esta organización, se estima alcanzar unas 540 horas de trabajo, que es la carga horaria estipulada por la Facultad de Informática de Barcelona para este tipo de proyectos [21]. En caso de que surjan inconvenientes que requieran más tiempo del previsto, se contempla la opción de ampliar el horario utilizando fines de semana.

En la Tabla 2 se puede ver las diferentes etapas por cada fase del proyecto, con su fecha de inicio y finalización.

Fase	Etapas	Inicio	Finalización
Gestión de proyectos	Contextualización y alcance	17-02-2025	23-02-2025
	Planificación	24-02-2025	02-03-2025
	Gestión económica y sostenibilidad	03-03-2025	9-03-2025
	Documento final	10-03-2025	16-03-2025
Desarrollo	<i>Inception</i>	17-03-2024	23-03-2025
	<i>Sprint 1</i>	24-03-2025	06-04-2025
	<i>Sprint 2</i>	07-04-2025	27-04-2025
	<i>Sprint 3</i>	28-04-2025	18-05-2025
	<i>Sprint 4</i>	19-05-2025	01-06-2025
Finalización	Refinamiento	02-06-2025	08-06-2025
	Despliegue del sistema	09-06-2025	09-06-2025
	Acabar la memoria	10-06-2025	16-06-2025
	Preparación de la defensa	17-06-2025	24-06-2025

Tabla 2: Planificación de cada etapa. Elaboración propia.

5.1 Tareas

Por cada fase del proyecto, se definirán un conjunto de tareas asociadas a la fase. Por cada tarea, se pondrá una descripción, una estimación de horas a dedicar y las dependencias con otras tareas.

5.1.1 Gestión del proyecto

GP1: Contextualización y alcance (20 horas).

Redactar la primera entrega de GEP, donde hay que contextualizar el proyecto, limitar el alcance y definir la metodología, los objetivos, los requisitos y los riesgos.

Dependencias: -

GP2: Planificación (20 horas).

Redactar la segunda entrega de GEP, donde hay que definir las tareas, estimar su duración y la manera de gestionar los riesgos.

Dependencias: GP1.

GP3: Gestión económica y sostenibilidad (15 horas).

Redactar la tercera entrega de GEP, donde hay que calcular los presupuestos y hacer el informe de sostenibilidad.

Dependencias: GP2.

GP4: Integración documento final (5 horas).

Se integran las tres entregas de GEP con las correcciones propuestas por los profesores.

Dependencias: GP3.

GP5: Especificación del proyecto (25 horas).

Realizar los diagramas de casos de uso con sus descripciones. También el modelo conceptual y el modelo de comportamiento.

Dependencias: -.

GP6: Diseño de la arquitectura (30 horas).

Definir la arquitectura lógica y física del sistema, diseñar el esquema de la base de datos, desarrollar tanto la estructura interna como externa de la capa de presentación del *front-end* y especificar los patrones de diseño, junto con los diagramas de secuencia y los diagramas de clases del dominio en el *back-end*.

Dependencias: GP5.

5.1.2 Desarrollo

5.1.2.1 Inception

I1: Definición de las historias de usuario (5 horas).

Definir las historias de usuario en JIRA, con los criterios de aceptación y la dificultad.

Dependencias: GP5.

I2: Configuración del entorno de trabajo (5 horas).

Instalar todos los programas necesarios para empezar a desarrollar el proyecto.

Dependencias: GP6.

I3: Creación de la base del proyecto (15 horas).

Preparar la base de datos del proyecto para su uso en los siguientes *sprints*, así como implementar una primera versión de la página principal que sirva como base.

Dependencias: I2.

5.1.2.2 Sprint 1

D1: Autenticación (20 horas).

Implementar las funcionalidades de autenticación para el usuario, farmacéutico y administrador en el caso que corresponda: registro, iniciar sesión, cerrar sesión, eliminar cuenta y recuperar contraseña.

Dependencias: I3.

D2: Gestión de cuentas (10 horas).

Implementar las funcionalidades de gestionar cuentas para el administrador y farmacéutico en el caso que corresponda: añadir y eliminar usuarios, añadir y eliminar farmacéuticos y consultar usuarios y farmacéuticos.

Dependencias: D1.

D3: Consultar el catálogo de productos (15 horas).

Implementar la funcionalidad de mostrar el catálogo de productos al usuario. Hay que recoger los datos de los productos de la API CIMA [22].

Dependencias: I3.

D4: Filtrar productos en el catálogo (15 horas).

Implementar la funcionalidad de filtrar los productos en el catálogo en función de los filtros mencionados anteriormente.

Dependencias: D3.

RS1: Retrospectiva Sprint 1 (2 horas).

Hacer una reunión de seguimiento para evaluar si se han cumplido todos los objetivos de este *sprint*, detectar posibles errores para corregirlos en el siguiente *sprint* y actualizar el JIRA.

Dependencias: D1, D2, D3, D4.

DC1: Documentación Sprint 1 (5 horas).

Redactar la documentación de la memoria de la implantación de este *sprint*.

Dependencias: -

5.1.2.3 Sprint 2

D5: Ver y editar perfil (15 horas).

Implementar la pantalla del perfil y la modificación de los datos del perfil para usuarios, administradores y farmacéuticos.

Dependencias: D1.

D6: Búsqueda semántica de productos (30 horas).

Implementar la búsqueda semántica de productos para los usuarios.

Dependencias: D3.

D7: Historial de búsquedas de productos (15 horas).

Implementar la funcionalidad de guardar y mostrar el historial de búsqueda del usuario.

Dependencias: D6.

D8: Gestión de recetas (25 horas).

Implementar la carga, consulta y eliminación de recetas privadas y electrónicas.

Dependencias: D1.

RS2: Retrospectiva Sprint 2 (5 horas).

Hacer una reunión de seguimiento para evaluar si se han cumplido todos los objetivos de este *sprint*, detectar posibles errores para corregirlos en el siguiente *sprint* y actualizar el JIRA.

Dependencias: D5, D6, D7, D8.

DC2: Documentación Sprint 2 (5 horas).

Redactar la documentación de la memoria de la implantación de este *sprint*.

Dependencias: -

5.1.2.4 Sprint 3

D9: Recomendaciones personalizadas (15 horas).

Implementar la sección de recomendaciones personalizadas basadas en el historial de compras del usuario.

Dependencias: D7.

D10: Encargar productos (30 horas).

Implementar las funcionalidades de añadir productos con y sin receta al carrito, y de encargar esos productos por parte del usuario. También de consultar sus encargos.

Dependencias: D3, D8.

D11: Gestión de encargos (20 horas).

Implementar las funcionalidades de consultar encargos del usuario, cambiar estado del encargo e informar al usuario que el encargo está listo, por parte del farmacéutico.

Dependencias: D10.

D12: Valorar búsquedas y recomendaciones (15 horas).

Implementar la valoración de búsquedas y recomendaciones. Guardar los datos para hacer una analítica de los resultados.

Dependencias: D6, D9.

RS3: Retrospectiva Sprint 3 (5 horas).

Hacer una reunión de seguimiento para evaluar si se han cumplido todos los objetivos de este *sprint*, detectar posibles errores para corregirlos en el siguiente *sprint* y actualizar el JIRA.

Dependencias: D9, D10, D11, D12.

DC3: Documentación Sprint 3 (5 horas).

Redactar la documentación de la memoria de la implantación de este *sprint*.

Dependencias: -

5.1.2.5 Sprint 4

D13: Notificaciones personalizadas (10 horas).

Implementar el envío de recordatorios de medicamentos y de ofertas al usuario.

Dependencias: D1, D10.

D14: Asesoramiento farmacéutico (20 horas).

Implementar la funcionalidad de resolver dudas farmacológicas del usuario. El usuario hace una consulta por la web y un farmacéutico le responde (mensajería asíncrona).

Dependencias: D1.

D15: Gestión de productos (15 horas).

Implementar las funcionalidades de añadir, editar y eliminar productos por parte del administrador.

Dependencias: D1, D3.

D16: Consultar FAQ (5 horas).

Implementar la sección de preguntas y respuestas frecuentes.

Dependencias: D1.

RS4: Retrospectiva Sprint 4 (5 horas).

Hacer una reunión de seguimiento para evaluar si se han cumplido todos los objetivos de este *sprint*, detectar posibles errores para corregirlos en el siguiente *sprint* y actualizar el JIRA.

Dependencias: D13, D14, D15, D16.

DC4: Documentación Sprint 4 (5 horas).

Redactar la documentación de la memoria de la implantación de este *sprint*.

Dependencias: -

5.1.3 Finalización

D17: Refinamiento, testing y refactoring (30 horas).

Acabar de mejorar detalles finales del sistema, hacer las pruebas con los *testers* seleccionados y aplicar las correcciones recibidas.

Dependencias: Sprint 4.

D18: Despliegue del sistema (3 horas).

Desplegar el sistema *front-end* en Vercel [23] y el *back-end* el Railway [24] para que esté disponible para su uso.

Dependencias: D17.

DC5: Acabar la memoria (30 horas).

Acabar los últimos apartados de la memoria, así como hacer una revisión global de todos los apartados.

Dependencias: -

GP7: Preparación de la defensa (25 horas).

Realizar y preparar la presentación final del proyecto para defenderlo delante del tribunal.

Dependencias: DC5.

5.2 Recursos

Todos los proyectos necesitan una serie de recursos para hacer posible su funcionamiento. A continuación, en la Tabla 3 se muestran los recursos humanos y en la Tabla 4 los recursos materiales.

5.2.1 Recursos humanos

Identificador	Rol	Encargado
RHJP	Jefe de proyecto	Albert Comas Pacheco (autor)
RHDF	Desarrollador <i>Fullstack</i> ⁴	Albert Comas Pacheco (autor)
RHUX	Diseñador UX ⁵	Albert Comas Pacheco (autor)
RHTE	Testers	Yolanda Pacheco Culebras Laura Pedraza Iglesia
RHPO	<i>Product Owner</i>	Albert Comas Pacheco (autor)
RHSM	<i>Scrum Master</i>	Cristina Gómez Seoane
RHDT	Directora TFG	Cristina Gómez Seoane
RHTG	Tutor GEP	Fernando Barrabés Naval

Tabla 3: Recursos humanos del proyecto. Elaboración propia.

5.2.2 Recursos materiales

Identificador	Descripción
RMET	Espacio de trabajo físico: residencia del estudiante, silla, mesa, internet, etc.
RMPT	Portátil MacBook Air M3 (16GB RAM) [25]
RMRT	Ratón Trust Fyda [26]
RMVS	Visual Studio Code: IDE para desarrollar el proyecto [27].
RMPPM	Postman: herramienta para probar llamadas a APIs [28].
RMDB	MongoDB: sistema para la gestión de la base de datos [29].
RMVE	Vercel: para el despliegue del sistema <i>front-end</i> .
RMRW	Railway: para el despliegue del sistema <i>back-end</i> .
RMAWS	Amazon Web Services: para almacenar imágenes de usuarios [30].
RMGH	Github: controlador de versiones del código.
RMJI	JIRA: herramienta para aplicar la metodología <i>Scrum</i> .
RMLC	Lucidchart: herramienta para dibujar diagramas y esquemas [31].

⁴ Fullstack es un programador que sabe tanto de front-end como de back-end.

⁵ UX hace referencia a User eXperience. Engloba lo que influya en la experiencia del usuario en general.

RMWX	Wix: herramienta para hacer páginas web (en mi caso para hacer la representación visual de la página). [32]
RMGW	Google Workspace: herramientas de Google como Gmail y Meet para la comunicación con las partes interesadas de la UPC. También herramientas como Drive y Google Docs para la redacción y almacenamiento de la memoria.
RMAT	Atenea: recurso de la UPC para poner a disposición del estudiante la información necesaria para desarrollar correctamente el TFG.
RMCH	Google Chrome: para la búsqueda de información.
RMRE	Resend: herramienta para enviar correos electrónicos. [33]

Tabla 4: Recursos materiales del proyecto. Elaboración propia.

6. Estimaciones y diagrama de Gantt

6.1 Estimaciones

En la Tabla 5 se puede observar un resumen de las estimaciones en horas, las dependencias y los recursos de las tareas.

Id	Tarea	Horas	Dependencias	Recursos
GP	GESTIÓN DEL PROYECTO	115		
GP1	Contextualización y alcance	20	-	RHJP, RHPO, RHDT, RHTG, RMET, RMRT, RMPT, RMGW, RMAT, RMCH
GP2	Planificación	20	GP1	RHJP, RHPO, RHDT, RHTG, RMET, RMRT, RMPT, RMGW, RMAT, RMCH
GP3	Gestión económica y sostenibilidad	15	GP2	RHJP, RHTG, RMET, RMRT, RMPT, RMGW, RMAT, RMCH
GP4	Integración documento final	5	GP3	RHJP, RHDT, RHTG, RMET, RMRT, RMPT, RMGW, RMAT, RMCH
GP5	Especificación del proyecto	25	-	RHJP, RHDT, RMET, RMRT, RMPT, RMLC, RMGW, RMAT, RMCH
GP6	Diseño de la arquitectura	30	GP5	RHJP, RHDT, RMET, RMRT, RMPT, RMLC, RMWX, RMGW, RMAT, RMCH
D	DESARROLLO	337		
I	INCEPTION	25		
I1	Definición de las historias de usuario	5	GP5	RHPO, RHDF, RMET, RMRT, RMPT, RMJI, RMAT, RMCH
I2	Configuración del entorno de trabajo	5	GP6	RHDF, RMET, RMRT, RMPT, RMVS, RMAWS, RMGH, RMDB, RMCH
I3	Creación de la base del proyecto	15	I2	RHDF, RHUX, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
S1	SPRINT 1	67		
D1	Autenticación	20	I3	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH, RMRE
D2	Gestión de cuentas	10	D1	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
D3	Consultar el catálogo de productos	15	I3	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMPM, RMDB, RMCH
D4	Filtrar productos en el catálogo	15	D3	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMPM, RMDB, RMCH
RS1	Retrospectiva <i>Sprint</i> 1	2	D1,D2,D3,D4	RHDF, RHSM, RHPO, RMJI, RMET, RMRT, RMPT, RMGW
DC1	Documentación <i>Sprint</i> 1	5	-	RHJP, RHDT, RMET, RMRT, RMPT, RMGW
S2	SPRINT 2	95		

D5	Ver y editar perfil	15	D1	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH, RMAWS
D6	Búsqueda semántica de productos	30	D3	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
D7	Historial de búsquedas de productos	15	D6	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
D8	Gestión de recetas	25	D1	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH, RMRE
RS2	Retrospectiva <i>Sprint</i> 2	5	D5,D6,D7,D8	RHDF, RHSM, RHPO, RMJI, RMET, RMRT, RMPT, RMGW
DC2	Documentación <i>Sprint</i> 2	5	-	RHJP, RHDT, RMET, RMRT, RMPT, RMGW
S3	SPRINT 3	90		
D9	Recomendaciones personalizadas	15	D7	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
D10	Encargar productos	30	D3,D8	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH, RMRE
D11	Gestión de encargos	20	D10	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
D12	Valorar búsquedas y recomendaciones	15	D6,D9	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
RS3	Retrospectiva <i>Sprint</i> 3	5	D9,D10,D11,D12	RHDF, RHSM, RHPO, RMJI, RMET, RMRT, RMPT, RMGW
DC3	Documentación <i>Sprint</i> 3	5	-	RHJP, RHDT, RMET, RMRT, RMPT, RMGW
S4	SPRINT 4	60		
D13	Notificaciones personalizadas	10	D1,D10	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH, RMRE
D14	Asesoramiento farmacéutico	20	D1	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH, RMRE
D15	Gestión de productos	15	D1,D3	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
D16	Consultar FAQ	5	D1	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
RS4	Retrospectiva <i>Sprint</i> 4	5	D13,D14,D15,D16	RHDF, RHSM, RHPO, RMJI, RMET, RMRT, RMPT, RMGW
DC4	Documentación <i>Sprint</i> 4	5	-	RHJP, RHDT, RMET, RMRT, RMPT, RMGW
	FINALIZACIÓN	88		
D17	Refinamiento, <i>testing</i> y <i>refactoring</i>	30	S4	RHDF, RHUX, RHTE, RMET, RMRT, RMPT, RMVS, RMGH, RMDB, RMCH
D18	Despliegue del sistema	3	D17	RHDF, RMET, RMPT, RMVS, RMVE,RMRW
DC5	Acabar la memoria	30	-	RHJP, RHDT, RMET, RMRT, RMPT, RMGW
GP7	Preparación de la defensa	25	DC5	RHJP, RHDT, RMET, RMRT, RMPT, RMGW
TOTAL		540		

Tabla 5: Estimación de horas, dependencias y recursos de cada tarea. Elaboración propia.

6.2 Diagrama de Gantt

En la Figura 4 se puede observar el diagrama de *Gantt* de este proyecto, con un color diferente por cada fase y *sprint*. El primer factor para la organización cronológica de las tareas es que las más importantes para el proyecto se han programado primero, asegurando una base sólida desde el inicio y reduciendo riesgos a futuro. El segundo factor es la combinación estratégica de tareas en los *sprints*, equilibrando aquellas clave y de mayor dedicación con otras de menor complejidad. Esto permite un progreso continuo sin sobrecargar el desarrollo en ninguna fase.

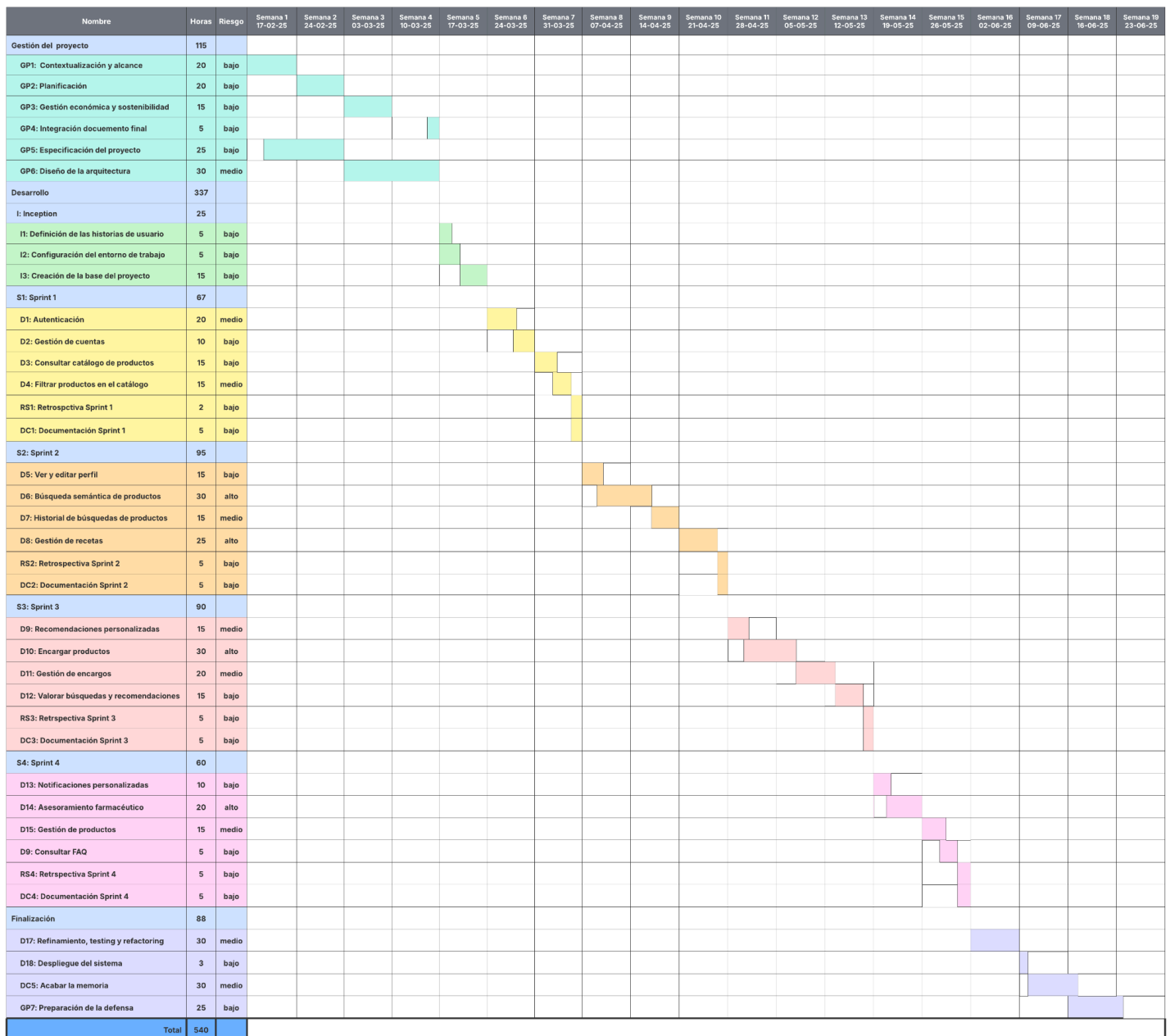


Figura 4: Diagrama de Gantt. Elaborado con Lucidchart.

7. Gestión de riesgos

En apartados anteriores, se hace mención a los posibles riesgos que pueden afectar a la planificación del proyecto. En la Tabla 6 se muestra por cada riesgo, su impacto y la probabilidad de que sucedan. Además, se explicará el plan de acción para abordarlos.

Riesgo	Impacto	Probabilidad
Fecha de entrega	Moderado - Alto	Moderada
Bugs	Moderado	Alta
API de productos	Bajo	Baja
Cumplimiento de la normativa	Alto	Moderada

Tabla 6: *Impacto y probabilidad de riesgos. Elaboración propia.*

Fecha de entrega

Si se detecta un posible retraso en la entrega, lo primero que se haría es intentar mitigar el retraso trabajando el fin de semana. Si la primera solución no funciona o es inviable, se replanificarán las tareas en JIRA priorizando las más críticas. Además, se reduciría el alcance de algunas funcionalidades no esenciales si fuera necesario.

Bugs

Es muy probable que se detecten *bugs* durante el desarrollo. Es por eso que en la estimación de tareas en las que estoy poco familiarizado he incluido el tiempo para resolver posibles fallos. En tal caso, se implementará un sistema de *hotfix* con ramas específicas en GitHub para corregirlos rápidamente. Además, también se ha incluido el tiempo necesario para hacer los tests en la estimación de cada tarea.

API de productos

Si la API que se usa para obtener los productos (CIMA) fallase, se cuenta con una API alternativa llamada Medipim [34]. Esto garantizará la disponibilidad de información sobre productos sin afectar la operatividad del sistema. En ese caso, se tendría que reformular los filtros de búsqueda de productos a los compatibles con la nueva API.

Cumplimiento de la normativa

Si se identifica un incumplimiento legal en el sistema, se procederá a una actualización inmediata de la funcionalidad afectada. Se consultará con expertos en regulación farmacéutica y, si es necesario, se deshabilitarán temporalmente las funciones no conformes hasta que se realicen las correcciones.

8. Gestión económica

En este apartado se identifican los costes asociados a este proyecto, así como su estimación. Además, se detalla el control de gestión que se tendrá en cuenta para garantizar el seguimiento eficaz del proyecto.

8.1 Identificación de los costes

Es fundamental evaluar la viabilidad económica de cualquier proyecto y justificar su coste. En este apartado, se calculará el presupuesto considerando un escenario fuera del ámbito universitario. Para ello, se simulará una situación realista, asumiendo los roles habituales en un proyecto estándar.

8.1.1 Costes humanos

Los costes humanos corresponden a la remuneración del equipo involucrado en el proyecto, que varía según el rol de cada miembro. Para calcular el presupuesto destinado a estos salarios, se multiplicará el sueldo medio por hora de cada rol por el número de horas dedicadas al proyecto.

En la Tabla 7, se muestran los sueldos brutos por hora de los diferentes roles con y sin tener en cuenta la seguridad social. Los sueldos anuales se han obtenido de la página web Glassdoor [35], a partir de la media de los salarios en esos puestos en Barcelona y se han dividido entre 2080: 40 horas a la semana multiplicada por 52 semanas al año.

Identificador	Rol	Sueldo bruto (€/h)	Sueldo bruto + seguridad social (30%) (€/h)
RHJP	Jefe de proyecto	22,11	28,74
RHDF	Desarrollador <i>Fullstack</i>	15,86	20,62
RHUX	Diseñador UX	14,56	18,93
RHTE	Testers	12,67	16,47
RHPO	Product Owner	23,07	30,00
RHSM	Scrum Master	21,63	28,12

Tabla 7: Sueldos por rol y hora. Elaboración propia.

Una vez se ha calculado el coste de cada rol por hora, podemos calcular el precio por cada tarea. En la Tabla 8 se puede observar las horas dedicadas por cada tarea y su coste asociado.

Id	Tarea	Total Horas	Horas						Coste (€)
			JP	DF	UX	TE	PO	SM	
	GESTIÓN DEL PROYECTO								3.330,30
GP1	Contextualización y alcance	20	10				10		587,4
GP2	Planificación	20	10				10		587,4
GP3	Gestión económica y sostenibilidad	15	15						431,1
GP4	Integración documento final	5	5						143,7
GP5	Especificación del proyecto	25	25						718,5
GP6	Diseño de la arquitectura	30	30						862,2
D	DESARROLLO								7.010,74
I	INCEPTION	25							529,19
I1	Definición de las historias de usuario	5		3			2		121,86
I2	Configuración del entorno de trabajo	5		5					103,1
I3	Creación de la base del proyecto	15		12	3				304,23
S1	SPRINT 1	67							1.390,32
D1	Autenticación	20		15	4	1			401,49
D2	Gestión de cuentas	10		6	3	1			196,98
D3	Consultar el catálogo de productos	15		10	4	1			298,39
D4	Filtrar productos en el catálogo	15		11	3	1			300,08
RS1	Retrospectiva <i>Sprint</i> 1	2		1			0,5	0,5	49,68
DC1	Documentación <i>Sprint</i> 1	5	5						143,7
S2	SPRINT 2	95							1.967,81
D5	Ver y editar perfil	15		10	4	1			298,39
D6	Búsqueda semántica de productos	30		22	6	2			600,16
D7	Historial de búsquedas de productos	15		11	3	1			300,08
D8	Gestión de recetas	25		17	6	2			497,06
RS2	Retrospectiva <i>Sprint</i> 2	5		2			1,5	1,5	128,42
DC2	Documentación <i>Sprint</i> 2	5	5						143,7
S3	SPRINT 3	90							1.863,02
D9	Recomendaciones personalizadas	15		10	4	1			298,39
D10	Encargar productos	30		22	6	2			600,16
D11	Gestión de encargos	20		13	5	2			395,65
D12	Valorar búsquedas y recomendaciones	15		9	5	1			296,7

RS3	Retrospectiva Sprint 3	5		2			1,5	1,5	128,42
DC3	Documentación Sprint 3	5	5						143,7
S4	SPRINT 4	60							1.260,40
D13	Notificaciones personalizadas	10		8	1	1			200,36
D14	Asesoramiento farmacéutico	20		12	6	2			393,96
D15	Gestión de productos	15		10	4	1			298,39
D16	Consultar FAQ	5		2	2	1			95,57
RS4	Retrospectiva Sprint 4	5		2			1,5	1,5	128,42
DC4	Documentación Sprint 4	5	5						143,7
	FINALIZACIÓN	88							2.253,12
D17	Refinamiento, testing y refactoring	30		10	10	6	2	2	610,56
D18	Despliegue del sistema	3		3					61,86
DC5	Acabar la memoria	30	30						862,2
GP7	Preparación de la defensa	25	25						718,5
	TOTAL	540	170	228	79	27	29	7	12.594,16

Tabla 8: Costes humanos por cada tarea. Elaboración propia.

8.1.2 Costes materiales

Para el cálculo de la amortización en los costes materiales, voy a usar la siguiente fórmula:

$$\frac{\text{coste (€)}}{\text{vida útil (años)} * \text{días laborables anuales} * \text{dedicación diaria (horas)}} * \text{uso en el proyecto (horas)}$$

La vida útil del hardware son 4 años y la del software son 3 [36], 2025 tiene 250 días laborables [37], la dedicación diaria son 6 horas y la duración del proyecto son 540 horas.

8.1.2.1 Hardware

Para poder llevar a cabo las tareas se necesita un hardware básico, compuesto únicamente por un portátil MacBook y un ratón Trust.

En la Tabla 9, se muestran los costes de hardware y su amortización.

Hardware	Coste (€)	Amortización (€)
Portátil Macbook Air M3 (16GB / 256GB)	1299	116,91
Ratón Trust Fyda	17,99	1,62
Total	1316,99	118,53

Tabla 9: Costes y amortización del hardware. Elaboración propia.

8.1.2.2 Software

En este proyecto no se prevé que haya costes de software, a parte del despliegue en Railway, ya que se usarán las versiones gratuitas de las herramientas. En la Tabla 10 se muestran los costes de software y su amortización.

Software	Coste (€)	Amortización (€)
Visual Studio Code	Gratuito	0
Postman	Gratuito	0
MongoDB	Gratuito	0
Vercel	Gratuito	0
Railway	5	0,6
Amazon Web Services	Gratuito	0
Github	Gratuito	0
JIRA	Gratuito	0
Lucidchart	Gratuito	0
Google Workspace	Gratuito	0
Atenea	Gratuito	0
Google Chrome	Gratuito	0
Total	5	0,6

Tabla 10: Costes y amortización del software. Elaboración propia.

8.1.2.3 Indirectos

Se consideran dentro de los costes indirectos los asociados a contar con un espacio físico para trabajar. Aunque el desarrollo se ha llevado a cabo desde la residencia del estudiante, se plantea como si se hubiese realizado en un *coworking*⁶ de Barcelona. Dicho espacio se localizó a través del portal *CoworkingSpain* [38]. Esta oficina ofrece acceso continuo, sala de reuniones, impresora, wifi rápido, cocina, climatización y capacidad para cinco personas.

⁶ Coworking es un espacio de trabajo compartido donde profesionales de diferentes sectores trabajan de forma independiente pero en un entorno colaborativo.

En la Tabla 11, se muestra el coste de este entorno de trabajo.

Sitio	Coste mensual con IVA (€)	Meses	Total (€)
CoworkingSpain	260,15	4	1.040,6

Tabla 11: Coste de entorno de trabajo. Elaboración propia.

En la Tabla 12 se muestra un resumen de los costes materiales.

Tipo de coste	Coste (€)
Hardware	118,53
Software	0,6
Indirectos	1.040,6
Total	1.159,73

Tabla 12: Costes materiales. Elaboración propia.

8.2 Estimación de los costes

8.2.1 Contingencias

Para la estimación de los costes totales, primero se ha de tener en cuenta la contingencia. La contingencia añade un 10% de más al coste total del proyecto. En la Tabla 13 se muestran los nuevos costes humanos y materiales.

Tipo de coste	Coste (€)	Contingencia (%)	Contingencia (€)
Costes humanos	12594,16	10	1.259,42
Costes materiales	1159,73	10	115,97
Total	13753,89	10	1.375,39

Tabla 13: Costes con contingencias. Elaboración propia.

8.2.1 Imprevistos

Por último, se ha de tener en cuenta los costes que suponen tener imprevistos durante el proyecto. En la Tabla 14 se puede observar los costes de los posibles imprevistos.

Imprevisto	Probabilidad	Tiempo (horas / rol)	Coste (€)	Coste x probabilidad (€)
Retardo en fecha de entrega	30%	5 Desarrollador 1 Product Owner	133,1	39,93
Aparición de <i>bugs</i>	70%	10 Desarrollador	103,1	72,17
Fallo en la API de productos	10%	10 Desarrollador	103,1	10,31
Incumplimiento de la normativa	20%	3 Desarrollador 1 Product Owner	91,86	18,37
Total				140,78

Tabla 14: Costes de imprevistos. Elaboración propia.

8.2.1 Coste total

Como resumen de los diferentes costes presentados en este apartado, en la Tabla 15 se puede ver como queda el presupuesto final del proyecto.

Tipo de coste	Coste (€)
Costes humanos	12.594,16
Costes materiales	1.159,13
Contingencias	1.375,39
Imprevistos	140,78
Total	15.269,46

Tabla 15: Presupuesto final. Elaboración propia.

8.3 Control de gestión

Para garantizar un seguimiento eficaz del proyecto, es necesario contrastar las estimaciones iniciales con los datos reales recopilados a lo largo del desarrollo. Esto implica registrar el tiempo real empleado en cada tarea. La evaluación de las desviaciones del proyecto se realizará a través de las siguientes métricas:

- **Desviación de horas por tarea:**
horas estimadas - horas reales
- **Desviación de los costes humanos por tarea:**
(coste estimado - coste real) * horas reales

- **Desviación total de los costes humanos:**
coste humano estimado - coste humano real
- **Desviación de costes de hardware:**
coste hardware estimado - coste hardware real
- **Desviación de costes de software:**
coste software estimado - coste software real
- **Desviación de costes indirectos:**
coste indirecto estimado - coste indirecto real
- **Desviación de costes de imprevistos:**
costes imprevistos estimados - costes imprevistos reales
- **Desviación total de horas:**
horas totales estimadas - horas totales reales
- **Desviación total de costes:**
costes totales estimados - costes totales reales

Al final del proyecto se hará una valoración de la gestión realizada en el proyecto.

9. Especificación

En la sección de [alcance](#), se han identificado los requisitos que el sistema debe cumplir para satisfacer las necesidades de los usuarios. En este capítulo, se especificarán detalladamente estos requisitos funcionales. A continuación, se presentan estas funcionalidades con mayor profundidad, describiendo claramente el comportamiento del sistema desde la perspectiva de los actores y su contexto.

9.1 Diagrama de casos de uso

En esta parte se incluyen los diagramas de casos de uso que representan cómo interactúan los distintos actores con el sistema. Estos esquemas ofrecen una representación clara de las funciones, facilitando la comprensión del comportamiento del sistema. Cada caso de uso se centra en un conjunto de tareas que los actores pueden ejecutar, organizadas en categorías que reflejan áreas clave del sistema.

Usuario, administrador y farmacéutico: Gestión de cuentas

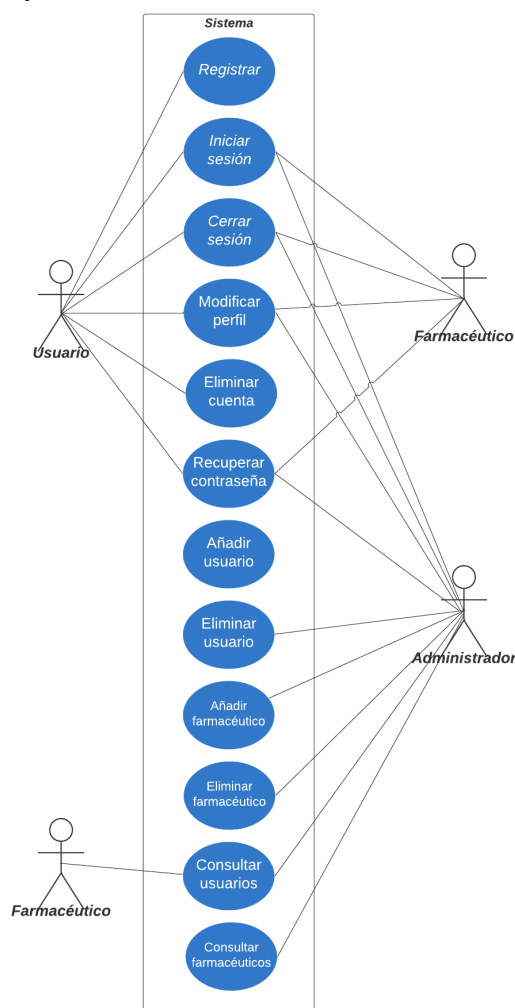


Figura 5: Diagrama de casos de uso de gestión de cuentas. Elaborado con Lucidchart.

Usuario: Búsqueda y recomendaciones

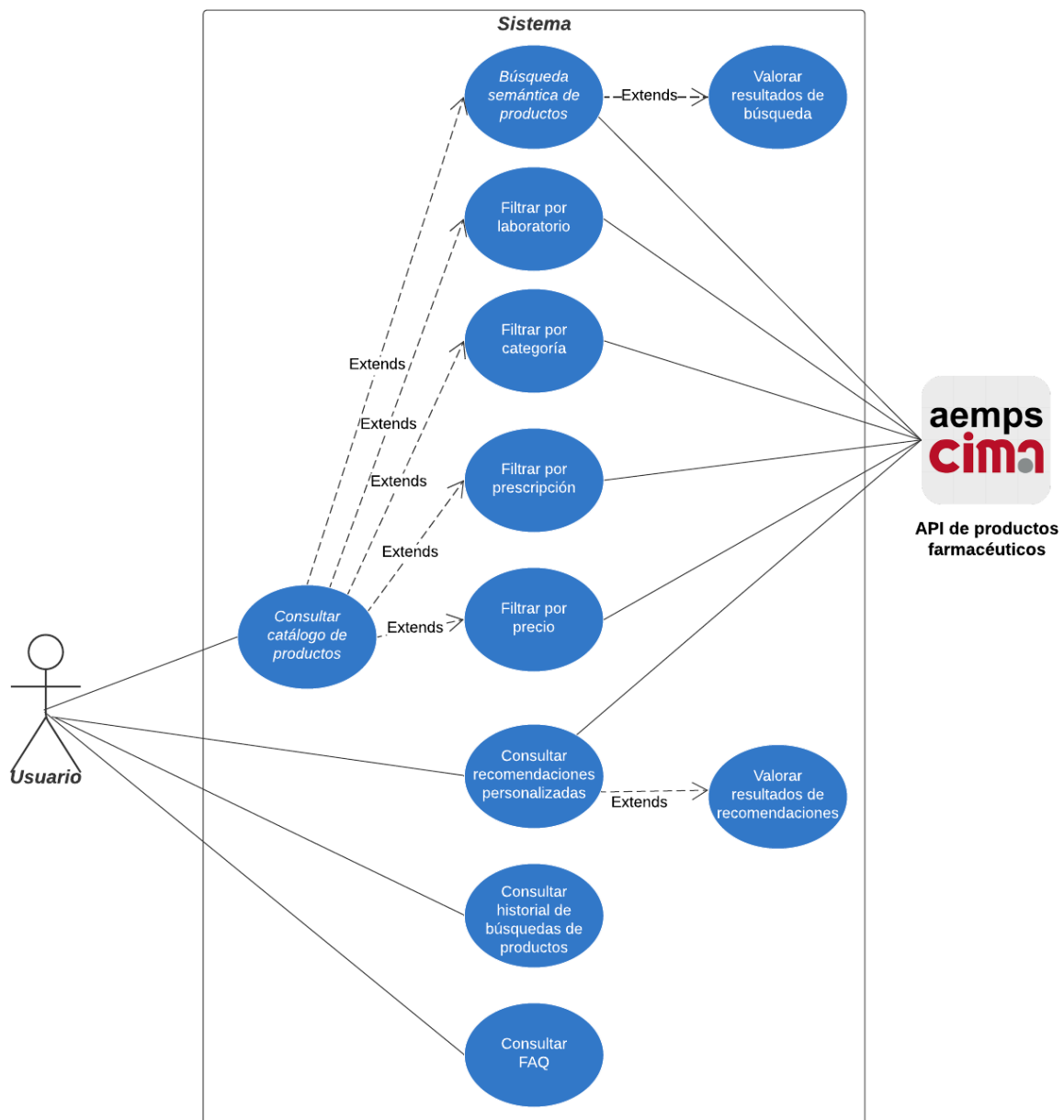


Figura 6: Diagrama de casos de uso de búsqueda y recomendaciones. Elaborado con Lucidchart.

Usuario: Gestión de recetas

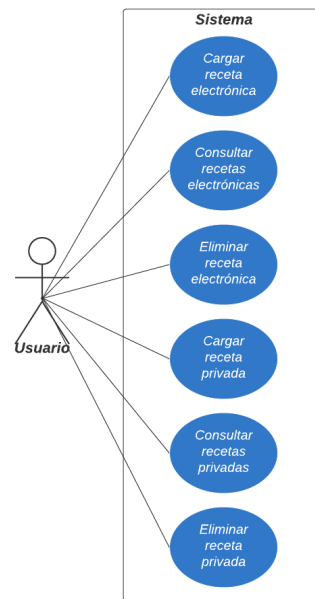


Figura 7: Diagrama de casos de uso de gestión de recetas. Elaborado con Lucidchart.

Timer: Notificaciones personalizadas

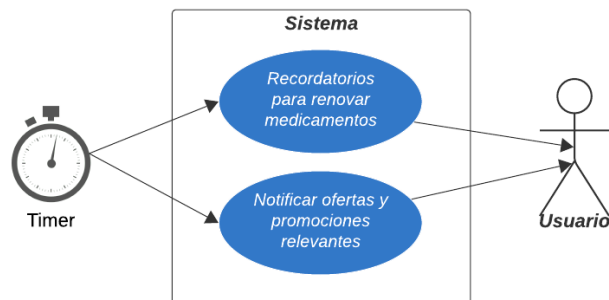


Figura 8: Diagrama de casos de uso de notificaciones personalizadas. Elaborado con Lucidchart.

Usuario y farmacéutico: Asesoramiento farmacéutico

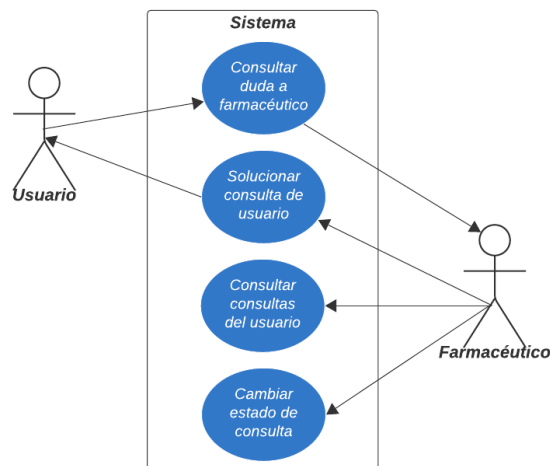


Figura 9: Diagrama de casos de uso de asesoramiento farmacéutico. Elaborado con Lucidchart.

Usuario y farmacéutico: Gestión de encargos

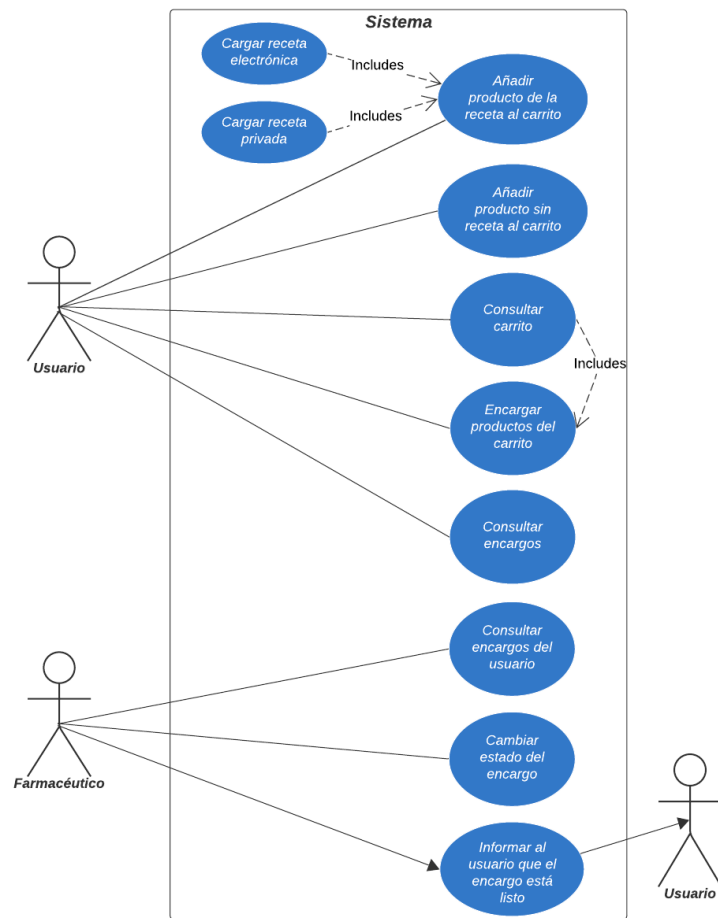


Figura 10: Diagrama de casos de uso de gestión de encargos. Elaborado con Lucidchart.

Administrador: Gestión de productos

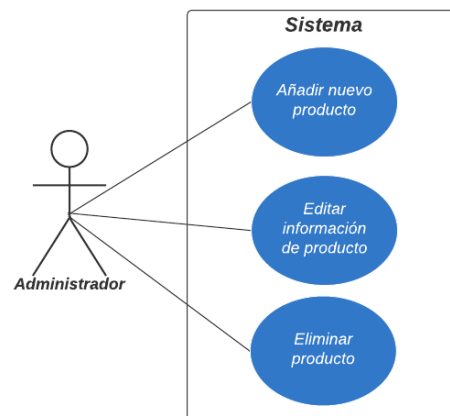


Figura 11: Diagrama de casos de uso de gestión de productos. Elaborado con Lucidchart.

9.2 Descripción de casos de uso

Una vez identificados los casos de uso del sistema, se describirán en profundidad únicamente los más relevantes. Para lograrlo, se estructurará toda la información necesaria en una tabla donde se indicará el nombre del caso de uso, el actor que lo interpreta, las precondiciones que deben darse (en caso de que las haya), el suceso que activa el caso de uso, el escenario principal de éxito y sus extensiones. Una extensión describe una serie de acciones que suceden en caso de que alguna acción del escenario principal no haya funcionado correctamente.

Las descripciones del resto de casos de uso pueden consultarse en el [Anexo 1](#).

Usuario, farmacéutico y administrador: Gestión de cuentas

Caso de uso	#4 Modificar perfil	Actor principal	Usuario, Farmacéutico y Administrador
Precondiciones	El actor está dado de alta en el sistema.		
Disparador	El actor quiere modificar la información de su cuenta.		
Escenario principal de uso			
1. El actor indica que quiere modificar los datos de su perfil e introduce los nuevos datos. 2. El sistema valida los nuevos datos. 3. El sistema actualiza los datos correspondientes a la cuenta en la base de datos. 4. El sistema indica al actor que los datos se han actualizado correctamente.			
Extensiones			
3a. El sistema determina que algún dato no es correcto. 3a1. El sistema avisa al actor qué dato no es correcto y permite al actor volver a intentarlo.			

Caso de uso	#6 Recuperar contraseña	Actor principal	Usuario, Farmacéutico y Administrador
Precondiciones	-		
Disparador	El actor quiere recuperar la contraseña de una cuenta.		

Escenario principal de uso
1. El actor indica que desea recuperar la contraseña de una cuenta e ingresa el correo. 2. El sistema valida que el correo existe. 3. El sistema envía un correo con un código al correo asociado a la cuenta. 4. El actor ingresa al sistema el código recibido, junto con la nueva contraseña, y solicita que se cambie. 5. El sistema valida el código y la nueva contraseña. 6. El sistema almacena la nueva contraseña en la base de datos. 7. El sistema redirige al actor a iniciar sesión.
Extensiones
3a. El sistema determina que el correo no existe. 3a.1. El sistema avisa al actor que el correo no existe y le permite intentarlo nuevamente. 5a. El sistema determina que el código introducido por el actor no es válido. 5a.1. El sistema avisa al actor que el código no es válido y le permite intentarlo nuevamente. 5b. El sistema determina que la nueva contraseña introducida por el actor no es válida. 5b.1. El sistema avisa al actor que la nueva contraseña no es válida y le permite intentarlo nuevamente.

Usuario: Búsqueda y recomendaciones

Caso de uso	#13 Búsqueda semántica de productos	Actor principal	Usuario
Precondiciones	El usuario tiene acceso al catálogo de productos en el sistema.		
Disparador	El usuario quiere hacer una búsqueda semántica de productos del catálogo.		
Escenario principal de uso			
1. El usuario hace una búsqueda semántica de productos en el sistema. 2. El sistema muestra al usuario los productos relacionados con la búsqueda indicada. 3. Por cada producto que se muestre se verá la foto, el nombre y el precio.			

Caso de uso	#14 Filtrar por laboratorio	Actor principal	Usuario
Precondiciones	El usuario tiene acceso al catálogo de productos en el sistema.		
Disparador	El usuario quiere filtrar los productos del catálogo por laboratorio.		
Escenario principal de uso			
1. El usuario hace una búsqueda semántica de productos seleccionando laboratorios. 2. El sistema muestra los productos relacionados con la indicación del usuario.			

Caso de uso	#18 Consultar recomendaciones personalizadas	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión.		
Disparador	El usuario quiere ver las recomendaciones sugeridas por el sistema en función de su historial de búsqueda.		
Escenario principal de uso			
1. El usuario indica que quiere ver las recomendaciones de productos que le sugiere el sistema. 2. El sistema muestra las recomendaciones al usuario.			

Caso de uso	#21 Valorar resultados de búsqueda	Actor principal	Usuario
Precondiciones	El usuario ha realizado al menos una búsqueda en el sistema. El sistema ha mostrado resultados para la búsqueda realizada.		
Disparador	El usuario quiere valorar la relevancia de los resultados de la búsqueda.		
Escenario principal de uso			
1. El usuario indica su valoración después de realizar una búsqueda. 2. El sistema muestra al usuario la valoración realizada.			

Usuario: Gestión de recetas

Caso de uso	#23 Cargar receta electrónica	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión.		
Disparador	El usuario quiere subir una receta electrónica.		
Escenario principal de uso			
1. El usuario indica que quiere cargar una receta electrónica, selecciona el archivo y lo confirma. 2. El sistema confirma que es una receta electrónica de la Seguridad Social y guarda la receta asociada al usuario en la base de datos. 3. El sistema notifica al usuario que la receta se ha cargado con éxito. 4. El sistema muestra al usuario los productos de la receta.			
Extensiones			
2a. El sistema determina que el archivo no es compatible. 2a1. El sistema indica al usuario que cargue el archivo en formato PDF y permite al usuario volver a intentarlo. 2b. El sistema determina que el archivo está vacío o es ilegible. 2b1. El sistema indica al usuario que el archivo no contiene información válida y permite al usuario volver a intentarlo. 2c. El sistema determina que el archivo no es una receta de la Seguridad Social. 2c1. El sistema indica al usuario que el archivo no es una receta de la Seguridad Social y permite al usuario volver a intentarlo. 4a. El sistema no puede encontrar un producto de la receta. 4a1. El sistema indica al usuario que no ha podido encontrar un producto de la receta.			

Caso de uso	#26 Cargar receta privada	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión.		
Disparador	El usuario quiere subir una receta privada.		
Escenario principal de uso			
1. El usuario indica que quiere cargar una receta privada, selecciona el archivo y lo confirma. 2. El sistema guarda la receta asociada al usuario en la base de datos. 3. El sistema notifica al usuario que la receta se ha cargado con éxito. 4. El sistema muestra al usuario los productos de la receta.			
Extensiones			

2a. El sistema determina que el archivo no es compatible.
 2a1. El sistema indica al usuario que cargue el archivo en formato PDF y permite al usuario volver a intentarlo.
 2b. El sistema determina que el archivo está vacío o es ilegible.
 2b1. El sistema indica al usuario que el archivo no contiene información válida y permite al usuario volver a intentarlo.
 4a. El sistema no puede encontrar un producto de la receta.
 4a1. El sistema indica al usuario que no ha podido encontrar un producto de la receta.

Timer: Notificaciones personalizadas

Caso de uso	#29 Recordatorios de medicamentos	Actor principal	Timer
Precondiciones	El usuario ha iniciado sesión. El usuario tiene configurado al menos un medicamento con recordatorios activos en su perfil.		
Disparador	El sistema detecta que se aproxima la fecha configurada para recibir recordatorios.		
Escenario principal de uso			
<			

Caso de uso	#30 Notificar ofertas y promociones relevantes	Actor principal	Timer
Precondiciones	El usuario ha iniciado sesión o tiene una cuenta activa. El usuario ha aceptado recibir notificaciones de ofertas y promociones.		
Disparador	El sistema detecta una nueva oferta o promoción que coincide con los intereses del usuario.		
Escenario principal de uso			

1. El Timer del sistema verifica periódicamente las ofertas disponibles.
2. El sistema detecta que una oferta o promoción es relevante para el usuario.
3. El sistema manda un correo electrónico al usuario indicando la oferta o promoción.
4. El usuario recibe la notificación y, si está interesado, accede al sistema para obtener más detalles o aprovechar la promoción.

Usuario y farmacéutico: Asesoramiento farmacéutico

Caso de uso	#31 Consultar duda a farmacéutico	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión. Existe al menos un farmacéutico en el sistema.		
Disparador	El usuario quiere consultar una duda a un farmacéutico.		
Escenario principal de uso			
1. El usuario indica que quiere consultar una duda a un farmacéutico. 2. El usuario rellena el formulario para hacer la consulta. 3. El sistema guarda el mensaje y notifica al farmacéutico de la nueva consulta. 4. El farmacéutico accede al sistema y responde. 5. El usuario recibe una notificación cuando el farmacéutico ha respondido y puede ver la respuesta en el sistema.			

Caso de uso	#32 Solucionar consulta de usuario	Actor principal	Farmacéutico
Precondiciones	El farmacéutico ha iniciado sesión. El usuario existe en el sistema. La consulta existe en el sistema.		
Disparador	El farmacéutico quiere solucionar una consulta del usuario.		
Escenario principal de uso			
1. El farmacéutico accede al sistema y responde al usuario. 2. El sistema guarda la respuesta y notifica al usuario de la actualización. 3. El usuario accede al sistema y visualiza la respuesta del farmacéutico. 4. La consulta queda marcada como respondida.			

Caso de uso	#34 Cambiar estado de consulta	Actor principal	Farmacéutico
Precondiciones	El farmacéutico ha iniciado sesión. El usuario existe en el sistema. La consulta existe en el sistema.		
Disparador	El farmacéutico quiere cambiar el estado de la consulta.		
Escenario principal de uso			
1. El farmacéutico indica que quiere cambiar el estado de la consulta del usuario. 2. El sistema actualiza el estado de la consulta del usuario en la base de datos. 3. El sistema muestra al farmacéutico y al usuario el nuevo estado de la consulta del usuario.			

Usuario y farmacéutico: Gestión de Encargos

Caso de uso	#35 Añadir producto de la receta al carrito	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión. El usuario ha cargado una receta en el sistema. La receta contiene productos disponibles para su encargo.		
Disparador	El usuario quiere añadir un producto de la receta al carrito.		
Escenario principal de uso			
1. El usuario indica que quiere añadir un producto de la receta al carrito. 2. El sistema añade el producto disponible al carrito y actualiza el número de productos en el carrito.			

Caso de uso	#36 Añadir producto sin receta al carrito	Actor principal	Usuario
Precondiciones	-		
Disparador	El usuario quiere añadir un producto sin receta al carrito.		
Escenario principal de uso			
1. El usuario indica que quiere añadir un producto sin receta del catálogo al carrito. 2. El sistema añade el producto disponible al carrito y actualiza el número de productos en el carrito.			

Caso de uso	#38 Encargar productos del carrito	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión. El carrito contiene al menos un producto.		
Disparador	El usuario quiere encargar los productos del carrito.		
Escenario principal de uso			
1. El usuario indica que quiere encargar los productos del carrito. 2. El sistema genera el encargo con los productos seleccionados. 3. El sistema informa al usuario de que el encargo ha sido registrado. 4. Si el encargo contiene productos con receta, el sistema informa al usuario de que deberá presentar la tarjeta sanitaria al recogerlos en la farmacia.			

Caso de uso	#41 Cambiar estado del encargo	Actor principal	Farmacéutico
Precondiciones	El farmacéutico ha iniciado sesión. Existe al menos un encargo hecho por un usuario.		
Disparador	El farmacéutico quiere cambiar el estado del encargo del usuario.		
Escenario principal de uso			
1. El farmacéutico indica que quiere cambiar el estado del encargo del usuario. 2. El sistema actualiza el estado del encargo del usuario en la base de datos. 3. El sistema muestra al farmacéutico y al usuario el nuevo estado del encargo del usuario.			

Caso de uso	#42 Informar al usuario que el encargo está listo	Actor principal	Farmacéutico
Precondiciones	El farmacéutico ha iniciado sesión.		
Disparador	El farmacéutico actualiza el estado del encargo a “listo para recoger”.		
Escenario principal de uso			
1. El farmacéutico actualiza el estado del encargo a “listo para recoger”. 2. El sistema actualiza el estado del encargo en la base de datos. 3. El sistema muestra al farmacéutico y al usuario el nuevo estado del encargo.			

- | |
|---|
| 4. El sistema informa por correo al usuario de que el encargo está listo para ser recogido. |
|---|

Administrador: Gestión de productos

Caso de uso	#44 Editar información de producto	Actor principal	Administrador
Precondiciones	El administrador ha iniciado sesión. Existe al menos 1 producto en el sistema.		
Disparador	El administrador quiere editar la información de un producto.		
Escenario principal de uso			
1. El administrador indica que quiere editar la información de un producto e introduce los nuevos datos. 2. El sistema valida los datos. 3. El sistema actualiza los datos correspondientes en la base de datos. 4. El sistema indica al administrador que los datos se han actualizado correctamente.			
Extensiones			
3a. El sistema determina que algún dato no es correcto. 3a1. El sistema avisa al administrador qué dato no es correcto y permite al administrador volver a intentarlo.			

9.3 Modelo conceptual

Seguidamente, se establecerá el modelo conceptual de datos, cuya finalidad es reflejar los elementos del dominio y las conexiones entre ellos. Este modelo permite obtener una representación visual del sistema enfocada en los datos y componentes que lo integran.

9.3.1 Esquema conceptual de los datos

En la Figura 12, se presenta la estructura de datos del sistema, donde se destacan las clases de objetos, sus atributos y las relaciones entre ellas.

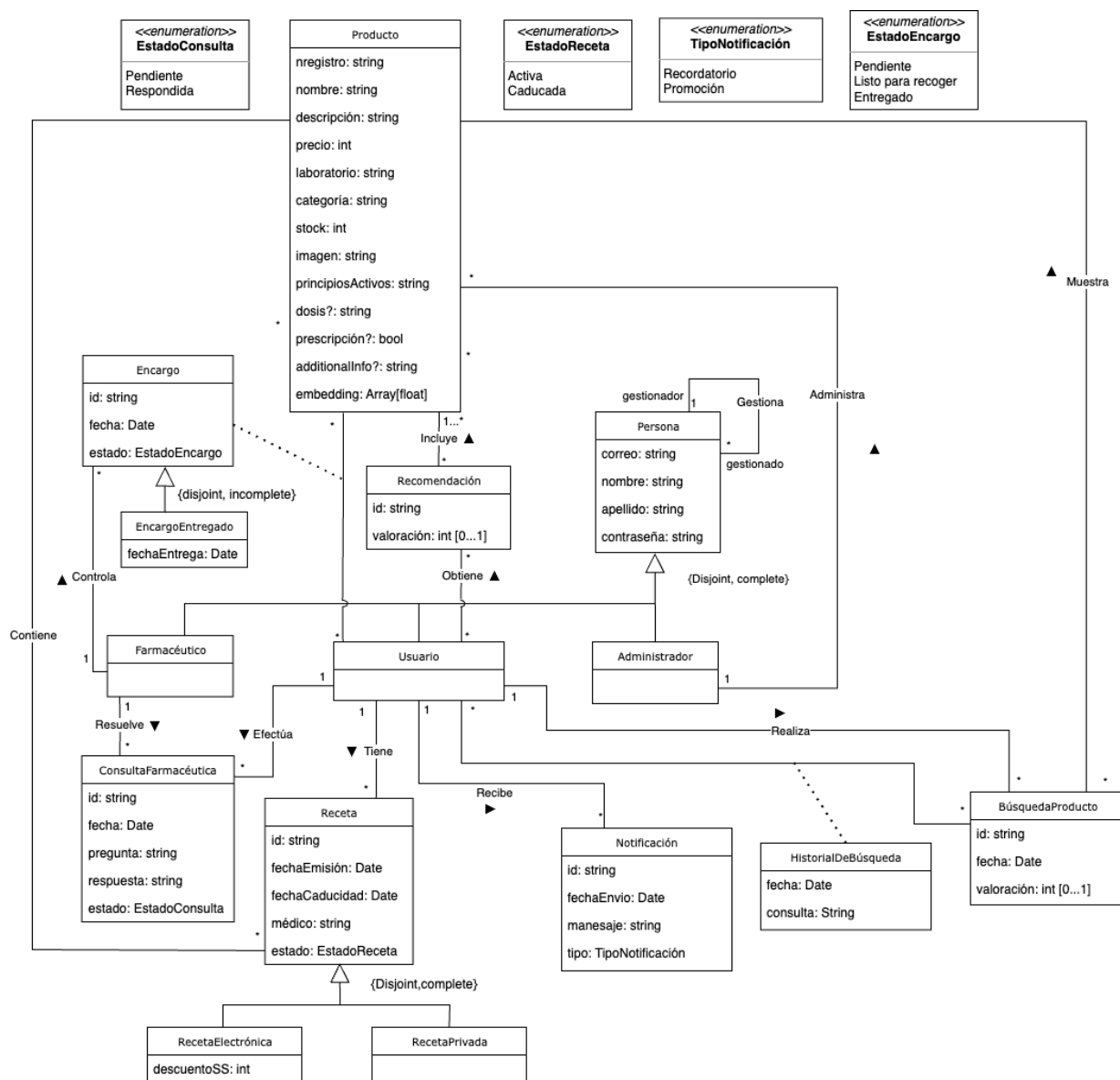


Figura 12: Esquema conceptual de los datos. Elaborado con Lucidchart.

9.3.2 Restricciones de integridad

Existen ciertas condiciones, conocidas como restricciones de integridad, que son fundamentales para garantizar el correcto comportamiento del sistema. Sin embargo, estas no pueden representarse de forma gráfica dentro del esquema conceptual.

- **Claves externas:** (Producto, nregistro), (Persona, correo), (Recomendación, id), (Receta, id), (Notificación, id), (BúsquedaProducto, id), (ConsultaFarmacéutica, id).
- **RT1:** Una valoración tiene una puntuación del 0 al 1 (booleano).
- **RT2:** Un *Usuario* como mínimo tendrá una *Recomendación* si ha realizado tres búsquedas.
- **RT3:** La relación *Gestiona* tiene como gestor a la *Persona* con rol de *Administrador* y como gestionados a la *Persona* con rol de *Usuario* o *Farmacéutico*.
- **RT4:** Un *Encargo* con estado "Entregado" debe tener una fecha de entrega registrada.
- **RT5:** Una *Recomendación* incluye productos que han sido obtenidos de una búsqueda realizada previamente por el usuario.
- **RT6:** Los productos encargados mediante una *Receta* deben estar incluidos en la receta asociada.
- **RT7:** Una valoración solo puede existir si está asociada a una *Búsqueda* realizada previamente o a una *Recomendación* obtenida previamente.

9.3.3 Descripción de las clases

A continuación, se explicará de una forma más detallada que es y qué representa cada clase y sus atributos.

Producto: Representa los productos disponibles en el sistema.

- nregistro: Número de registro único del producto.
- nombre: Nombre del producto.
- descripción: Breve explicación de las características del producto.
- precio: Costo del producto en la moneda correspondiente.
- marca: Marca asociada al producto.
- laboratorio: Nombre del laboratorio fabricante del producto.
- categoría: Categoría del producto.
- stock: Cantidad disponible en inventario.
- imagen: URL de la imagen del producto.
- principiosActivos: Componentes que definen la función del producto.
- dosis: Cantidad recomendada de producto por toma.
- composición: Componentes o ingredientes principales del producto.
- prescripción?: Indica si el producto requiere receta médica (true o false).
- additionalInfo?: Link al prospecto oficial del producto.
- embedding: El embedding relativo al producto.

BúsquedaProducto: Representa la búsqueda de productos en el sistema que realiza el usuario.

- id: Identificador único de la búsqueda realizada.
- fecha: Fecha en la que se realiza la búsqueda.
- valoración: Puntuación asignada a la búsqueda (opcional).

HistorialDeBúsqueda: Representa el historial de búsqueda de productos que tiene el usuario.

- fecha: Fecha en la que se realizó la búsqueda.
- consulta: Consulta de la búsqueda realizada.

Persona: Representa a los individuos que interactúan con el sistema.

- correo: Dirección de correo electrónico e identificador único de la persona.
- nombre: Primer nombre de la persona.
- apellido: Apellido de la persona.
- contraseña: Contraseña asociada al perfil de usuario.

Persona Administrador: Persona que representa un administrador.

Persona Usuario: Persona que representa un usuario.

Persona Farmacéutico: Persona que representa un farmacéutico.

Notificación: Mensaje enviado a un usuario por correo.

- id: Identificador único de la notificación.
- fechaEnvio: Fecha en la que se envía la notificación.
- mensaje: Contenido del mensaje enviado en la notificación.
- tipo: Categoría de la notificación (recordatorio o promoción).

ConsultaFarmacéutica: Representa las consultas farmacéuticas efectuadas por el usuario y respondidas por el farmacéutico.

- id: Identificador único de la consulta.
- fecha: Fecha en la que se hace la consulta.
- pregunta: Pregunta que hace el usuario.
- respuesta: Respuesta del farmacéutico.
- estado: Estado de la consulta (pendiente o respondida).

Receta: Representa el documento digital asociado a un usuario que detalla medicamentos recetados por un médico.

- id: Identificador único de la receta.
- fechaEmisión: Fecha en la que se emite la receta.
- fechaCaducidad: Fecha en la que se termina la receta.
- médico: Nombre del médico que emite la receta.
- estado: Estado de la receta (activa o caducada).

Receta RecetaElectrónica: Representa una receta electrónica de la seguridad social.

- descuentoSS: Descuento que tienen los productos recetados por la Seguridad Social.

Receta RecetaPrivada: Representa una receta privada de la sanidad privada.

Recomendación: Representa las recomendaciones que recibe el usuario.

- id: Identificador único de la recomendación.
- valoración: Puntuación asignada por el usuario a la recomendación (opcional).

Encargo: Representa el encargo de productos hecho por un usuario.

- id: Identificador único del encargo.
- fecha: Fecha en la que se realizó el encargo.
- estado: Estado del encargo (pendiente, listo para recoger, entregado).

Encargo EncargoEntregado: Representa el encargo con estado entregado.

- fechaEntrega: Fecha en la que se entregó el encargo.

9.4 Modelo de comportamiento

Para finalizar con la especificación del sistema, se ha elaborado un modelo de comportamiento para los casos de uso desarrollados en este proyecto. En este apartado se presentan únicamente los casos de uso más relevantes, incluyendo para cada uno un diagrama de secuencia que muestra las acciones del usuario y las respuestas del sistema ante dichas acciones. También se especifican las condiciones iniciales y finales correspondientes. Este modelo tiene como objetivo definir la interacción entre el usuario y el sistema, mostrando las causas de las acciones realizadas por el usuario y los efectos de las operaciones del sistema.

El resto de modelos de comportamiento puede consultarse en el [Anexo 2](#).

#4 Modificar Perfil

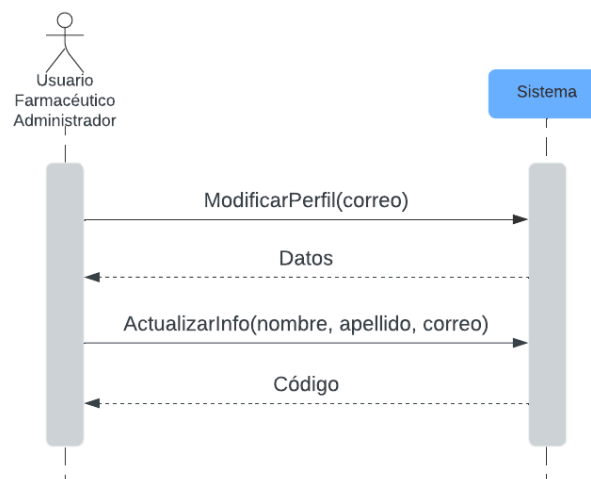


Figura 13: Diagrama de modificar perfil. Elaborado con Lucidchart.

ModificarPerfil(correo: String)	
Precondición	correo existe en el sistema.
Postcondición	-
Body	Se devuelven los datos actuales del actor (nombre, apellido, correo).

ActualizarInfo(nombre: String, apellido: String, correo: String)	
Precondición	Los datos tienen un formato válido.
Postcondición	Se actualizan los datos del actor (nombre, apellido, correo).

Body	Se devuelve un código de aceptación si se han actualizado los datos o un código de error de lo contrario.
-------------	---

#6 Recuperar contraseña

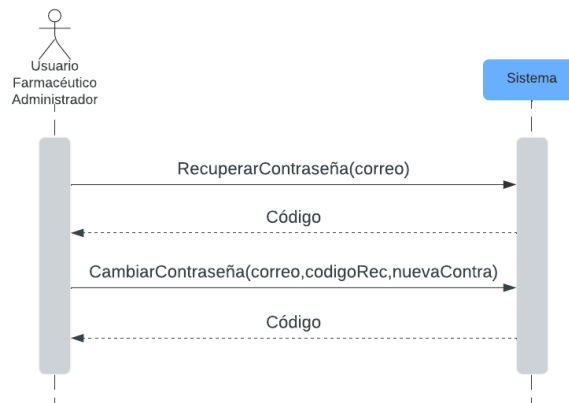


Figura 14: Diagrama de recuperar contraseña. Elaborado con Lucidchart.

RecuperarContraseña(correo: String)	
Precondición	-
Postcondición	Si el correo existe en el sistema, se envía un código de recuperación.
Body	Se devuelve un código de aceptación si se ha enviado el código de recuperación o un código de error si el correo no existe en el sistema.

CambiarContraseña(correo: String, codigoRec: String, nuevaContra: String)	
Precondición	-
Postcondición	Se cambia la contraseña del actor por <i>nuevaContra</i> .
Body	Se devuelve un código de aceptación si se ha realizado el cambio de contraseña o un código de error de lo contrario.

#13 Búsqueda semántica de productos

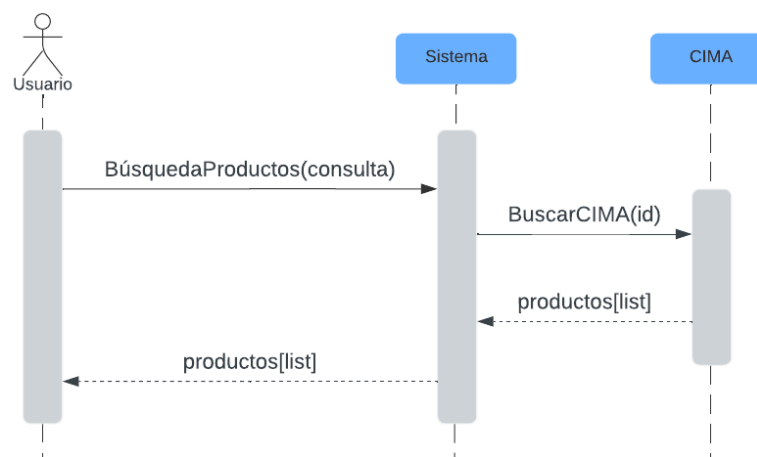


Figura 15: Diagrama de búsqueda semántica. Elaborado con Lucidchart.

BúsquedaProductos(consulta: String)	
Precondición	-
Postcondición	El sistema invoca la ejecución de la operación BuscarCIMA(id) del sistema de búsqueda de productos de CIMA y le pasa a la función el id del producto. El id del producto se obtiene a partir del análisis semántico de la consulta, utilizando embeddings para determinar la correspondencia más precisa entre consulta y datos almacenados.
Body	Se devuelve la lista de productos que devuelve la operación buscarCIMA(id).

Los casos de uso que van del **#14 Filtrar por laboratorio** al **#17 Filtrar por precio**, el modelo de comportamiento es idéntico a **#13 Búsqueda semántica de productos**. Lo único que cambia es el atributo por el que el sistema busca en CIMA. En vez de por id, sería por laboratorio, categoría, prescripción y precio. Por tanto, como el modelo de comportamiento es igual, no es necesario definir los diagramas de secuencia ni las tablas con las precondiciones y postcondiciones, ya que sólo varía el atributo.

#18 Consultar recomendaciones personalizadas

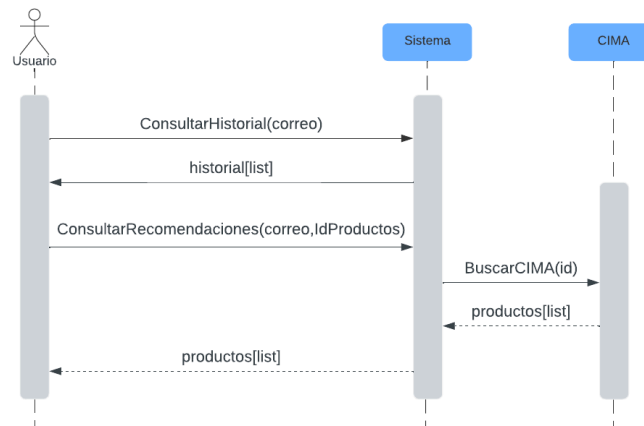


Figura 16: Diagrama de consultar recomendaciones. Elaborado con Lucidchart.

ConsultarHistorial(correo: String)	
Precondición	correo existe en el sistema.
Postcondición	-
Body	Se devuelve la lista de productos del historial que ha buscado el usuario.

ConsultarRecomendaciones(correo: String, IdProductos: Array[String])	
Precondición	correo existe en el sistema.
Postcondición	-
Body	Se devuelve una lista de los productos recomendados al usuario en base a las 5 últimas búsquedas de su historial.

#21 Valorar resultados de búsqueda

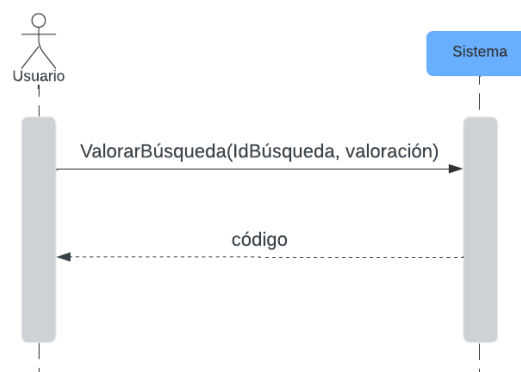


Figura 17: Diagrama de valorar búsqueda. Elaborado con Lucidchart.

ValorarBúsqueda(IdBúsqueda: String, valoración: Int)	
Precondición	<i>IdBúsqueda</i> existe en el sistema.
Postcondición	Se ha guardado la búsqueda <i>IdBúsqueda</i> con una valoración <i>valoración</i> entre 1 y 5.
Body	Se devuelve un código de aceptación si se ha realizado la valoración correctamente o uno de error de lo contrario.

El modelo de comportamiento del caso de uso **#22 Valorar recomendación personalizada** es igual al #21 Valorar resultados de búsqueda. Lo único que cambia es que en vez de pasar el atributo *IdBúsqueda* se le pasa el atributo *IdRecomendación*.

#23 Cargar receta electrónica



Figura 18: Diagrama de cargar receta. Elaborado con Lucidchart.

CargarReceta(correo: String, receta: archivo)	
Precondición	<i>correo</i> existe en el sistema. <i>receta</i> es un archivo de tipo PDF o imagen.
Postcondición	-
Body	Se devuelve un código de aceptación y la lista de productos asociados a la receta si se ha cargado correctamente, o uno de error y la lista de productos vacía de lo contrario.

#29 Recordatorios para medicamentos

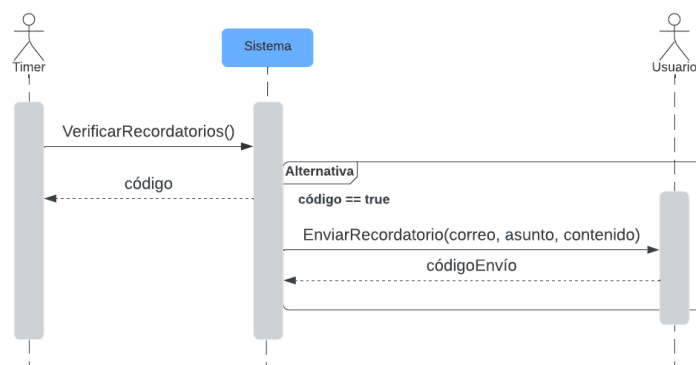


Figura 19: Diagrama de recordatorios. Elaborado con Lucidchart.

VerificarRecordatorios()	
Precondición	-
Postcondición	-
Body	Se devuelve un código a true si hay recordatorios a enviar a usuarios y a false si no hay.

EnviarRecordatorios(correo: String, asunto: String, contenido: String)	
Precondición	correo existe en el sistema.
Postcondición	El usuario recibe por correo el recordatorio del medicamento si el código es true.
Body	Se devuelve un código de envío de aceptación si se ha enviado el recordatorio correctamente, de error de lo contrario.

El modelo de comportamiento del caso de uso **#30 Notificar ofertas y promociones relevantes** es igual al #29 Recordatorios para renovar medicamentos. Lo único que cambia es que en vez de verificar y enviar recordatorios, se verifican y envían ofertas.

#31 Consultar duda a farmacéutico

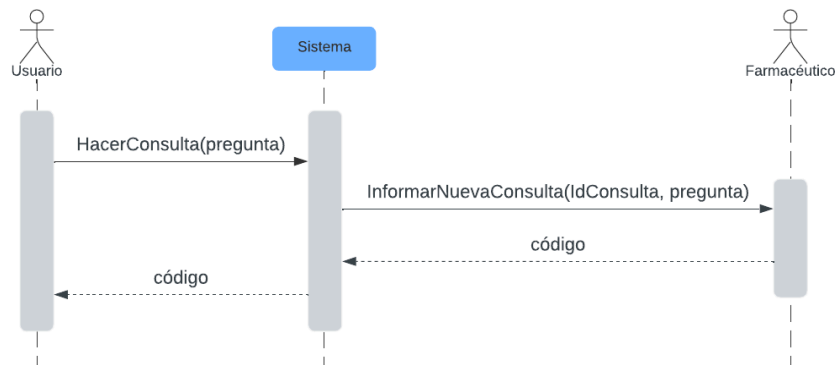


Figura 20: Diagrama de consultar duda. Elaborado con Lucidchart.

HacerConsulta(pregunta: String)	
Precondición	-
Postcondición	El sistema guarda la pregunta del usuario.
Body	Se devuelve un código a true si la consulta se ha realizado correctamente, a false si ha habido algún error.

InformarNuevaConsulta(IdConsulta: String, pregunta: String)	
Precondición	<i>IdConsulta</i> existe en el sistema.
Postcondición	El farmacéutico es notificado por el sistema de una nueva consulta.
Body	Se devuelve un código a true si se ha informado correctamente, a false si ha habido algún error.

#32 Solucionar consulta de usuario

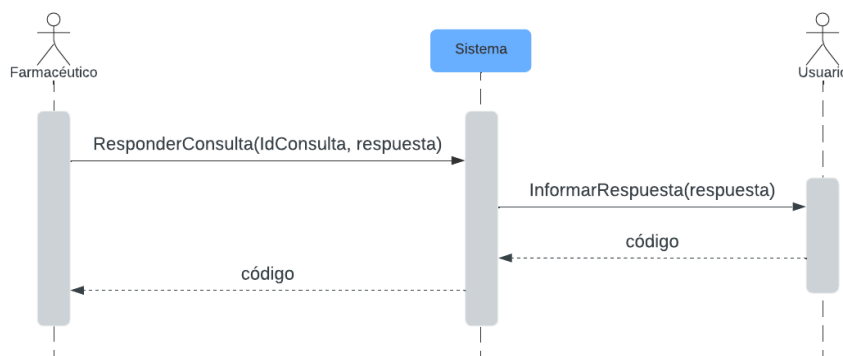


Figura 21: Diagrama de solucionar duda. Elaborado con Lucidchart.

ResponderConsulta(IdConsulta: String, respuesta: String)	
Precondición	<i>IdConsulta</i> existe en el sistema.
Postcondición	El sistema guarda la respuesta del farmacéutico.
Body	Se devuelve un código a true si la consulta se ha respondido correctamente, a false si ha habido algún error.

InformarRespuesta(respuesta: String)	
Precondición	-
Postcondición	El usuario es notificado por el sistema de una nueva consulta.
Body	Se devuelve un código a true si se ha informado correctamente, a false si ha habido algún error.

El modelo de comportamiento del caso de uso **#33 Consultar consultas del usuario** es igual al #24 Consultar recetas electrónicas, lo que cambia es que devuelve un `consultas[list]`.

#34 Cambiar estado de consulta

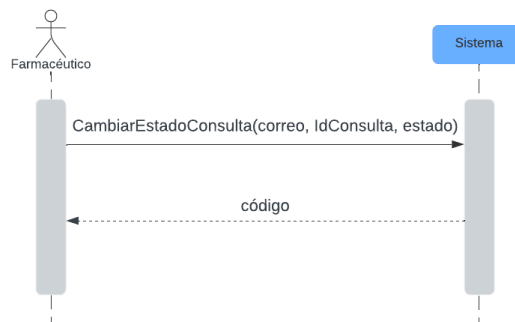


Figura 22: Diagrama de cambiar estado consulta. Elaborado con Lucidchart.

CambiarEstadoConsulta(correo: String, IdConsulta: String, estado: EstadoEncargo)	
Precondición	<i>correo</i> existe en el sistema. <i>IdConsulta</i> existe en el sistema.
Postcondición	Actualiza el estado de la consulta <i>IdConsulta</i> del usuario <i>correo</i> .
Body	Se devuelve un código de aceptación si se ha cambiado el estado, sino se devuelve código de error.

#35 Añadir producto de la receta al carrito



Figura 23: Diagrama de añadir producto de receta a carrito. Elaborado con Lucidchart.

AñadirProductoRecetaCarrito(IdProducto: String)	
Precondición	correo existe en el sistema. IdProducto existe en el sistema.
Postcondición	El producto <i>idProducto</i> se añade al carrito.
Body	Se devuelve un código de aceptación si se ha añadido el producto al carrito, sino se devuelve código de error.

El modelo de comportamiento del caso de uso **#36 Añadir producto sin receta al carrito** es igual al #35 Añadir producto de la receta al carrito. Lo único que cambia es que no se pasa el parámetro *correo*.

#38 Encargar productos del carrito

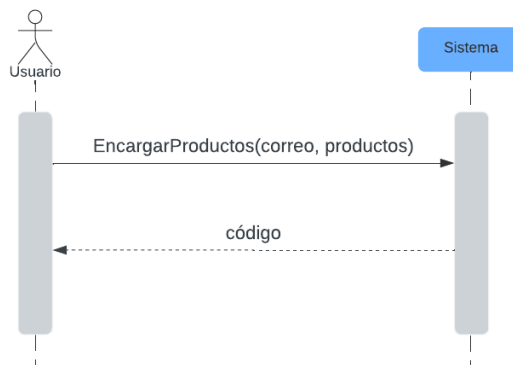


Figura 24: Diagrama de encargar productos. Elaborado con Lucidchart.

EncargarProductos(correo: String, productos: Array[String])	
Precondición	correo existe en el sistema. productos no está vacío.

Postcondición	-
Body	Se devuelve un código de aceptación si se ha hecho el encargo, sino se devuelve código de error.

El modelo de comportamiento del caso de uso **#41 Cambiar estado del encargo** es igual que el **#34 Cambiar estado de consulta**. Lo único que cambia es que se pasa el *IdEncargo* en vez del *Idconsulta*.

#42 Informar al usuario que el encargo está listo

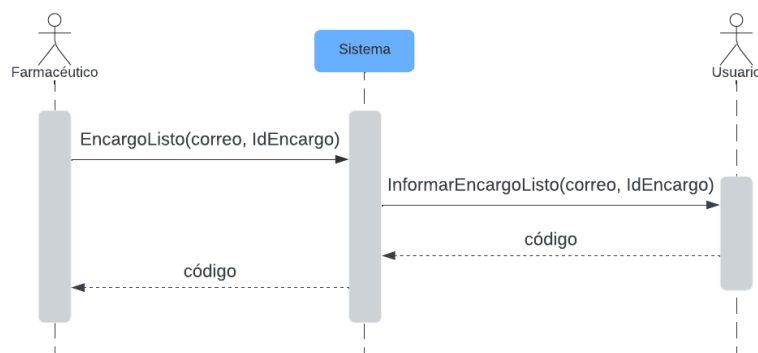


Figura 25: Diagrama de informar usuario sobre encargo. Elaborado con Lucidchart.

EncargoListo(correo: String, IdEncargo: String)	
Precondición	<i>correo</i> existe en el sistema. <i>IdEncargo</i> existe en el sistema.
Postcondición	Se actualiza el estado del encargo <i>IdEncargo</i> del usuario <i>correo</i> .
Body	Se devuelve un código de aceptación si se ha cambiado el estado y enviado el correo al usuario. Sino se devuelve código de error.

InformarEncargoListo(correo: String, IdEncargo: String)	
Precondición	<i>correo</i> existe en el sistema. <i>IdEncargo</i> existe en el sistema.
Postcondición	El usuario recibe la confirmación por correo de que el encargo está listo para ser recogido.
Body	Se devuelve un código de aceptación si se ha enviado el correo al usuario, sino se devuelve código de error.

#44 Editar información de producto

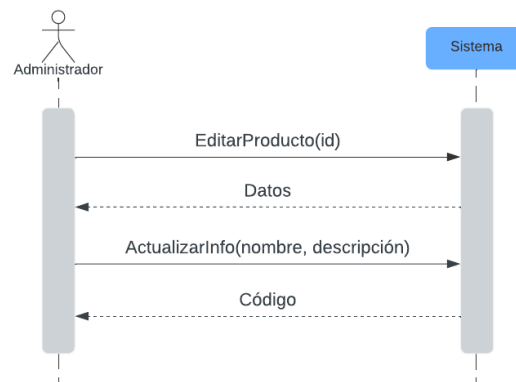


Figura 26: Diagrama de editar producto. Elaborado con Lucidchart.

EditarProducto(id: String)	
Precondición	El <i>id</i> del producto existe.
Postcondición	-
Body	Se devuelven los datos del producto.

ActualizarInfo(nombre: String, precio: Int, descripción: String)	
Precondición	Los datos tienen un formato válido.
Postcondición	Se actualizan los datos del producto.
Body	Se devuelve un código de aceptación si se han actualizado los datos o un código de error de lo contrario.

10. Diseño

Después de haber descrito el sistema junto con sus requisitos y funcionalidades, se procede a definir su organización interna y los elementos que lo componen. En las siguientes secciones se abordará la arquitectura general, el funcionamiento interno y, posteriormente, el diseño de la interfaz del sistema.

10.1 Arquitectura física

Definir una buena arquitectura física es clave para garantizar el rendimiento, la escalabilidad y la seguridad del sistema. En este apartado, se describe el entorno en el que se ejecuta el sistema y su organización para garantizar un buen rendimiento.

La Figura 27 muestra el esquema de la arquitectura física del sistema.

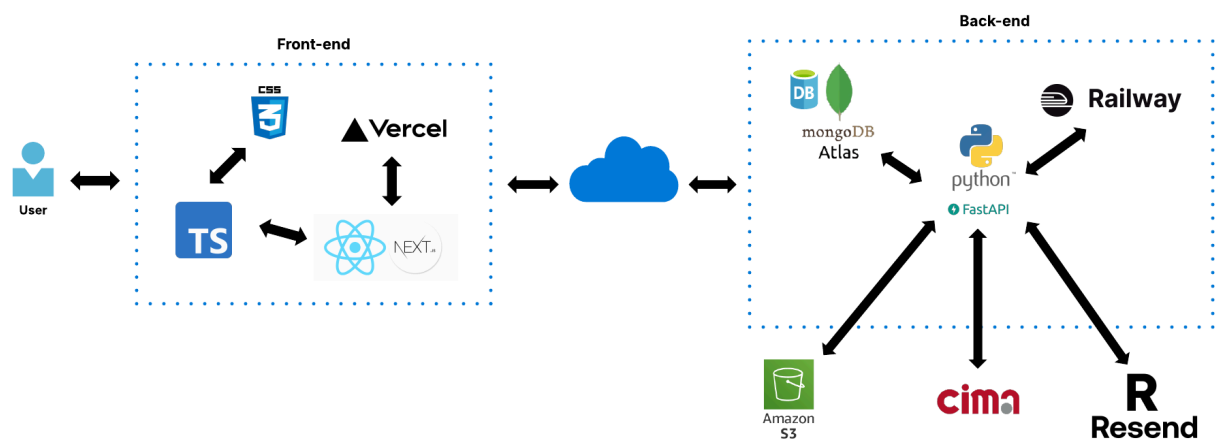


Figura 27: Arquitectura física del sistema. Elaborado con Lucidchart.

El sistema se estructura en dos partes principales: el *front-end*, que gestiona la interfaz de usuario, y el *back-end*, que se encarga de la lógica de negocio, el almacenamiento de datos y la comunicación con servicios externos.

10.1.1 Front-end

El desarrollo del *front-end* se basa en tecnologías que permiten una experiencia de usuario rápida, intuitiva y bien optimizada. Para diseñar la interfaz, se ha optado por usar **React** junto a **Next.js**, un *framework*⁷ que facilita el desarrollo de aplicaciones web [39]. Next.js ayuda a mejorar el rendimiento, ya que puede generar páginas de forma estática y hacer renderizado del lado del servidor, lo que acelera los tiempos de carga. Esto es esencial para cumplir nuestros requisitos de rendimiento.

⁷ Un *framework* es un marco de trabajo que ofrece una estructura base para elaborar un proyecto con objetivos específicos [40].

El lenguaje de programación utilizado en el *front-end* será **TypeScript**, en lugar de *JavaScript*, debido a sus ventajas en cuanto a tipado estático y detección temprana de errores. Esto contribuye a un código más estructurado y mantenible a largo plazo. Además, para el diseño de la interfaz, se ha decidido emplear *Bootstrap* y *Tailwind CSS*, *frameworks* de estilos que proporcionan una gran cantidad de componentes predefinidos y una adaptación responsiva sin necesidad de escribir demasiado código **CSS** manualmente.

Una particularidad del sistema es que no se usarán archivos HTML directamente, ya que *Next.js* genera dinámicamente la estructura de las páginas a partir de los componentes de *React*.

Para el despliegue del *front-end*, se ha optado por **Vercel**, ya que es la plataforma oficial de despliegue para aplicaciones creadas con *Next.js*. Esto garantiza compatibilidad total y optimizaciones automáticas.

10.1.2 Back-end

Para la implementación del *back-end*, se ha elegido **Python** junto con el *framework* **FastAPI**. Este *framework* es conocido por ser muy rápido y simple para crear APIs REST⁸. *FastAPI* no solo hace que el código sea más eficiente, sino que también soporta el tipado de *Python*, lo que ayuda a detectar errores más fácilmente y facilita el mantenimiento del software.

Los datos del sistema se guardan en **MongoDB Atlas**, una base de datos NoSQL (Not Only SQL) en la nube que ofrece almacenamiento flexible y que puede escalar según las necesidades. Al trabajar con una base de datos NoSQL, se pueden agregar nuevas estructuras de datos sin definirlas previamente, lo que mejora el rendimiento y permite un desarrollo ágil con iteraciones rápidas y significativas.

Además del almacenamiento de datos, el sistema requiere una gestión eficiente de archivos, especialmente imágenes de los usuarios. Para ello, se ha optado por **AWS S3**, un servicio de almacenamiento en la nube que ofrece *Amazon Web Service* que permite guardar y recuperar archivos de manera segura y escalable.

Un elemento importante del sistema es su integración con la **API CIMA**, proporcionada por la Agencia Española de Medicamentos y Productos Sanitarios. El *back-end* se conectará con CIMA para recuperar datos relevantes y los enviará al *front-end* para su visualización.

⁸ REST es un estilo de diseño de servicios web que usa HTTP para intercambiar datos entre sistemas de forma sencilla y eficiente [41].

Otro servicio externo en el sistema es **Resend**, que se utiliza para gestionar el envío de correos electrónicos automatizados. A través de esta API, el sistema puede enviar notificaciones, confirmaciones de pedidos y otras comunicaciones a los usuarios de manera eficiente.

Para el despliegue del *back-end*, se ha optado por **Railway**, que permite gestionar variables de entorno, desplegar directamente desde Git y escalar el servicio fácilmente.

10.2 Arquitectura lógica

La arquitectura lógica del sistema define la organización de sus componentes en dos capas principales: *front-end* y *back-end*. En la Figura 28, se presenta un esquema visual de esta estructura, mostrando la interacción entre sus módulos y los servicios externos involucrados.

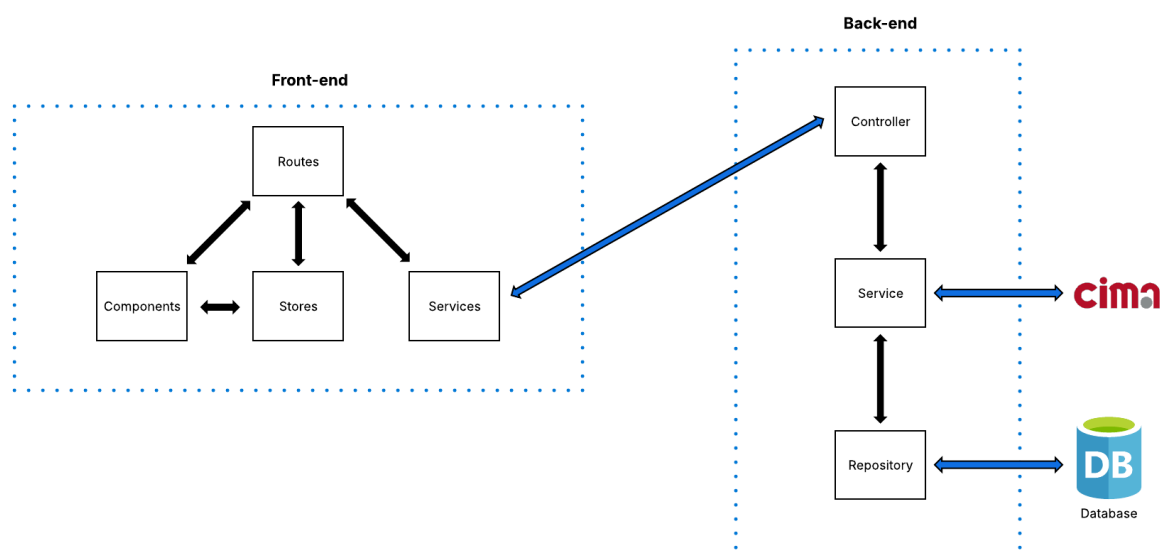


Figura 28: Arquitectura lógica del sistema. Elaborado con Lucidchart.

A continuación, se va a explicar detalladamente la arquitectura lógica del *front-end* y del *back-end*, así como los patrones de diseño que se van a usar.

10.2.1 Front-end

El frontend de la aplicación está diseñado siguiendo una arquitectura modular basada en App Router de Next.js [42], lo que permite una gestión eficiente de rutas, estado global y comunicación con el *back-end*. En la Figura 29, se puede observar la arquitectura lógica del *front-end*.

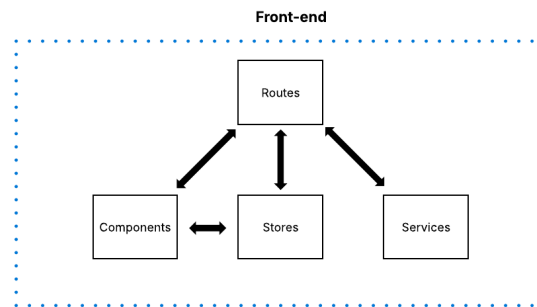


Figura 29: Arquitectura lógica del *front-end*. Elaborado con Lucidchart.

Las **rutas** son responsables de gestionar la navegación y la obtención de datos del *back-end* a través de los servicios. También se encargan de construir la interfaz de usuario mediante componentes que se pueden reutilizar. Cada carpeta dentro de esta estructura representa una ruta, y puede incluir archivos como `page.tsx` para el contenido principal. Estas rutas tienen acceso a los *stores*, que gestionan los estados compartidos y ayudan a mantener los datos organizados.

Los **componentes** son elementos reutilizables de la interfaz gráfica que presentan los datos al usuario. Pueden ser *Server Components* (para evitar el uso innecesario de JavaScript en el cliente) o *Client Components* (cuando necesitan manejar interactividad, estado o efectos). Los componentes acceden a los *stores* para obtener información compartida, como configuraciones o datos de sesión.

Los **servicios** actúan como la capa de comunicación entre el *front-end* y el *back-end*, gestionando las peticiones a la API. Son utilizados por las rutas para obtener y enviar información. Esta capa permite centralizar las llamadas al *back-end* y evitar que los componentes o las rutas realicen peticiones directas a la API.

Por último, los **stores** manejan el estado que se comparte dentro de la aplicación. Esto ayuda a reducir llamadas innecesarias a la API y mantiene la coherencia de los datos en toda la interfaz. Tanto las rutas como los componentes pueden acceder a estos *stores* para obtener información persistente y actualizar la interfaz con los datos más recientes.

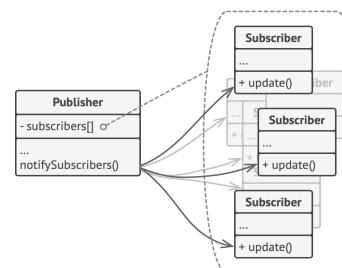
Para garantizar una arquitectura sólida y bien estructurada, se implementan varios patrones de diseño en el desarrollo del *front-end*. A continuación, se van a explicar cada uno de ellos.

Patrón Contenedor-Presentacional:

En esta arquitectura, las rutas actúan como contenedores que manejan la obtención de datos y la gestión de la navegación, mientras que los componentes se encargan de la presentación y visualización. Esto facilita la reutilización del código y mejora la mantenibilidad del sistema [43].

Patrón Observador:

Los diferentes componentes y rutas se suscriben a cambios en los stores de Zustand y se actualizan automáticamente cuando el estado cambia. Esto minimiza la necesidad de múltiples llamadas a la API y asegura que la interfaz siempre refleje los datos más actualizados [44]. En la Figura 30 se muestra un ejemplo del patrón.



El notificador notifica a los suscriptores invocando el método de notificación específico de sus objetos.

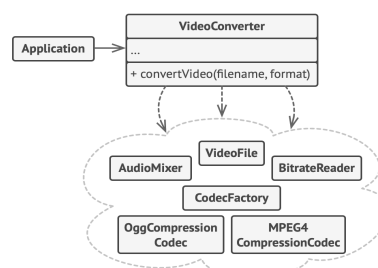
Figura 30: Patrón Observador. [44].

Patrón Módulo:

Organiza el código en archivos separados y reutilizables, agrupando funcionalidades específicas dentro de módulos bien definidos. En este caso, la aplicación está estructurada en módulos como rutas, componentes, servicios y stores, lo que contribuye a una arquitectura escalable y fácil de mantener [45].

Patrón Fachada:

Se implementa en la capa de servicios, proporcionando una interfaz simplificada para interactuar con el *back-end*. Gracias a este patrón, se ocultan los detalles de implementación de las peticiones y se facilita el consumo de datos por parte de las rutas y los componentes del sistema [46]. En la Figura 31 se muestra un ejemplo del patrón.



Un ejemplo de aislamiento de múltiples dependencias dentro de una única clase fachada.

Figura 31: Patrón Fachada. [46].

Patrón Singleton:

Garantiza que cada store sea una única instancia en toda la aplicación, asegurando que no haya múltiples instancias del mismo estado. Esto evita la creación innecesaria de estados duplicados y mantiene la coherencia en toda la aplicación [47]. En la Figura 32 se muestra un ejemplo del patrón.

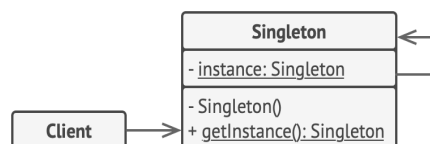


Figura 32: Patrón Singleton. [47].

10.2.2 Back-end

El *back-end* del sistema sigue una arquitectura en capas que permite una separación clara de responsabilidades, mejorando el mantenimiento y la escalabilidad [48]. En la Figura 33, se puede observar la arquitectura lógica del *back-end*.

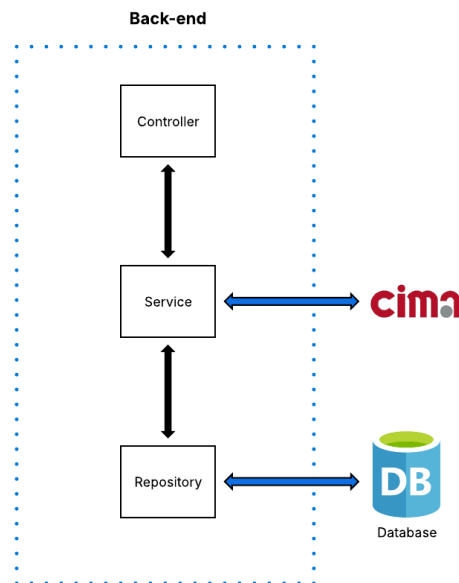


Figura 33: Arquitectura lógica del *back-end*. Elaborado con Lucidchart.

La capa de **controlador** actúa como el punto de entrada de las solicitudes provenientes del *front-end*, gestionando la comunicación y delegando la lógica de negocio a la capa de servicio.

La capa de **servicio** es responsable de procesar los datos, aplicar validaciones y ejecutar las reglas de negocio antes de interactuar con la base de datos. Esta capa se encarga de mantener la coherencia del sistema y asegurar que todas las operaciones cumplan con los requisitos establecidos.

Por último, la capa de **repositorio** gestiona la interacción directa con la base de datos MongoDB, ejecutando consultas y operaciones de almacenamiento. Al centralizar esta funcionalidad, se facilita el acceso a los datos y se mantiene una mayor flexibilidad ante posibles cambios en la estructura de la base de datos.

Para mejorar la organización y eficiencia del sistema, se implementan distintos patrones de diseño en el *back-end*. A continuación, se van a explicar cada uno de ellos.

Patrón Estado:

En el caso de la entidad Encargo, se implementa el patrón Estado para gestionar la transición específica al estado de "Entregado". Esto permite encapsular la lógica asociada únicamente a los encargos entregados, evitando una gestión rígida mediante simples *strings* y asegurando que las transiciones sean controladas. De este modo, el sistema garantiza coherencia en los datos y facilita la evolución del modelo sin afectar el comportamiento de otros estados [49]. En la Figura 34 se muestra un ejemplo del patrón.

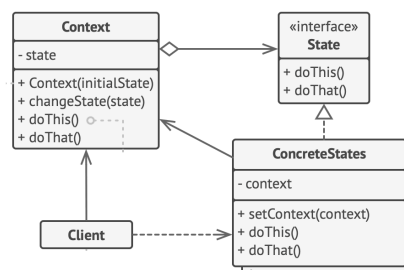


Figura 34: Patrón Estado. [49].

Patrón Repositorio:

Este patrón se aplica a la capa de repositorio, proporcionando una abstracción sobre MongoDB para gestionar el acceso a los datos. En lugar de interactuar directamente con la base de datos en diferentes partes del código, la lógica de consultas y modificaciones se centraliza en una capa específica, facilitando la reutilización y el mantenimiento [50].

Patrón Inyección de Dependencias:

Este patrón se implementa en la gestión de servicios, permitiendo que las dependencias (como repositorios o servicios externos) sean inyectadas en lugar de ser creadas dentro de las clases. Esto mejora la modularidad, facilita la prueba de unidades y permite reemplazar implementaciones fácilmente si en el futuro se cambia la base de datos o se integran nuevos servicios [51].

10.3 Diseño del front-end

A continuación, se mostrará cómo se verá cada sección de la página web y cómo se navega a través de ella.

10.3.1 Diseño externo

En este apartado se muestran los *mockups*⁹ de las secciones más importantes de la página web, con una breve explicación de cada una. Las demás páginas estarán disponibles en el [Anexo 3](#). Es posible que en la versión final del proyecto haya modificaciones con respecto a los mockups mostrados, ya que el diseño está en constante evolución.

Inicio (usuarios):



Figura 35: *Diseño inicio de usuarios*. Elaborado con Wix.

La pantalla de inicio es la primera vista del sistema web, diseñada para ofrecer un acceso rápido a las funcionalidades principales, como la gestión de recetas, la consulta del catálogo de medicamentos, los encargos y el contacto. También permite el acceso al carrito de compras y al menú desplegable del usuario, donde hay la información privada del usuario y la opción de cerrar sesión. Incorpora una barra de búsqueda semántica, permitiendo a los usuarios encontrar productos y servicios mediante descripciones intuitivas. Además, se describe el nuevo funcionamiento de esta barra de búsqueda.

⁹ Un mockup es una representación visual estática de una interfaz, que muestra su diseño y estructura sin funcionalidad interactiva.

Catálogo de Productos (usuarios):

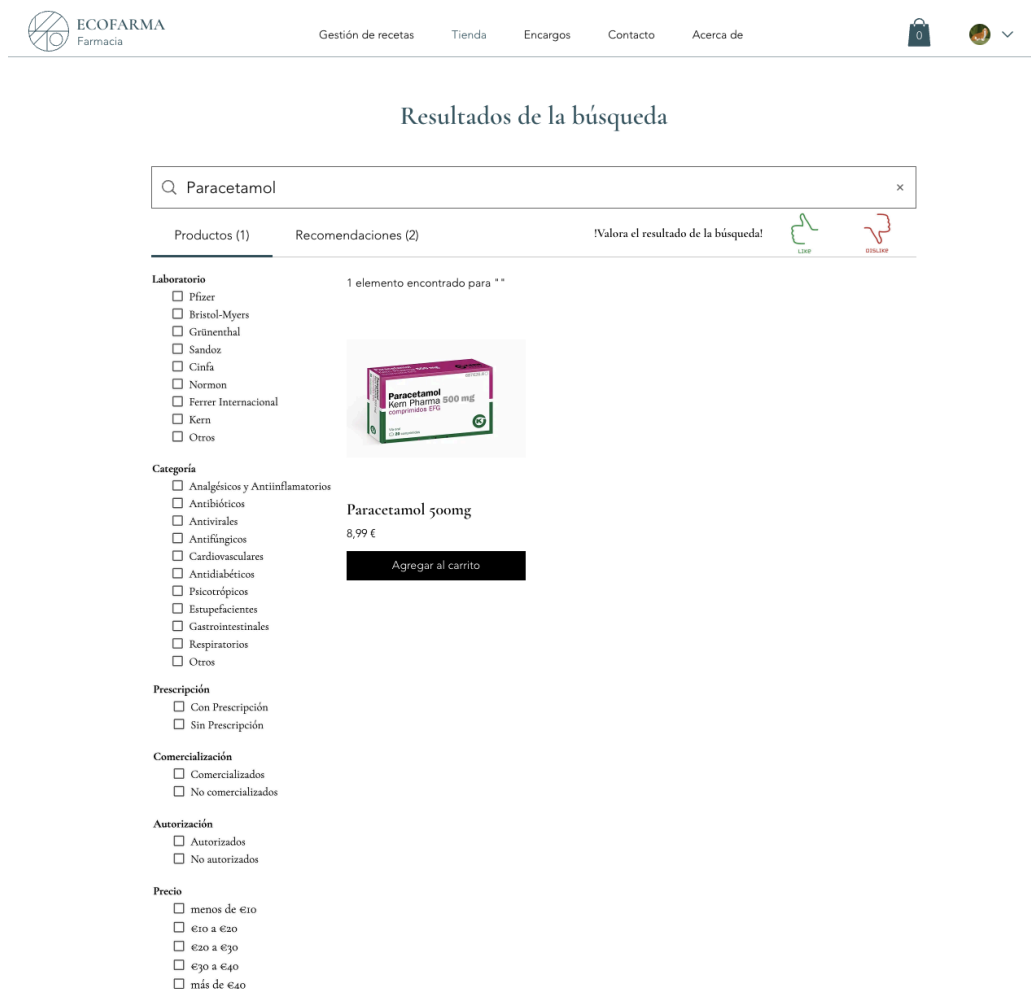


Figura 36: Diseño catálogo de productos. Elaborado con Wix.

La página de catálogo de productos permite a los usuarios explorar los medicamentos disponibles en la farmacia. Se muestra una lista de productos con filtros avanzados que permiten refinar la búsqueda según laboratorio, categoría, necesidad de prescripción, rango de precios y otros criterios relevantes.

Cuenta con una barra de búsqueda semántica con lenguaje natural, permitiendo encontrar productos de manera intuitiva sin necesidad de términos exactos. Además, cada producto tiene la opción de ser añadido al carrito de compras de forma rápida y sencilla, optimizando la experiencia del usuario. En la parte derecha cuenta con una encuesta de valoración de la búsqueda realizada. Esta retroalimentación ayuda a mejorar la precisión del sistema y personalizar aún más las sugerencias.

Detalle de Producto (usuarios):



[Gestión de recetas](#) [Tienda](#) [Encargos](#) [Contacto](#) [FAQ](#) [Acerca de](#)



Paracetamol 500mg
SKU: 364215376135199
8,99 €

Cantidad

[Agregar al carrito](#)

Composición —
El principio activo es paracetamol. Cada comprimido contiene 1 g de paracetamol. Los demás componentes son: almidón de maíz pregelatinizado, ácido estéarico y povidona.

Advertencias y precauciones +

Contraindicaciones +

Cómo tomar +

Reacciones adversas +

Posología +

Conservación +

Este medicamento está indicado para el tratamiento sintomático del dolor de cualquier causa de intensidad moderada, como por ejemplo: dolores postoperatorios y del parto, dolores reumáticos (artritis y artritis reumatoide), lumbago, tortícolis, cólica, neuralgias, dolor de espalda, dolores musculares, dolores menstruales, dolor de cabeza y dolor dental. Estados febriles y en las molestias que acompañan al resfriado y a la gripe.

También te puede interesar



Espidol 400mg



Ibuprofeno Calox



Geloprofen 400mg

Figura 37: Diseño de detalle de producto. Elaborado con Wix.

La página de detalle del producto muestra información completa sobre un medicamento disponible en la farmacia. Incluye una imagen del producto, una descripción general, el precio y un selector de cantidad, junto con un botón para añadirlo al carrito de encargos de manera sencilla.

Además, se proporciona información detallada sobre el producto, incluyendo su composición, advertencias, contraindicaciones, modo de uso, posibles reacciones adversas, posología y condiciones de conservación. Por último, contiene un apartado de productos relacionados. Esta sección permite a los usuarios conocer en profundidad cada producto antes de realizar un encargo.

Recomendaciones Personalizadas (usuarios):



[Gestión de recetas](#) [Tienda](#) [Encargos](#) [Contacto](#) [Acerca de](#)

Resultados de la búsqueda

Productos (1)

Recomendaciones (2)

!Valora el resultado de las recomendaciones!
 

2 elementos encontrados para **



Espidol 400mg
€6,99



Pharmagrip
€11,99

Figura 38: Diseño de recomendaciones. Elaborado con Wix.

La página de recomendaciones muestra una lista de productos sugeridos en función de las búsquedas y encargos previos del usuario registrado. Además, incluye una breve encuesta de valoración, permitiendo a los usuarios valorar la utilidad de las recomendaciones. Esta retroalimentación ayuda a mejorar la precisión del sistema y personalizar aún más las sugerencias.

Gestión de Recetas (usuarios):

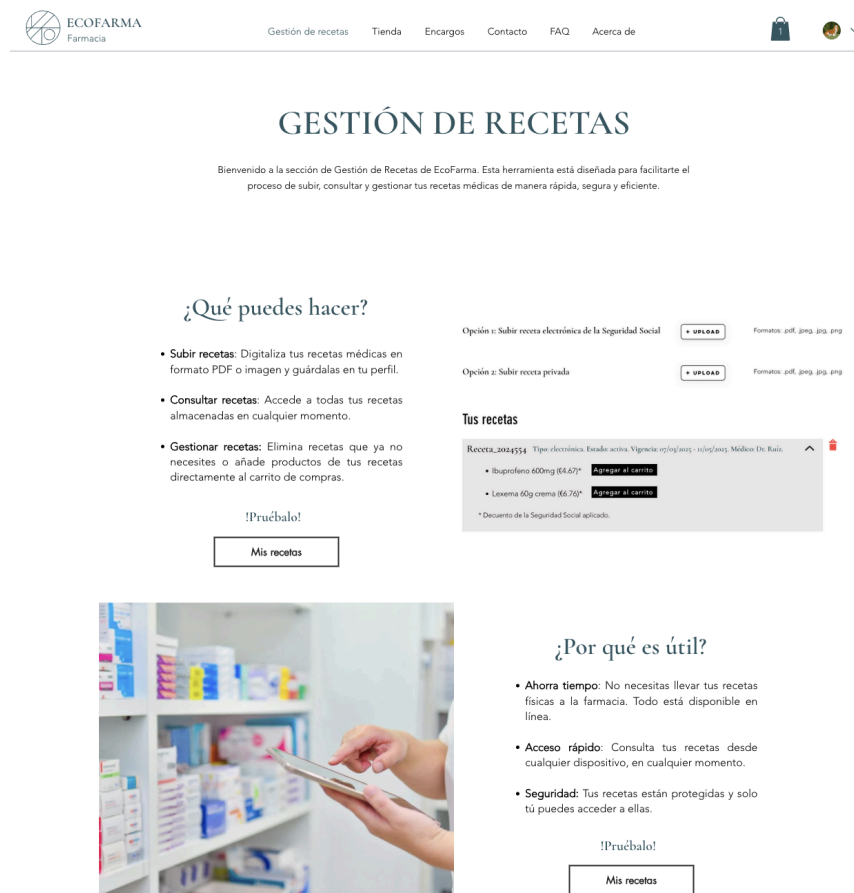


Figura 39: Diseño de gestión de recetas público. Elaborado con Wix.

Esta página es accesible para todos los usuarios, estén o no registrados, y proporciona una introducción a la gestión de recetas en la plataforma. Explica cómo los usuarios pueden cargar recetas electrónicas y privadas para agilizar el proceso de adquisición de medicamentos, destacando su utilidad y facilidad de uso.

Además, incluye un botón de acceso a la sección "Mis recetas", donde los usuarios registrados podrán gestionar sus recetas y hacer uso de todas las funcionalidades disponibles, como la carga de sus prescripciones.

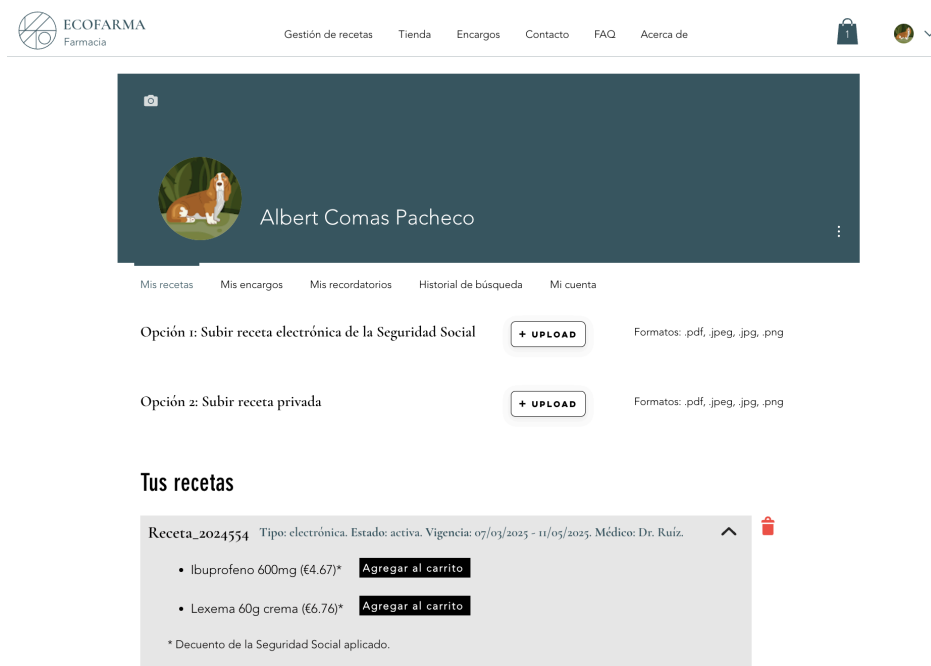


Figura 40: Diseño de gestión de recetas. Elaborado con Wix.

Disponible solo para usuarios registrados, esta página permite gestionar las recetas médicas de manera sencilla. Los usuarios pueden subir nuevas recetas eligiendo entre dos opciones: receta electrónica o receta privada (imagen o documento PDF).

En la parte inferior, se muestra un listado con las recetas subidas por el usuario. Para cada receta se indica el tipo, estado, vigencia, médico prescriptor y los medicamentos asociados. Cada medicamento cuenta con un botón para añadirlo al carrito, y también se ofrece la opción de eliminar recetas que ya no sean necesarias.

Carrito de Encargos (usuarios):

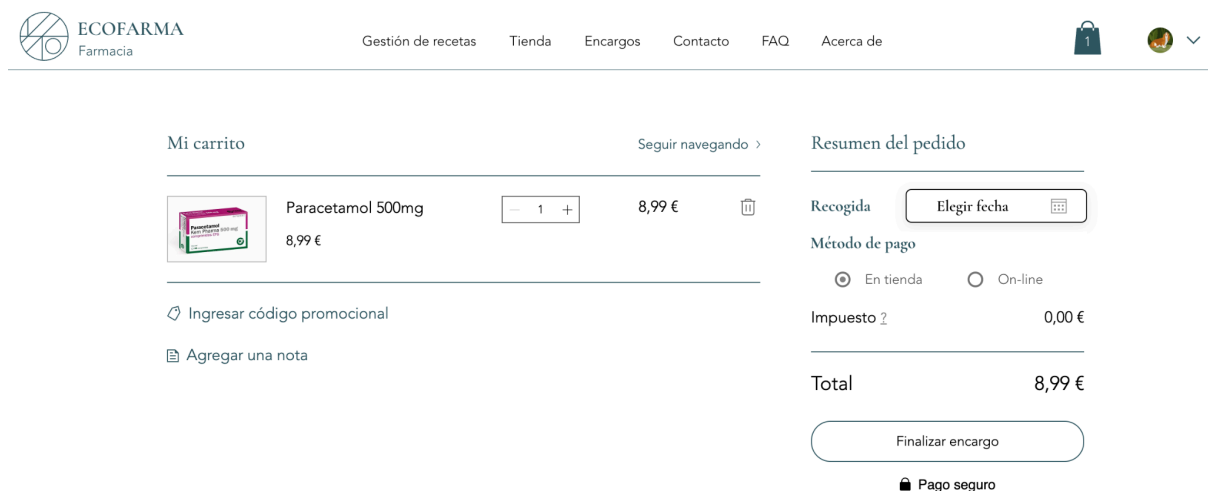


Figura 41: Diseño de carrito. Elaborado con Wix.

La página del carrito de encargos permite a los usuarios gestionar los productos que han seleccionado para su compra. En esta página, se muestra un listado detallado de los productos añadidos al carrito, incluyendo la cantidad, el precio unitario y el precio total por producto. Cada producto cuenta con un botón para eliminarlo del carrito, brindando flexibilidad al usuario para ajustar su pedido antes de finalizarlo.

Además, la página ofrece la opción de ingresar un código promocional o añadir una nota al pedido, como instrucciones especiales para la recogida. En la sección de resumen del pedido, se indica la fecha de recogida, el método de pago (en tienda o online) y el total del encargo. Finalmente, un botón de "Finalizar Encargo" permite al usuario confirmar y completar su pedido de manera sencilla y segura.

Encargos (usuarios):



Figura 42: *Diseño de encargos público. Elaborado con Wix.*

Esta página está diseñada para todos los usuarios y se explica por qué hacer encargos en la farmacia, cómo empezar a realizar encargos y dónde recogerlos. Incluye un botón de "Mis Encargos" que redirige a los usuarios registrados a su lista de pedidos.

Además, esta página sirve como punto de entrada para fomentar el registro de nuevos usuarios, mostrando las ventajas de tener una cuenta.

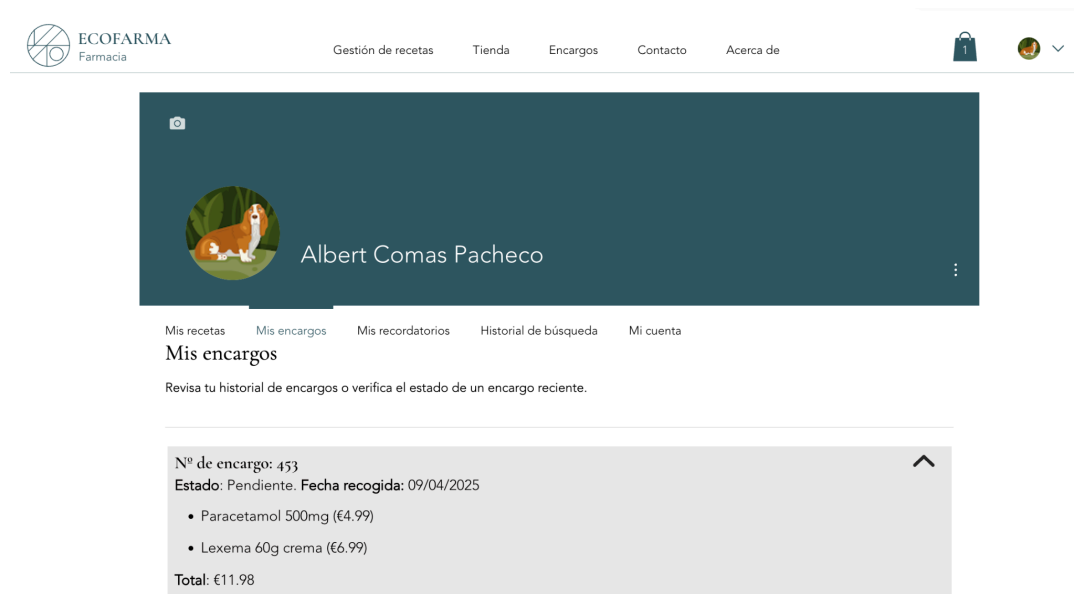


Figura 43: Diseño de encargos. Elaborado con Wix.

La página de "Mis Encargos" está destinada exclusivamente a usuarios registrados y muestra un listado detallado de los pedidos realizados. Cada encargo incluye información como el número de encargo, el estado (pendiente, listo para recoger, entregado), la fecha de recogida y una lista de los medicamentos solicitados con su precio unitario y total. Esta organización permite al usuario tener un control completo sobre sus pedidos.

Contacto (usuario):



Contáctanos mediante las siguientes opciones

Teléfono
+34 937 832 334

Email
infoecofarma@gmail.com

Dirección
Calle Dolores 10, Mataró
Barcelona 08350

Figura 44: Diseño de contacto. Elaborado con Wix.

La página de contacto permite a los usuarios enviar consultas a los farmacéuticos mediante un formulario, donde pueden describir sus dudas. Estas consultas son respondidas directamente en la misma página, facilitando una comunicación clara y organizada. Además, se muestra un listado de consultas anteriores, incluyendo la pregunta, la respuesta, el estado (resuelta/pendiente) y la fecha.

En la parte inferior de la página, se ofrece una alternativa de contacto tradicional, con el teléfono, correo electrónico y la ubicación física de la farmacia. Esta combinación de opciones asegura que los usuarios puedan elegir el método de contacto que mejor se adapte a sus necesidades.

Inicio (farmacéuticos):



Figura 45: *Diseño inicio de farmacéuticos. Elaborado con Wix.*

La página de inicio para farmacéuticos ofrece un acceso rápido a las funcionalidades clave mediante tres botones principales: Gestionar Encargos, Asesoramiento Farmacéutico y Consultar Usuarios. En el menú desplegable del perfil hay un botón que permite al farmacéutico cerrar sesión. Este diseño simplificado permite a los farmacéuticos navegar de manera eficiente. La interfaz es clara y está optimizada para mejorar la productividad y la atención al usuario.

Asesoramiento a Usuarios (farmacéuticos):



ASESORAMIENTO A USUARIOS

Asesoramiento en línea

Figura 46: Diseño de asesoramiento a usuarios. Elaborado con Wix.

La página de asesoramiento a usuarios está diseñada para que los farmacéuticos gestionen y respondan las consultas enviadas por los usuarios. En ella se muestra un listado de todas las consultas recibidas, con detalles como el asunto, la fecha, el estado (pendiente o respondida), el usuario que la envió y el mensaje de la consulta.

Cada consulta incluye un cuadro de texto donde el farmacéutico puede redactar y enviar su respuesta. Una vez enviada, la respuesta queda registrada y visible junto con la consulta original. Además, el farmacéutico tiene la opción de cambiar el estado de la consulta, facilitando la organización y el seguimiento de las solicitudes.

Inicio (administradores):



Figura 47: Diseño inicio de administrador. Elaborado con Wix.

La página de inicio del administrador sirve como panel de control para gestionar la plataforma. Presenta cuatro botones principales que permiten acceder a la gestión de productos, farmacéuticos y usuarios, además de una sección para visualizar estadísticas del sistema. En el menú desplegable del perfil hay un botón que permite al farmacéutico cerrar sesión.

Cada opción facilita la administración eficiente del sitio, permitiendo al administrador mantener actualizada la información y supervisar el funcionamiento general de la farmacia en línea.

Gestión del Catálogo de Productos (administradores):



Figura 48: *Diseño de gestión de productos.* Elaborado con Wix.

La pantalla de gestión de productos permite a los administradores visualizar y administrar el catálogo disponible en la farmacia. Se muestra un listado de productos con su nombre e identificador único, junto con un buscador para facilitar la localización de artículos específicos.

Cada producto en la lista incluye opciones para editar o eliminarlo. Además, se proporciona un botón para añadir nuevos productos, lo que permite actualizar el catálogo según sea necesario. Al hacer clic en un producto, el usuario es redirigido a su página de detalles.

10.3.2 Diseño interno

En este apartado se presenta el diseño interno de las clases. Dado que elaborar un diagrama de todas las clases que intervienen en el *front-end* del sistema resultaría muy extenso, se mostrará como ejemplo el diseño de la funcionalidad de mostrar productos a partir de una búsqueda semántica (y el usuario acaba haciendo clic en un producto en concreto).

La Figura 49 muestra las rutas, los componentes, los stores y los servicios que han intervenido en la muestra de productos a partir de una búsqueda semántica.

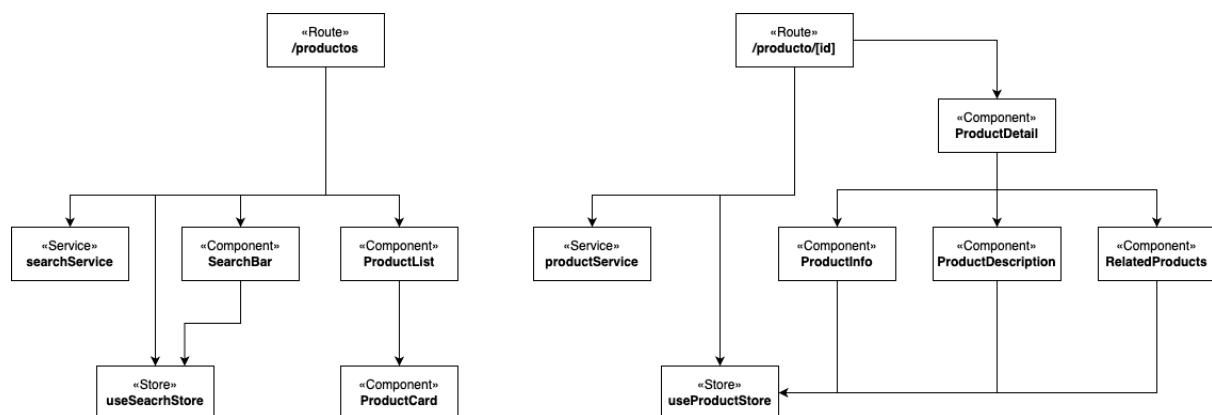


Figura 49: Diseño interno del front-end. Elaborado con Lucidchart.

A continuación, se detallan las funciones principales de los componentes, stores y servicios más relevantes.

Servicios (Service)

- **searchService**
getProducts(query: string, filters?: object)
Realiza una solicitud a la API para obtener productos basados en una búsqueda.
- **productService**
getProductById(id)
Recupera los detalles de un producto específico desde la API.

Stores

- **useSearchStore**
setProducts(products: Product[])
Almacena la lista de productos en el estado global del store.
getProducts()
Devuelve la lista de productos almacenados en el estado global.

- useProductStore.ts
setProduct(product: Product)
 Almacena el producto en el estado global del store.
getProductInfo()
 Devuelve la información del producto (nombre, imagen, precio) almacenado en el estado global.
getProductDescription()
 Devuelve la descripción del producto (descripción, componentes, etc) almacenado en el estado global.
getRelatedProducts()
 Devuelve los productos relacionados con el producto mostrado almacenado en el estado global.

Componentes (Components)

- SearchBar
handleSearch(event: ChangeEvent<HTMLInputElement>)
 Captura la entrada del usuario.
renderSearchBar()
 Renderiza la barra de búsqueda de productos.
- ProductList
renderProductList()
 Muestra la lista de productos obtenidos de la búsqueda.
handleSelectProduct(event: productId)
 Captura el producto seleccionado por el usuario.
- ProductDetail
renderProductDetail()
 Renderiza el producto seleccionado.
 - ProductInfo
renderProductInfo()
 Renderiza la información del producto seleccionado.
 - ProductDescription
renderProductDescription()
 Renderiza la descripción del producto seleccionado.
 - RelatedProducts
renderRelatedProducts()
 Renderiza los productos relacionados del producto seleccionado.

Para entender mejor cómo interactúan los componentes se mostrarán dos diagramas de secuencia. El primer diagrama, la Figura 50, muestra la navegación inicial a la página del catálogo de productos y la posterior búsqueda semántica. El segundo diagrama, la Figura 51, muestra la navegación a la página de un producto en concreto cuando el usuario hace clic.

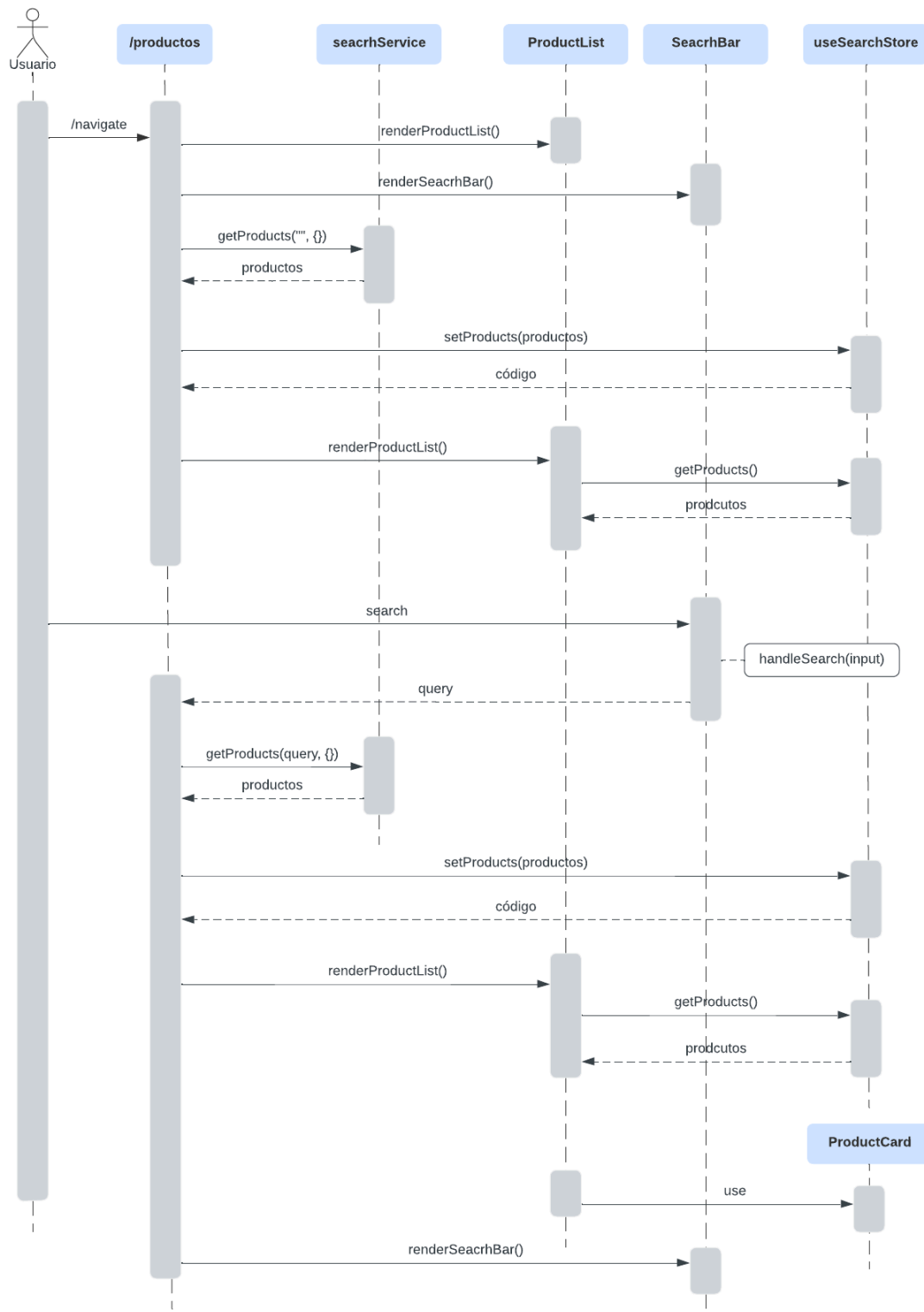


Figura 50: Diagrama de secuencia de navegación inicial. Elaborado con Lucidchart,

Cuando el usuario navega a **/productos**, React hace un primer *render* automático de **ProductList** y **SearchBar**, pero la lista de productos está vacía porque aún no hay datos en el estado global. Luego, **/productos** llama a `searchService.getProducts("", {})` para obtener los productos del *back-end*. Al recibir los datos, **searchService** actualiza el estado global a través de `useSearchStore.setProducts(productos)`, lo que provoca un segundo *render* automático de **ProductList**, ahora con los productos cargados. Este segundo *render* es automático debido al cambio de estado en el **store**.

Cuando el usuario introduce una búsqueda en el componente **SearchBar**, se ejecuta la función `handleSearch(input)`, que captura la entrada del usuario. Luego, igual que antes, **/productos** obtiene los productos del *back-end* con la query del usuario y actualiza el estado global **useSearchStore**. **ProductList** obtiene los productos del estado global y usa a **ProductCard** para mostrarlos.

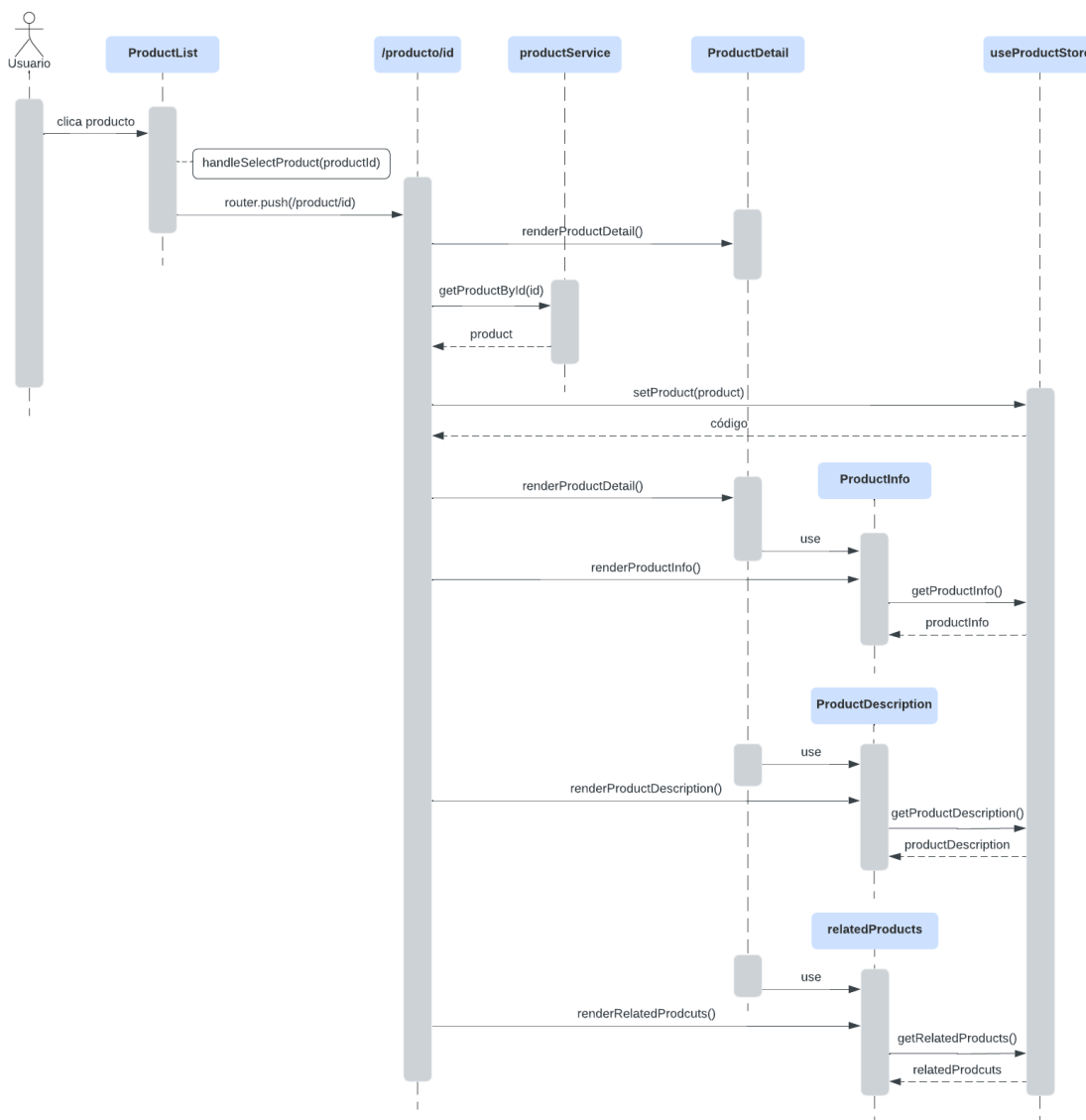


Figura 51: Diagrama de secuencia de producto concreto. Elaborado con Lucidchart.

Cuando el usuario clic en un producto mostrado de la búsqueda, **ProductList** llama a la función `handleSelectProduct(productId)`, que hace un `router.push(/product/id)` para navegar a la nueva ruta. Luego, `/product/id` coge el producto de **productService** mediante la función `getProductById(id)` y lo guarda en el store **useProductStore**. Finalmente, por cada subcomponente de **ProductDetail**, se renderiza y se coge la información necesaria del store para cada subcomponente.

10.4 Diseño del *back-end*

Una vez explicado el diseño del *front-end*, queda por describir el diseño del *back-end*. En primer lugar, se listarán los métodos públicos que ofrecerá la API. A continuación, se mostrará un diagrama de clases y, para finalizar, se presentará un ejemplo de diagrama de secuencia de una funcionalidad específica, siguiendo el mismo enfoque utilizado en el apartado anterior.

10.4.1 Métodos públicos

En esta sección se muestran los métodos públicos del sistema, separados por grupos. Se utilizan cuatro operaciones básicas: GET (para leer), POST (para crear), PUT (para editar) y DELETE (para borrar).

Cuenta:

Tipo	Método	Descripción
POST	/login	Inicia sesión y devuelve un token JWT ¹⁰ .
POST	/register	Registra un nuevo usuario.
POST	/logout	Invalida el token de sesión.
GET	/users	Obtiene todos los usuarios.
GET	/users/:id	Obtiene datos de un usuario específico.
PUT	/users/:id	Actualiza datos del usuario.
DELETE	/users/:id	Elimina un usuario.
GET	/checkEmail/:email	Comprueba si un email ya está registrado.
POST	/forgot-password	Envía un correo para resetear la contraseña.
PUT	/new-password	Cambia la contraseña con un token temporal.
POST	/pharmacists	Crear farmacéutico (admin)
PUT	/pharmacists/:id	Actualizar datos del farmacéutico.
DELETE	/pharmacists/:id	Eliminar un farmacéutico.
GET	/pharmacists	Ver a todos los farmacéuticos.

¹⁰ Un token JWT es un formato compacto y seguro para transmitir información entre partes en formato JSON [52].

GET	/pharmacists/:id	Ver farmacéutico en específico.
GET	/faq	Ver las FAQ del sistema.

Productos:

Tipo	Método	Descripción
GET	/products	Obtiene productos (con filtros, búsqueda semántica o ambos).
GET	/products/:id	Obtiene datos de un producto específico.
POST	/products	Crea un nuevo producto.
PUT	/products/:id	Actualiza los datos de un producto.
DELETE	/products/:id	Elimina un producto.
GET	/products/recommendations	Recomendaciones para el usuario.
POST	/products/recommendations/rate/:recommendationId	Valorar una recomendación.

Búsqueda:

Tipo	Método	Descripción
GET	/search-history/:userId	Historial de búsqueda de productos.
POST	/search-history/rate/:searchId	Valora una búsqueda

Recetas:

Tipo	Método	Descripción
POST	/prescription	Sube una receta electrónica o privada.
GET	/prescription/:id	Obtiene los detalles de una receta cargada.
POST	/prescription/validate	Valida una receta.
DELETE	/prescription/:id	Eliminar una receta.

Encargos:

Tipo	Método	Descripción
POST	/orders	Crea un nuevo encargo.
GET	/orders/:id	Obtiene la información de un encargo.
GET	/orders/:user	Obtiene todos los encargos de un usuario.
PUT	/orders/:id	Cambia el estado de un encargo.

Consultas farmacéuticas:

Tipo	Método	Descripción
POST	/queries	Crea una nueva consulta.
GET	/queries/:id	Obtiene la información de una consulta.
GET	/queries/:user	Obtiene todas las consultas de un usuario.
PUT	/queries/:id	El farmacéutico responde a la consulta.

10.4.2 Diagrama de clases

El diagrama de clases representa la estructura del sistema, dividido en tres capas principales: *Controller*, *Service* y *Repository*. Cada capa tiene clases específicas para cada entidad del sistema: como *Product*, *User*, *Order*, *Prescription* y *Query*. Las relaciones entre las clases se basan en el principio de inversión de dependencia, donde los *Controllers* dependen de los *Services*, y estos a su vez dependen de los *Repositories*. Este enfoque modular permite una fácil expansión y mantenimiento del sistema, además de promover la reutilización de código y la claridad en la estructura del proyecto.

En la Figura 52, se muestra el diagrama de clases del *back-end* con las funciones correspondientes a cada clase. La clase *Producto* y la clase *Usuario* incluyen también los parámetros de las funciones.

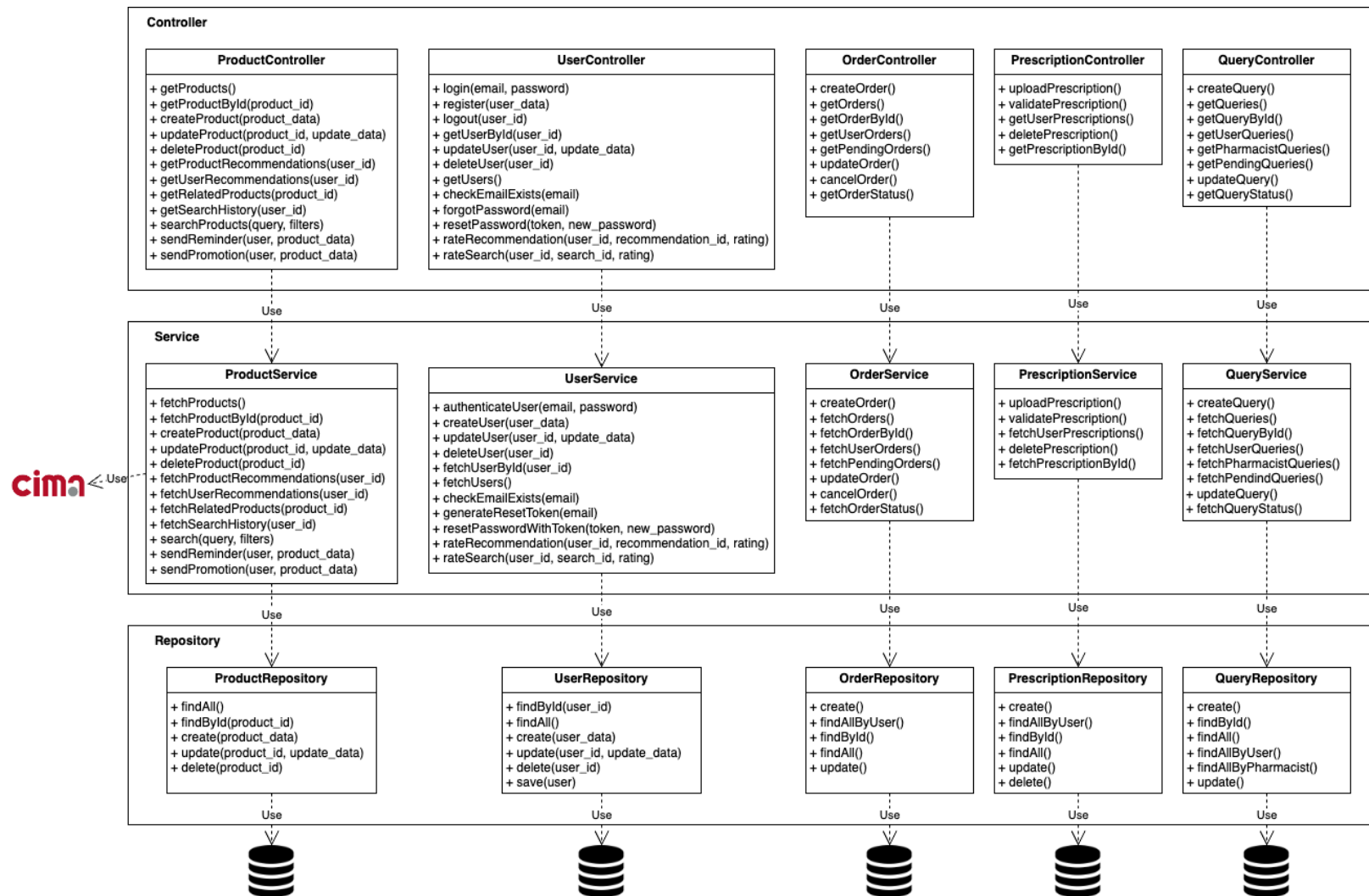


Figura 52: Diagrama de clases del back-end. Elaborado con Lucidchart.

10.4.3 Diagrama de secuencia

En la Figura 53, se puede observar el diagrama de secuencia de la petición de registrar a un usuario. En el proceso, el `UserController` recibe la solicitud `registerUser(bodyRequest)` con los datos del usuario y la envía a `UserService`. Este verifica si el correo ya está registrado llamando a `UserRepository.findByEmail(correo)`. Si el correo existe, lanza una excepción `EmailAlreadyInUseException` y devuelve un `409 Conflict`. Si el correo no está en uso, `UserService` crea un nuevo objeto `User(params)` y lo guarda en la base de datos. Finalmente, `UserService` retorna el resultado `201 Created` al `UserController`.

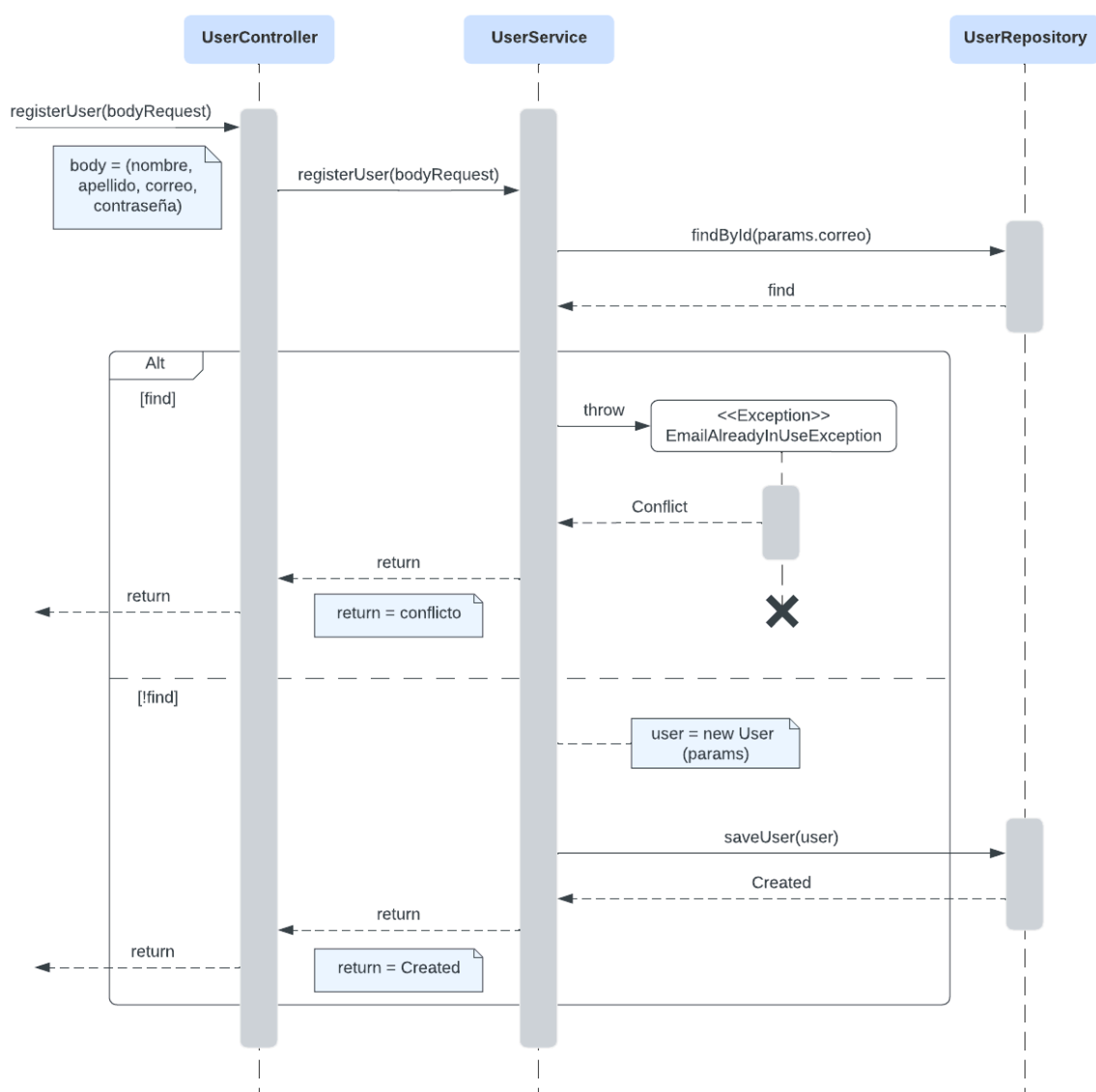


Figura 53: Diagrama de secuencia de registro de usuario. Elaborado con Lucidchart.

10.4.4 Esquema de la base de datos

Para finalizar el apartado de diseño, se muestra el esquema de la base de datos. En MongoDB, el diseño se basa en colecciones y documentos JSON anidados o referenciados, priorizando la eficiencia en consultas frecuentes y evitando joins complejos.

Se ha optado por una base de datos de documentos frente a una relacional ya que, al trabajar con una base NoSQL, se pueden agregar nuevos datos sin necesidad de definirlos previamente, lo que mejora el rendimiento y permite desarrollos ágiles con iteraciones rápidas y flexibles.

Las consultas a la base de datos se realizan desde los repositorios del *back-end*, que encapsulan el acceso a datos siguiendo el patrón repositorio. A modo de ejemplo, una consulta básica en Python para obtener un usuario por su correo electrónico sería:

```
Python
user = db.usuarios.find_one({"email": "usuario@ejemplo.com"})
```

Donde “db” es la base de datos, “usuarios” es la colección dentro de la base de datos y “find_one” es la función para obtener un usuario por un atributo.

A continuación, se detallan las diferentes colecciones con sus respectivos atributos.

Colección Persona

```
JavaScript
{
  _id: ObjectId,
  correo: { type: String, required: true, unique: true },
  nombre: { type: String, required: true },
  apellido: { type: String, required: true },
  contraseña: { type: String, required: true },
  rol: {
    type: String,
    required: true,
    enum: ["usuario", "farmacéutico", "administrador"]
  },
  newsletter: { type: Boolean, required: true }
}
```


Colección Producto

JavaScript

```
{
  _id: ObjectId,
  nregistro: { type: String, required: true, unique: true },
  nombre: { type: String, required: true },
  descripción: { type: String, required: true },
  precio: { type: Number, required: true },
  laboratorio: { type: String, required: true },
  categoría: { type: String, required: true },
  stock: { type: Number, required: true },
  imagen: { type: String, required: true },
  principiosActivos: { type: String, required: true },
  dosis: { type: String, required: true },
  prescripción: { type: Boolean },
  composición: { type: String },
  additionalInfo: { type: String },
  embedding: { type: [Number] }
}
```

Colección Encargo

JavaScript

```
{
  _id: ObjectId,
  usuario_id: { type: ObjectId, ref: "Persona", required: true },
  productos: [
    {
      producto_id: { type: ObjectId, ref: "Producto", required: true }
    }
  ],
  fecha: { type: Date, required: true },
  estado: {
    type: String,
    required: true,
    enum: ["Pendiente", "Listo para recoger", "Entregado"]
  },
  fechaEntrega: {
    type: Date,
    required: function() { return this.estado === "Entregado"; }
  },
  farmacéutico_id: { type: ObjectId, ref: "Persona" }
}
```

Colección ConsultaFarmacéutica

JavaScript

```
{
  _id: ObjectId,
  usuario_id: { type: ObjectId, ref: "Persona", required: true },
  farmacéutico_id: { type: ObjectId, ref: "Persona" },
  fecha: { type: Date, required: true },
  asunto: { type: String, required: true },
  pregunta: { type: String, required: true },
  respuesta: { type: String },
  estado: {
    type: String,
    required: true,
    enum: ["Pendiente", "Respondida"]
  }
}
```

Colección Valoración

JavaScript

```
{
  _id: ObjectId,
  date: { type: Date, required: true },
  type: { type: String, ref: "Producto", required: true },
  rate: { type: Number, min: 0, max: 1, required: true }
}
```

Colección Receta

JavaScript

```
{
  _id: ObjectId,
  usuario_id: { type: ObjectId, ref: "Persona", required: true },
  fechaEmision: { type: Date, required: true },
  fechaCaducidad: { type: Date, required: true },
  médico: { type: String, required: true },
  estadoReceta: {
    type: String,
    required: true,
    enum: ["Activa", "Caducada"]
  },
  tipo: {
    type: String,
    required: true,
  }
}
```

```

    enum: ["RecetaElectrónica", "RecetaPrivada"]
  },
  descuentoSS: {
    type: Number,
    required: function() { return this.tipo === "RecetaElectrónica"; }
  },
  productos: [
    {
      producto_id: { type: ObjectId, ref: "Producto", required: true }
    }
  ],
}

```

Colección HistorialDeBúsquedas

```

JavaScript
{
  _id: ObjectId,
  usuario_id: { type: String, required: true, ref: "Persona" },
  fecha: { type: Date, required: true },
  consulta: { type: String, required: true },
  embedding: { type: [Number] }
}

```

Colección Recordatorio

```

JavaScript
{
  _id: ObjectId,
  usuario_id: { type: ObjectId, ref: "Persona", required: true },
  fechaRecordatorio: { type: Date, required: true },
  productNregistro: { type: String, ref: "Producto", required: true },
  productName: { type: String, ref: "Producto", required: true }
}

```

11. Implementación

Se han fijado los objetivos que se deben alcanzar, se han definido los requisitos que el sistema debe cumplir, se ha representado el diseño que debe tener el sistema y se ha determinado la planificación y la metodología a seguir. Por tanto, ya es hora de comenzar con la fase de implementación.

11.1 Implementación del *front-end*

En esta sección se mostrará las diferentes funcionalidades implementadas en el *front-end* a lo largo de los *sprints*. Sin embargo, primero se va a mostrar la estructura de las carpetas para dar un mayor contexto. Dentro de la carpeta *src*, la estructura del *front-end* está organizada para mantener un código limpio y modular, como se muestra en la Figura 54.

- **App** contiene las páginas del proyecto siguiendo la convención de rutas de Next.js.
- **Components** se agrupan los componentes reutilizables de la interfaz.
- **Constants** almacena valores globales, como textos fijos o claves.
- **Hooks** incluye hooks personalizados que encapsulan lógica reutilizable.
- **Services** gestiona la comunicación con APIs externas.
- **Stores** maneja el estado global de la aplicación, como la autenticación o los productos.
- **Types** define los tipos TypeScript utilizados para mantener un tipado estricto y consistente.

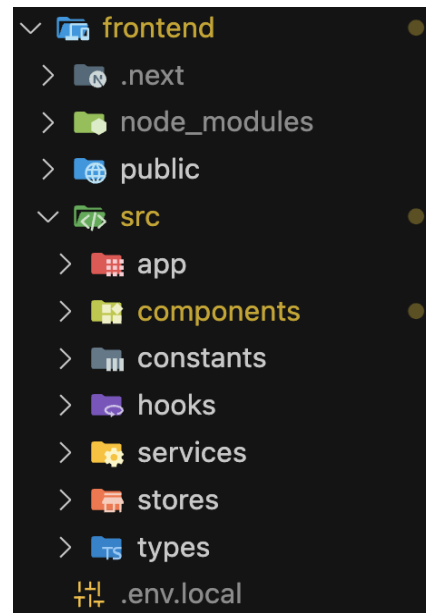


Figura 54: Carpetas del *front-end*.
Elaboración propia.

En la carpeta *stores*, se utiliza **Zustand** [53] para gestionar el estado global de forma eficiente. Se han creado *stores* específicos como *authStore* para la autenticación y *productStore* para almacenar información de productos, manteniendo la lógica separada y reutilizable.

11.1.1 Sprint 1

Durante este primer *sprint*, se han implementado varias funcionalidades enfocadas en la autenticación, la gestión de cuentas, la consulta del catálogo de productos y la posibilidad de aplicar filtros sobre este catálogo. Esto ha sentado las bases para ofrecer una experiencia más sólida a los usuarios y también para distinguir claramente los diferentes roles que pueden tener.

Se diseñaron y desarrollaron los formularios necesarios para que los usuarios puedan acceder al sistema de manera sencilla: registrarse, iniciar sesión y recuperar su contraseña. Además, se integró el inicio de sesión con Google, permitiendo a los usuarios autenticarse mediante su cuenta personal de Google. En ambos casos, una vez que el usuario se verifica, el sistema recibe del *back-end* un *token JWT* junto con datos importantes como su correo electrónico y su rol. Todos estos datos se guardan en el **authStore**, que se encarga de mantener la sesión activa y de que la información esté disponible durante toda la navegación.

Con el objetivo de proteger las rutas y componentes sensibles de la aplicación, se han creado dos componentes específicos:

- **authCheck**: Este componente tiene tres funcionalidades: verifica si el usuario está autenticado, comprueba si el *token JWT* ha expirado y, si sigue activo, actualiza el token automáticamente. Si el usuario no está autenticado o el token ha expirado, redirige a la página de inicio de sesión. En la Figura 55 puedes ver el código que se usa para actualizar el token.

```
const timeLeft = tokenExp - Date.now();
// Si faltan menos de 2 minutos y el usuario ha estado activo en los últimos 5 minutos
if (timeLeft < 2 * 60 * 1000 && Date.now() - lastActivityRef.current < 5 * 60 * 1000) {
  try {
    const newToken = await refreshToken(token!);
    const payload = JSON.parse(atob(newToken.split(".")[1]));

    setUser(newToken, payload.rol, payload.correo, payload.imagen, payload.nombre, payload.apellido);
  } catch (err) {
    logout();
    router.push("/login");
  }
}

// Programar la próxima verificación
const nextCheck = Math.max(timeLeft - 2 * 60 * 1000, 5000); // Mínimo 5s, cuando falten 2 min
timeoutRef.current = setTimeout(checkAuth, nextCheck);
```

Figura 55: Código de actualizar el token. Elaboración propia.

- **withAuth:** es un *higher-order component*¹¹ que encapsula componentes protegidos. Si el usuario no está autenticado o no tiene los roles adecuados, automáticamente lo envía a la página de login o a la página principal, según corresponda. La lógica de esto se explica en la Figura 56.

```
useEffect(() => {
  if (!isAuthenticated) {
    router.push("/login");
  } else if (!allowedRoles.includes(userRole!)) {
    router.push("/"); // Redirige a la página de inicio si no tiene permiso
  } else {
    setLoading(false);
  }
}, [isAuthenticated]);
```

Figura 56: Código de comprobar rol y autenticación de usuario. Elaboración propia.

Gracias a estos componentes, aseguramos que solo los usuarios que han iniciado sesión puedan acceder a las áreas privadas, que solo los roles permitidos puedan ver ciertas páginas y que la sesión se mantenga segura en todo momento.

Dependiendo del rol que detecte al inicio de sesión, el usuario será redirigido a su pantalla principal, diseñada específicamente para reflejar las funciones de cada tipo de usuario.

En el caso del administrador, se ha implementado una interfaz para consultar y gestionar los usuarios registrados, incluyendo tanto usuarios como farmacéuticos. Desde esta misma vista, es posible eliminar cuentas no deseadas. Los farmacéuticos también pueden visualizar la lista de usuarios que gestionan.

En el caso del usuario, se ha diseñado la interfaz de la página de tienda, centrada en facilitar la exploración y navegación por el catálogo de productos. Al ingresar por primera vez, la aplicación utiliza la función `fetchProducts` para obtener una lista predeterminada de productos desde el backend, asegurando que el usuario vea contenido desde el primer momento sin necesidad de realizar búsquedas manuales. Esta función se muestra en la Figura 57.

¹¹ Un High-Order Component (HOC) es una función que toma un componente y devuelve uno nuevo con funcionalidad añadida, como lógica reusable

```
// Función para obtener productos predeterminados
const fetchProducts = async () => {
  try {
    const filteredProducts = await getFilteredProducts({prescription: false, limit: 30});
    setProducts(filteredProducts);
    handleSortChange('sin-prescripcion', filteredProducts);
    clearProducts();
    clearSearchQuery();
  } catch (error) {
    toast.error('Error al obtener los productos');
  } finally {
    setLoading(false);
  }
};
```

Figura 57: Código de obtener productos iniciales. Elaboración propia.

La interfaz cuenta con un sistema de filtros que ayuda al usuario a mejorar la búsqueda, permitiendo refinar los resultados según diferentes criterios. Puedes filtrar por laboratorio, categoría farmacéutica, por prescripción y por rango de precios. Estos filtros se combinan, de modo que cada vez que aplicas una condición, los resultados se actualizan en tiempo real. La Imagen de la Figura 58 muestra cómo son estos filtros.

```
try {
  const filtersQuery = buildFiltersQuery(newFilters);
  if (Object.keys(filtersQuery).length === 0) return;

  // si hay productos en el store, aplica filtros localmente
  const filteredProducts = await getFilteredProducts(
    filtersQuery,
    productsStore.length > 0 ? productsStore : undefined
  );

  setProducts(filteredProducts);
  handleSortChange('sin-prescripcion', filteredProducts);

  clearProducts();
  setProductsStore(filteredProducts);
} catch (error) {
  toast.error("Error al aplicar filtros");
}
```

Figura 58: Código de filtros para búsquedas. Elaboración propia.

Seguidamente, se desarrolló la página de detalle del producto, en la que se decidió mostrar sólo la información más relevante y fácil de entender para quienes la usan: una breve descripción, la composición y un link directo al prospecto oficial en PDF.

Para hacer que todo funcione mejor y que la experiencia sea más agradable, se implementó un store específico para los productos. Este store guarda los productos que se muestran en ese momento, así se evita hacer solicitudes innecesarias a la base de datos y los filtros permanecen activos mientras navegas.

11.1.2 Sprint 2

Durante el segundo *sprint* se avanzó en funcionalidades centradas principalmente en la personalización del perfil de usuario, la implementación de un sistema de búsqueda semántica avanzada y la gestión de recetas.

Se desarrolló una página dedicada al perfil del usuario autenticado, accesible desde el menú de navegación. En esta página, los usuarios pueden cambiar su nombre, apellidos, correo electrónico y subir una foto de perfil. Sin embargo, si alguien se registró con Google, el campo del correo está bloqueado para evitar cambios. Además, los usuarios tienen la opción de eliminar su cuenta cuando quieran.

Un gran avance fue integrar la búsqueda semántica en la tienda. Se diseñó toda la lógica para que, cuando alguien escribe en la barra, la consulta se envía al *back-end*. Desde allí, se recibe una lista de productos que se parecen semánticamente a lo que busca la persona, considerando la descripción y las propiedades de cada uno. También se devuelve el *embedding* de la consulta, que es útil en otros aspectos del sistema.

Esta función de búsqueda semántica funciona muy bien junto con los filtros ya existentes, y los resultados se ajustan automáticamente a la combinación de ambos criterios: la similitud semántica y los filtros clásicos.

Seguidamente, se añadió un sistema de historial de búsqueda. Cada vez que un usuario autenticado realiza una búsqueda, esa información se guarda en la base de datos junto con su *embedding*. Esto no solo permite que los usuarios puedan ver fácilmente su historial desde su perfil, sino que también abre la puerta a futuras funciones, como ofrecer recomendaciones personalizadas.

En la Figura 59 se muestra como se recogen los productos y el *embedding* del *back-end* de la búsqueda, y cómo se guarda la búsqueda tanto en la base de datos como en el *productStore*.


```

const handleSearch = async (searchTerm: string) => {
  try {
    router.replace(`/shop?search=${encodeURIComponent(searchTerm)}`);
    const { products, embedding } = await getSearchResults(searchTerm);
    setProducts(products);
    handleSortChange("sin-prescripcion", products);

    // guardar busqueda en la base de datos
    if(userCorreo && isAuthenticated) {
      setSearchData({
        searchTerm: searchTerm,
        date: new Date(),
        user: userCorreo,
        embedding: embedding ?? null
      });
    }

    clearProducts();
    setProductsStore(products);
    setSearchBaseProducts(products);
    handleSortChange(sortOption, products);
    // guardar busqueda en el store
    setSearchQuery({
      searchTerm: searchTerm,
      date: new Date(),
      user: userCorreo ?? null,
      embedding: embedding ?? null
    });
  } catch (error) {
    toast.error("Error al buscar productos");
  } finally {
    setLoading(false);
  }
};

```

Figura 59: Código de búsqueda semántica en el front-end. Elaboración propia.

Por último, se desarrolló la sección de gestión de recetas, una funcionalidad esencial para facilitar a los usuarios la tramitación digital de sus prescripciones médicas.

Se creó una página informativa pública accesible desde la interfaz principal, donde se explica al usuario cómo funciona el sistema de recetas dentro de la plataforma. En esta sección se detallan los pasos para subir una receta y se proporcionan enlaces que dirigen a la sección privada correspondiente: “Mis recetas”.

En el apartado privado “Mis recetas”, visible solo para usuarios autenticados, se ofrece una interfaz completa para la gestión personal de recetas. Desde esta vista, el usuario puede subir nuevas recetas (formato PDF), así como consultar el historial de recetas subidas, visualizando los detalles clave como: Tipo de receta (electrónica o privada), estado (activa o caducada), fecha de emisión y expiración, nombre del médico prescriptor y productos incluidos en la receta.

Una vez procesada la receta, el sistema busca en la base de datos si existe una coincidencia exacta con los productos prescritos. En caso afirmativo, se muestra directamente el producto correspondiente. Si no se encuentra una coincidencia exacta, el sistema ofrece hasta tres alternativas similares basadas en el principio activo y la descripción del medicamento. En la Figura 60 se muestra cómo se maneja la subida de una receta.

```
const handleFileChange = async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file || !currentType) return;
  if (file.size > 5 * 1024 * 1024) {
    toast.error("El archivo es demasiado grande. Máximo 5MB.");
    return;
  }

  const validTypes = ["application/pdf"];
  if (!validTypes.includes(file.type)) {
    toast.error("Formato no válido. Usa .pdf");
    return;
  }

  try {
    const user = userCorreo ?? "";
    const newPrescription = await uploadPrescription(user, currentType, file);
    console.log("Receta subida:", newPrescription);
    toast.success("Receta subida correctamente.");
    setPrescriptions((prev) => [...prev, newPrescription]);
  } catch (error) {
    toast.error("Hubo un problema subiendo la receta.");
  } finally {
    // limpiar input para permitir subir el mismo archivo otra vez
    if (fileInputRef.current) fileInputRef.current.value = "";
    setCurrentType(null);
  }
};
```

Figura 60: Código de manejo de subir receta. Elaboración propia.

Cada producto sugerido puede ser explorado en detalle mediante un enlace a su página individual. Además, se ha preparado la infraestructura para permitir al usuario añadir el producto al carrito desde esta misma vista, aunque esta funcionalidad aún no ha sido implementada en esta fase.

11.1.3 Sprint 3

Durante este sprint se han desarrollado funcionalidades orientadas a mejorar la personalización, la experiencia de compra y la gestión de encargos, tanto desde la perspectiva del usuario como del farmacéutico y el administrador.

Se ha implementado un sistema de recomendaciones personalizadas para los usuarios, basado en su historial de navegación y búsqueda. En la página principal del usuario, se muestran hasta diez productos recomendados, con el objetivo de facilitar el descubrimiento de medicamentos relevantes para sus necesidades.

Además, en la página de detalle de producto, se ha añadido un módulo de productos relacionados. Este componente muestra tres productos similares al que el usuario está visualizando, utilizando criterios como el principio activo, con el fin de fomentar la exploración de alternativas.

También se ha desarrollado el sistema de valoraciones para evaluar tanto las recomendaciones personalizadas como los resultados de búsqueda. Esta funcionalidad está disponible para usuarios logueados y no logueados, y permite enviar una valoración sobre la utilidad de la recomendación o búsqueda. Los resultados se reflejan en forma de gráficas accesibles para el administrador, permitiendo así obtener métricas sobre la calidad del sistema de personalización.

En cuanto a la experiencia de compra, se ha implementado la funcionalidad del carrito, que permite al usuario añadir productos desde diferentes secciones: la tienda, la página de detalle del producto y también desde la sección de recetas. Se ha creado una página específica del carrito, donde el usuario puede revisar los productos añadidos, modificar cantidades, dejar notas adicionales y aplicar códigos promocionales.

Desde la misma página del carrito, se ha integrado la opción de finalizar el encargo. Al confirmar el encargo, se envía automáticamente un correo de confirmación al usuario y otro correo al farmacéutico, con un resumen detallado de los productos solicitados y la información correspondiente.

Por otro lado, se ha desarrollado la página pública de encargos, en la que se explica su funcionamiento y utilidad, y la página privada donde el usuario puede consultar todos sus encargos anteriores y actuales, incluyendo detalles como los productos solicitados, el estado del encargo y las fechas relevantes.

Finalmente, se ha completado la interfaz de gestión de encargos para farmacéuticos. En esta sección, el farmacéutico puede visualizar los encargos que le han sido asignados y actualizar su estado. Cada cambio de estado genera una notificación por correo al usuario, y también queda reflejado en la página de consulta de encargos, permitiendo un seguimiento claro del proceso. En la Figura 61 se muestra cómo se maneja el cambio de estado.

```
const handleStatusChange = async (id: string, newStatus: string) => {
  try {
    await updateOrderStatus(id, newStatus);
    toast.success("Usuario notificado correctamente");
    setOrders((prev) =>
      prev.map((order) =>
        order.id === id ? { ...order, status: newStatus } : order
      )
    );
  } catch (err) {
    console.error("Error al actualizar el estado:", err);
  }
};
```

Figura 61: Código de manejo de cambio de estado. Elaboración propia.

11.1.4 Sprint 4

Durante el *sprint* 4 se completaron las funcionalidades finales del *front-end*, centradas en mejorar la experiencia del usuario y dotar al sistema de herramientas administrativas y comunicativas.

En primer lugar, se desarrolló la página de contacto, que incluye un formulario para que los usuarios puedan realizar consultas farmacéuticas. Estas consultas se asignan automáticamente a un farmacéutico disponible de forma aleatoria, permitiendo una gestión eficiente de las peticiones. Además, se implementó el historial de consultas del usuario, donde este puede revisar todas las consultas realizadas anteriormente y su estado actual. En la Figura 62 se muestra cómo se envía una consulta.

```
const handleSubmit = async (subject: string, message: string) => {
  if (!subject || !message) return;
  const pharmacist = await choosePharmacist();
  const query: Query = {
    user: userCorreo || "",
    pharmacist: pharmacist,
    date: new Date().toISOString(),
    subject,
    question: message,
    answer: null,
    status: QueryStatus.PENDING,
  };
  try {
    await sendQuery(query);
    const updatedQueries = await getQueriesByUser(userCorreo || "");
    setQueries(updatedQueries);
    toast.success("Consulta enviada con éxito");
  } catch (error) {
    toast.error('Error al enviar la consulta');
  }
};
```

Figura 62: Código de envío de consulta. Elaboración propia.

Seguidamente, se desarrolló una vista específica para los farmacéuticos, donde pueden visualizar las consultas que les han sido asignadas. Esta interfaz incluye un formulario para responder a las preguntas del usuario y actualizar el estado de cada consulta según corresponda.

Se incorporó también una sección destinada a la gestión de usuarios por parte del personal farmacéutico, permitiéndoles consultar información de los usuarios. Asimismo, el administrador dispone de una vista desde la que puede gestionar tanto a los usuarios como a los farmacéuticos registrados en el sistema.

Otra de las funcionalidades implementadas en este *sprint* fue el sistema de notificaciones personalizadas, que se divide en dos tipos: recordatorios y newsletter.

Para los recordatorios, se añadió una opción en la página de cada producto que permite al usuario programar un aviso. Este aviso consiste en un correo electrónico que se enviará en la fecha seleccionada por el usuario. Todos los recordatorios creados pueden ser consultados desde la zona privada del usuario. En la Figura 63 se muestra cómo se añade un recordatorio.

```
const handleSetReminder = async () => {  
  if (!reminderDate) { ...  
  }  
  if (!userCorreo) { ...  
  }  
  const date = new Date(reminderDate).toISOString();  
  const reminder = {  
    user: userCorreo,  
    productName: product.name,  
    productNregistro: nregistro,  
    date: date,  
    sent: false,  
  };  
  
  try {  
    await saveReminder(reminder);  
    toast.success("Recordatorio añadido");  
    handleCloseReminder();  
  }  
  catch (error) {  
    toast.error("Error al guardar el recordatorio");  
  }  
};
```

Figura 63: Código de añadir un recordatorio. Elaboración propia.

Respecto a la newsletter, se integró en el pie de página (*footer*) un formulario de suscripción. Si el usuario se suscribe, recibirá automáticamente un correo electrónico cada lunes a las 10:00 h con promociones o novedades relacionadas con la farmacia.

En cuanto a la administración de productos, se habilitó una sección exclusiva para el administrador, desde la cual puede añadir nuevos productos, editar su información o eliminarlos si es necesario. Esta gestión se restringe únicamente a los productos creados desde esta misma sección.

Por último, se implementaron dos páginas estáticas: una sección de preguntas frecuentes (FAQ), donde se resuelven las dudas comunes de los usuarios, y una página informativa “Sobre nosotros”, donde se describe brevemente la misión y el equipo de la farmacia.

11.2 Implementación del *back-end*

Anteriormente, se ha mostrado la estructura de carpetas del *front-end*. En esta sección cómo está organizado el *back-end* y las funcionalidades implementadas en cada uno de los *sprints*. La estructura del *back-end* sigue una arquitectura en capas bien organizada dentro del directorio *app*, como muestra la Figura 64.

- **Auth:** manejo de autenticación y autorización, como el manejo de credenciales, la generación y validación de tokens JWT.
- **Config:** configuración general del proyecto (como conexión a la base de datos).
- **Controllers, services, repositories:** separación clara de responsabilidades para manejar lógica de negocio, acceso a datos y controladores de rutas.
- **Routes:** define las rutas expuestas por la API.
- **Models y constants:** para modelos de datos y constantes globales.
- **Utils y tests:** para *scripts* reutilizables y pruebas, respectivamente.
- **Main.py:** ejecuta la aplicación *FastAPI*.

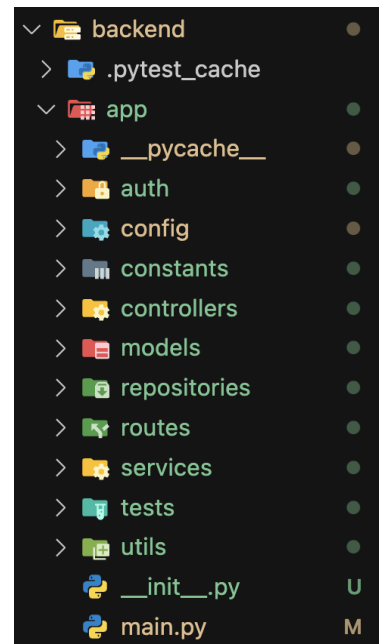


Figura 64: Carpetas del *back-end*.
Elaboración propia.

Al final de cada *sprint*, se han hecho los tests pertinentes.

11.2.1 Sprint 1

En el *sprint* 1 se inició el proyecto con el entorno de trabajo configurado. Durante la fase de *inception*, se implementó MongoDB como base de datos para gestionar los datos, y se desplegó *FastAPI* como framework backend para crear APIs rápidas y eficientes.

Primero de todo se hizo la lógica para crear usuarios. El *user_controller* recoge los datos enviados desde el *front-end* mediante el *endpoint* */users/register*. Este se los pasa al *user_service*, dónde verifica si el usuario ya existe en la base de datos. Si no existe, encripta la contraseña y le pasa los datos al *user_repository*. Además, el *service* genera el token del usuario para la sesión.

El proceso para iniciar sesión es similar, lo único que si el usuario existe en la base de datos y la contraseña que ha puesto el usuario coincide con la respectiva en la base de datos, se autoriza el login.

En la Figura 65, se muestra cómo se crea el usuario en la base de datos desde el repository.

```
def create_user(self, user: UserCreate, hashed_password: str):
    new_user = {
        "correo": user["correo"],
        "nombre": user["nombre"],
        "apellido": user["apellido"],
        "rol": user["rol"],
        "imagen": user["imagen"],
        "fromGoogle": user["fromGoogle"],
        "contraseña": hashed_password,
    }
    PERSONA_DB.insert_one(new_user)
    return new_user
```

Figura 65: Código de crear usuario. Elaboración propia.

Seguidamente, se implementó la autenticación con Google, integrando sus credenciales OAuth2 para acceder al sistema. Se extrajeron los datos esenciales del perfil del usuario (nombre, apellidos, correo e imagen de perfil) para personalizar la experiencia y gestionar la identificación dentro de la plataforma. En la Figura 66 se muestra el código.

```
async def handle_google_callback(request: Request, user_service: UserService):
    """Procesa la respuesta de Google y maneja el login o registro."""
    token = await oauth.google.authorize_access_token(request)
    id_token = token.get("id_token")
    if not id_token:
        raise ValueError("El token devuelto por Google no contiene un id.")

    google_user = jwt.decode(
        id_token,
        key=None,
        audience=GOOGLE_CLIENT_ID,
        access_token=token.get("access_token"),
    )

    nombre = google_user.get("given_name", "")
    apellido = google_user.get("family_name", "")
    correo = google_user.get("email", "")
    imagen = google_user.get("picture", "")

    existing_user = user_service.get_user_by_email(correo)
```

Figura 66: Código autenticación con Google. Elaboración propia.

Para acabar con esta parte de usuarios, se hizo la lógica de recuperar la contraseña. El sistema generaba un código de 6 dígitos en el back-end y lo enviaba por email al usuario, junto con un token JWT que contenía el correo, el código y su tiempo de expiración. Cuando el usuario introducía el código, el sistema validaba la información decodificando el token y comparando los datos con los proporcionados por el usuario. En la Figura 67 se muestra cómo se envía este correo al usuario.

```
def send_email(to_email: str, subject: str, content: str):
    """
    Envía un correo electrónico utilizando Resend.
    """
    try:
        response = resend.Emails.send({
            "from": "EcoFarma <notificaciones@ecofarmabcn.forum>",
            "to": [to_email],
            "subject": subject,
            "html": content,
        })

        if response.get("id"):
            print(f"Correo enviado a {to_email} con ID {response['id']}")
            return 200 # Éxito
        else:
            print(f"Fallo en el envío del correo: {response}")
            return 500

    except Exception as e:
        print(f"Error al enviar el correo con Resend: {e}")
        raise
```

Figura 67: Código de envío de correo con Resend. Elaboración propia.

El siguiente paso era poblar la base de datos de productos de CIMA. El proceso comenzó extrayendo los números de registro de los medicamentos disponibles, mediante la función `get_products_by_conditions(conditions: dict)`. Los registros obtenidos se almacenaron inicialmente en un archivo JSON para facilitar su posterior procesamiento.

A continuación, se realizó una llamada individual a la API de CIMA para cada número de registro mediante la función `get_product_by_nregistro(medicine_nregistro: str)`. Esta función permite recuperar todos los detalles específicos de un producto farmacéutico.

Una vez obtenida la información detallada de cada producto, se extrajeron los campos necesarios y se insertaron en la base de datos del sistema. Sin embargo, la API de CIMA no proporciona ciertos datos clave para el funcionamiento de la tienda, como el precio y el stock. Por esta razón, se desarrolló un script adicional que añadía estos campos a cada producto de manera aleatoria.

El último paso consistía en generar un embedding vectorial para cada producto. Inicialmente se utilizó el modelo *nomic-embed-text* de Ollama [54], dado que ofrecía buenos resultados y podía ejecutarse localmente. No obstante, su uso en producción iba a ser problemático debido a la complejidad de su integración y los requerimientos de infraestructura. Por este motivo, se optó por una alternativa más ligera y fácil de desplegar: el modelo *all-MiniLM-L6-v2* de Sentence Transformers [55].

El embedding final de cada producto se generó combinando los siguientes campos: el nombre del medicamento, el principio activo y la descripción. Esta información se concatenó en un único texto, que fue transformado en vector utilizando Sentence

Transformers. No puedo poner el código de cada paso mencionado anteriormente, ya que quedaría una sección muy larga y sería poco legible. De esta forma, solo voy a poner el código del script para generar el embedding y ponerlo en el producto de la base de datos, tal y como se muestra en la Figura 68.

```
# Función para generar el texto combinado para el embedding
def generar_texto_para_embedding(producto):
    partes = [
        producto.get("name", ""),
        f"Principios activos: {producto.get('principioAct', '')}",
        producto.get("description", "")
    ]
    return " ".join(p for p in partes if p).strip()

# Recorremos todos los productos y actualizamos su embedding
for producto in collection.find():
    texto = generar_texto_para_embedding(producto)
    try:
        embedding = generate_embedding_modelProduct(texto)

        # Actualizamos en MongoDB
        collection.update_one(
            {"_id": producto["_id"]},
            {"$set": {"embedding": embedding}}
        )
        print(f"✓ Embedding actualizado para producto {producto['nregistro']}")
    except Exception as e:
        print(f"✗ Error con producto {producto.get('nregistro', 'N/A')}: {e}")
```

Figura 68: Código de generación de embeddings. Elaboración propia.

De esta forma estaba todo listo para devolver los productos al front-end mediante las funciones `get_product_by_nregistro()` y `get_products_by_filters()`.

11.2.2 Sprint 2

Durante este *sprint*, se implementó la funcionalidad de actualizar la información del usuario en la base de datos, según la información proporcionada desde el front-end. Además, se implementó la lógica para gestionar imágenes de perfil de usuario utilizando AWS S3 como sistema de almacenamiento. Cuando un usuario actualiza su foto, el backend recibe la imagen, la valida y la sube a un bucket específico en S3, obteniendo una URL pública que se guarda en la base de datos para su uso posterior. Si ya existía una imagen anterior, esta se elimina del bucket antes de subir la nueva, evitando así archivos innecesarios.

En la Figura 69 se muestra la función para subir una imagen al S3.

```
def upload_image_to_s3(file: UploadFile) -> str:
    """
    Sube una imagen a AWS S3 y devuelve la URL pública del archivo.
    """
    file_extension = file.filename.split(".")[-1]
    file_key = f"user-images/{uuid.uuid4()}.{file_extension}"

    s3_client.upload_fileobj(
        file.file, BUCKET_NAME, file_key, ExtraArgs={"ACL": "private"}
    )

    return f"https://{BUCKET_NAME}.s3.amazonaws.com/{file_key}"
```

Figura 69: Código de subir una imagen a S3. Elaboración propia.

Para implementar la búsqueda semántica de productos, se realizó un proceso de fine-tuning¹² partiendo del modelo base *all-MiniLM-L6-v2*, especializado en generar embeddings de frases. Este modelo fue ajustado mediante un *dataset* propio compuesto por tripletas [56] en formato (query, positivo, negativo), donde se relacionaban preguntas frecuentes o síntomas con el medicamento adecuado y uno no relacionado. Por ejemplo, una tripleta sería: “me duele la cabeza”, “GELOCATIL 650 mg COMPRIMIDOS” (positivo), y “PRITORPLUS 40 mg/12,5 mg COMPRIMIDOS” (negativo). Este entrenamiento permitió al modelo comprender mejor la relación entre descripciones naturales y productos farmacéuticos reales. En la Figura 70 se muestra el código para el fine-tuning.

```
# 1. Cargar datos
df = pd.read_csv("query_product_triplets.csv")

# 2. Convertir a InputExample con tres textos (anchor, positive, negative)
train_examples = [
    InputExample(texts=[row["query"], row["positive"], row["negative"]])
    for _, row in df.iterrows()
]

# 3. Modelo base
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# 4. Preparar dataloader y función de pérdida
train_dataloader = DataLoader(train_examples, shuffle=True, batch_size=16)
train_loss = losses.TripletLoss(model=model)

# 5. Entrenar modelo
model.fit(
    train_objectives=[(train_dataloader, train_loss)],
    epochs=5,
    warmup_steps=10,
    show_progress_bar=True
)

# 6. Guardar modelo
model.save("modelo_productos_triplet_finetuned")
```

Figura 70: Código de fine-tuning productos. Elaboración propia.

¹² Fine-tuning es el proceso de ajustar un modelo preentrenado de machine learning en un conjunto de datos específico para adaptarlo a una tarea particular.

Primero, configura un `DataLoader` que carga los datos de entrenamiento en lotes de 16 muestras, aplicando mezcla aleatoria (`shuffle=True`) para mejorar el aprendizaje. Luego, define una función de pérdida `TripletLoss`, que optimiza las distancias entre *embeddings*, acercando muestras similares y alejando las diferentes. El entrenamiento se realiza durante 5 épocas completas, con 10 pasos de calentamiento inicial (`warmup_steps`) para ajustar gradualmente la tasa de aprendizaje inicial.

Una vez entrenado, el modelo se utilizó en el backend para procesar las consultas que llegan desde el buscador del frontend. Cada vez que un usuario realiza una búsqueda, se genera un *embedding* de la query utilizando el modelo fine-tuned. Luego, este *embedding* se compara contra los *embeddings* previamente almacenados de todos los productos en la base de datos utilizando la funcionalidad de búsqueda vectorial¹³ de MongoDB. De este modo, se obtienen los productos más semánticamente relevantes a la intención del usuario.

En la Figura 71 se muestra la función `search_by_vector`. Se usa el operador `$vectorSearch` de MongoDB que permite realizar búsquedas por similitud vectorial utilizando varios parámetros clave: el `queryVector` representa el vector de referencia que se usará para encontrar documentos similares; el campo `path` indica dónde están almacenados los vectores dentro de los documentos; `numCandidates` define el número inicial de documentos candidatos que se evaluarán, lo que afecta al equilibrio entre precisión y rendimiento; `limit` establece el número máximo de resultados devueltos; y finalmente `index` especifica el nombre del índice predefinido que optimiza la búsqueda vectorial. En el campo `$project` se ponen los campos que se quieren devolver del producto.

```
def search_by_vector(self, embedding: List[float], limit: int = 30):
    pipeline = [
        {
            "$vectorSearch": {
                "queryVector": embedding,
                "path": "embedding",
                "numCandidates": 300,
                "limit": limit,
                "index": "vector_index"
            }
        },
        {
            "$project": { ...
        }
    ]
    return list(PRODUCTO_DB.aggregate(pipeline))
```

Figura 71: Código de búsqueda vectorial. Elaboración propia.

¹³ La búsqueda vectorial en MongoDB permite encontrar documentos similares comparando vectores numéricos mediante algoritmos de similitud.

Para la validación de recetas médicas, se entrenó un segundo modelo fine-tuned partiendo también de *all-MiniLM-L6-v2*, esta vez utilizando un enfoque de redes siamesas [57]. Las redes siamesas se entrenan con pares de textos para aprender a medir su similitud: en este caso, un texto era el extraído de una receta electrónica pública del Sistema de Salud (SS) y el otro correspondía al contenido de una receta privada. El objetivo era que el modelo aprendiera a identificar cuándo dos textos representan el mismo tipo de receta aunque tuvieran diferencias en formato o redacción.

Tras entrenar el modelo, se generaron dos embeddings base, uno representando recetas electrónicas públicas y otro para recetas privadas. Para ello, se utilizó un ejemplo real de texto extraído de una receta electrónica (obtenido de recetas propias reales) y, para las recetas privadas, un ejemplo de receta electrónica privada digital de Adeslas [58]. Cabe destacar que este modelo solo es válido para recetas privadas digitales, pero gracias a la regulación vigente sobre el formato obligatorio de las recetas privadas en España [59], el sistema es compatible con recetas emitidas por otras aseguradoras o médicos privados que respeten dicho formato.

Estos embeddings base se almacenaron en la base de datos y se utilizan como referencia para validar las recetas que suben los usuarios: cuando se sube una receta, se extrae su texto mediante *pdfplumber* [60], se genera su embedding con el modelo fine-tuned y se compara con los embeddings base para determinar si se trata de una receta válida y de qué tipo. En la Figura 72 se muestra el proceso de subida de una receta. La similaridad se calcula con $\text{similarity} = 1 - \text{cosine}(\text{embedding1}, \text{embedding2})$, y si devuelve menos de 0.85 se declara como no válida.

```
def process_prescription_upload(self, file, type):
    texto = extract_text_from_file(file)
    print("Texto extraído:", texto)
    embedding = generate_embedding_modelReceipt(texto)

    # Recuperar embedding base del tipo
    base_embedding = self.get_base_embeddings_by_type(type)

    if not base_embedding or not isinstance(base_embedding, list):
        raise ValueError("No se pudo recuperar embedding base")

    # Validar contra embedding base
    valid = validate_prescription(embedding, base_embedding)

    return valid, texto
```

Figura 72: Código de proceso de subida de receta. Elaboración propia.

11.2.3 Sprint 3

En este *sprint*, para el sistema de recomendaciones personalizadas, se ha desarrollado un mecanismo basado en el historial de búsqueda del usuario. Concretamente, se obtienen los últimos cinco términos buscados por el usuario y sus respectivos embeddings, y se calcula un embedding medio que representa su interés general reciente. A partir de este vector medio, se realiza una búsqueda vectorial en la base de datos de productos para recuperar las diez recomendaciones más relevantes. En la Figura 73 se muestra la lógica para las recomendaciones.

```
def get_recommendations(self, user: str) -> List[dict]:
    search_history = self.get_search_history_by_user(user)
    # Si no hay historial suficiente, devolver una lista vacía
    if not search_history or len(search_history) < 3: return []

    # Ordenar el historial por fecha descendente
    search_history_sorted = sorted(search_history, key=lambda x: x["date"], reverse=True)
    recent_searches = search_history_sorted[:5]

    embeddings = [
        np.array(search["embedding"])
        for search in recent_searches
    ]

    average_embedding = np.mean(embeddings, axis=0)
    average_embedding = average_embedding.tolist()

    recommendations = self.product_repo.search_by_vector(average_embedding, 10)

    return recommendations
```

Figura 73: Código de recuperación de recomendaciones. Elaboración propia.

En cuanto a los productos similares, se utiliza el embedding asociado al producto actualmente visualizado en la página de detalle. Este embedding se emplea para realizar una consulta vectorial, recuperando tres productos cercanos en el espacio semántico, lo cual garantiza que las sugerencias estén contextualizadas según el producto consultado.

Respecto a las valoraciones de búsquedas y recomendaciones, se ha diseñado un sistema que guarda estos datos en la base de datos diferenciando el tipo de valoración. Esta distinción facilita el posterior análisis desde el *front-end* y la generación de estadísticas para el administrador.

La funcionalidad de finalización de encargos ha sido implementada con un flujo que, al confirmar un encargo desde el front-end, registra en la base de datos toda la información del pedido y envía un correo de confirmación al usuario y al farmacéutico, conteniendo el resumen del encargo.

Se han creado los endpoints necesarios para la gestión de encargos, tanto para los usuarios como para los farmacéuticos. Los usuarios pueden consultar el historial y estado de sus encargos, mientras que los farmacéuticos pueden acceder únicamente a

los encargos asociados a ellos, garantizando el control de acceso mediante los roles definidos.

Por último, se ha desarrollado la lógica para actualizar el estado de un encargo, función accesible sólo por el farmacéutico. Cuando se actualiza el estado, el sistema envía automáticamente una notificación por correo al usuario informando del cambio. En la Figura 74 se muestra el código para actualizar el estado.

```
def update_order_status(self, order_id: str, status: str):
    order = self.order_repo.update_order_status(order_id, status)
    if not order:
        raise HTTPException(
            status_code=404,
            detail="El encargo no existe"
        )
    order.date = datetime.fromisoformat(order.date.replace("Z", "")).strftime("%d/%m/%Y")
    order.pickupDate = datetime.fromisoformat(order.pickupDate.replace("Z", "")).strftime("%d/%m/%Y")

    self.send_email_change_status(order)
    return order
```

Figura 74: Código para actualizar el estado del encargo. Elaboración propia.

11.2.4 Sprint 4

En este último *sprint*, se ha implementado la lógica para almacenar en la base de datos todas las consultas realizadas por los usuarios desde el formulario correspondiente. Cada vez que un usuario envía una consulta, el sistema asigna automáticamente un farmacéutico disponible y se envía una notificación por correo electrónico al profesional asignado, informándole de la nueva solicitud pendiente.

Cuando el farmacéutico responde a la consulta, su respuesta queda registrada en la misma entrada de la base de datos, actualizando así el estado de la misma. Además, el sistema envía un correo electrónico al usuario para informarle de que su consulta ha sido respondida y puede consultarla en su historial.

Para la funcionalidad de notificaciones automáticas, tanto en los recordatorios personalizados como en el envío periódico de newsletters, se utilizó el componente *BackgroundScheduler* [61], que permite la ejecución de tareas en segundo plano de forma programada. En el caso de los recordatorios, el sistema revisa diariamente si existe algún recordatorio pendiente cuya fecha sea igual o anterior a la fecha actual, y en ese caso se envía el correspondiente correo al usuario. En cuanto a la newsletter, se programó el envío automático de correos cada lunes a las 10:00 h a todos los usuarios suscritos, con información sobre promociones o novedades de la farmacia.

En la Figura 75 se muestra cómo se ha configurado el *BackgroundScheduler*.

```
def start_scheduler():
    scheduler = BackgroundScheduler()

    # Reminder check every 24 hours
    scheduler.add_job(check_and_send_reminders, "interval", hours=24)

    # Weekly newsletter every Monday at 10:00 AM
    scheduler.add_job(
        send_weekly_newsletter,
        trigger="cron",
        day_of_week="mon",
        hour=10,
        minute=0
    )

    scheduler.start()
```

Figura 75: Código de configuración de BackgroundScheduler. Elaboración propia.

Por último, se completó la lógica del *back-end* para la gestión de productos. Se permitió almacenar los productos creados por el administrador, así como editarlos o eliminarlos según sea necesario, asegurando que solo los productos creados desde la interfaz administrativa puedan ser gestionados desde dicha sección.

12. Validación

Una vez implementado el sistema, es fundamental comprobar que se han cumplido todos los requisitos establecidos en la sección de [Requisitos](#). Esta etapa resulta clave para confirmar que el sistema satisface las necesidades identificadas y que resulta útil en el entorno para el cual fue creado.

La validación implica un análisis detallado de cada requisito, tanto funcional como no funcional, con el objetivo de confirmar su correcta aplicación. En caso de detectar que alguno no se ha cumplido, se examinarán las razones, considerando si se debe a una formulación poco clara de los objetivos o a cambios introducidos durante el desarrollo.

12.1 Validación de los requisitos funcionales

Para garantizar el cumplimiento de los requisitos funcionales del sistema, se ejecutaron dos tipos diferentes de tests centrados en el *front-end* y en el *back-end*. Para el *front-end*, se ha hecho un test manual. Estos tests han consistido en verificar la correcta implementación del diseño, la compatibilidad entre componentes visuales y el comportamiento básico de la interfaz de usuario.

Para el *back-end*, se han hecho tests unitarios dado que concentra la lógica de negocio crítica, el procesamiento de datos y la seguridad. Como se puede observar en la Figura 76, se han implementado 102 tests unitarios (“collected 102 items”) en todo el *back-end*.

```
===== test session starts =====
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.5.0
rootdir: /Users/albertcomas/Documents/Uni/TFG/Codigo/ecoFarma/backend
configfile: pytest.ini
testpaths: app/tests
plugins: anyio-4.9.0, asyncio-0.26.0, mock-3.14.0, cov-6.1.1
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 102 items

app/tests/auth/test_jwt_handler.py .....
app/tests/services/test_aws_service.py .
app/tests/services/test_email_service.py ..
app/tests/services/test_google_service.py ...
app/tests/services/test_order_service.py .....
app/tests/services/test_prescription_service.py .....
app/tests/services/test_product_service.py .....
app/tests/services/test_query_service.py .....
app/tests/services/test_user_service.py .....
```

Figura 76: Tests unitarios del *back-end*. Elaboración propia.

En la Figura 77 se muestra que se ha cubierto un 81% del código.

===== tests coverage =====			
_____ coverage: platform darwin, python 3.13.3-final-0 _____			
Name	Stmts	Miss	Cover
app/__init__.py	0	0	100%
app/auth/__init__.py	0	0	100%
app/auth/jwt_handler.py	26	1	96%
app/config/database.py	7	0	100%
app/constants/__init__.py	0	0	100%
app/constants/order_constants.py	3	0	100%
.....			
app/tests/services/test_query_service.py	55	0	100%
app/tests/services/test_user_service.py	136	0	100%
TOTAL	2022	389	81%

Figura 77: Cobertura de los tests del back-end. Elaboración propia.

A continuación se va a mostrar un ejemplo de test unitario. En cada archivo de test se incluyen los *fixtures*¹⁴ mostrados en la Figura 78 para hacer el *mock*¹⁵ del repositorio de productos (*mock_product_repository*) y configurar el *ProductService* (*product_service*). Esto permite probar el servicio de forma aislada, sin depender de la implementación real del repositorio.

```
@pytest.fixture
def mock_product_repository():
    """Fixture para mockear el repositorio de productos."""
    return MagicMock()

@pytest.fixture
def product_service(mock_product_repository):
    """Fixture para crear una instancia del servicio de productos con un repositorio mockeado."""
    service = ProductService()
    service.product_repo = mock_product_repository
    return service
```

Figura 78: Fixture de los tests del back-end. Elaboración propia.

En la Figura 79 se puede observar el test de obtener recomendaciones a partir del historial de búsqueda. Mediante *mocks*, simula un historial de búsquedas de usuario y comprueba que el servicio obtiene el historial del usuario correcto, calcula adecuadamente el embedding promedio de las últimas búsquedas y devuelve las recomendaciones esperadas basadas en este cálculo.

¹⁴ Fixture es una función de pytest (@pytest.fixture) que proporciona datos o configuraciones reutilizables para los tests.

¹⁵ Mock es un objeto simulado (MagicMock) que reemplaza una dependencia real para evitar interacciones externas durante las pruebas.

```

def test_get_recommendations_success(product_service, mock_product_repository):
    """Test que verifica que se obtienen correctamente las recomendaciones."""
    # Arrange
    user = "test_user"
    mock_search_history = get_mock_search_history()
    mock_recommendations = get_mock_recommendations()

    mock_product_repository.get_search_history_by_user.return_value = mock_search_history
    mock_product_repository.search_by_vector.return_value = mock_recommendations

    # Act
    result = product_service.get_recommendations(user)

    # Assert
    assert result == mock_recommendations
    mock_product_repository.get_search_history_by_user.assert_called_once_with(user)
    average_embedding = np.mean(
        [np.array(search["embedding"]) for search in mock_search_history[:5]], axis=0
    ).tolist()
    # Usar pytest.approx para comparar con tolerancia
    mock_product_repository.search_by_vector.assert_called_once_with(
        pytest.approx(average_embedding, rel=1e-9), 10
    )

```

Figura 79: Test de obtener recomendaciones. Elaboración propia.

En la Figura 80 se muestra el mock que se ha usado para el historial de búsquedas.

```

def get_mock_search_history() -> List[SearchData]:
    return [
        SearchData(
            searchTerm="dolor de cabeza",
            date="2023-04-01T10:00:00Z",
            user="user1@example.com",
            embedding=[0.1, 0.2, 0.3],
        ),
        SearchData(
            searchTerm="tos seca",
            date="2023-04-02T10:00:00Z",
            user="user2@example.com",
            embedding=[0.2, 0.3, 0.4],
        ),
        SearchData(
            searchTerm="fiebre alta",
            date="2023-04-03T10:00:00Z",
            user="user3@example.com",
            embedding=[0.3, 0.4, 0.5],
        ),
        SearchData(
            searchTerm="alergia primavera",
            date="2023-04-04T10:00:00Z",
            user="user4@example.com",
            embedding=[0.4, 0.5, 0.6],
        ),
        SearchData(
            searchTerm="vitaminas",
            date="2023-04-05T10:00:00Z",
            user="user5@example.com",
            embedding=[0.5, 0.6, 0.7],
        ),
    ]

```

Figura 80: Mock de historial de búsquedas. Elaboración propia.

12.2 Validación de los requisitos no funcionales

Apariencia y facilidad de uso

Se realizó una encuesta a 15 usuarios finales, entre ellos un grupo de farmacéuticos experimentados. La encuesta consistirá en calificar la claridad, diseño, coherencia visual y la facilidad de uso del sistema web en una escala del 1 al 5. Para validar los requisitos ambos resultados deberán obtener más del 80% de votos con una calificación de 4 o 5.

[Link a encuesta.](#)

En la Figura 81 se muestra el resultado de la pregunta: Evalúa del 1 al 5 la estética de la página.



Figura 81: Resultados de la primera pregunta de la encuesta. Elaborado con Google Forms.

En la Figura 82 se muestra el resultado de la pregunta: Evalúa del 1 al 5 la usabilidad de la página. (1 es necesario disponer de un manual de uso o que me expliquen cómo usarlo y 5 muy fácil de usar e intuitivo).

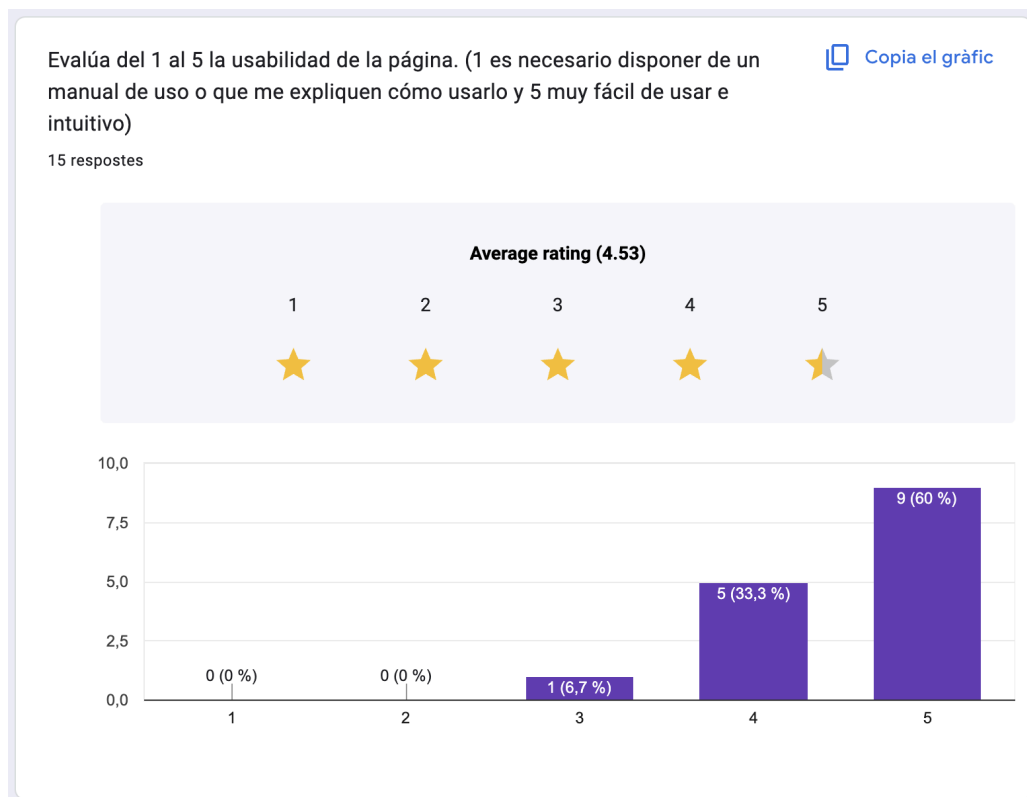


Figura 82: Resultados de la segunda pregunta de la encuesta. Elaborado con Google Forms.

La Figura 83 muestra las respuestas a la pregunta: ¿Qué aspectos añadirías o mejorarías para la estética?.

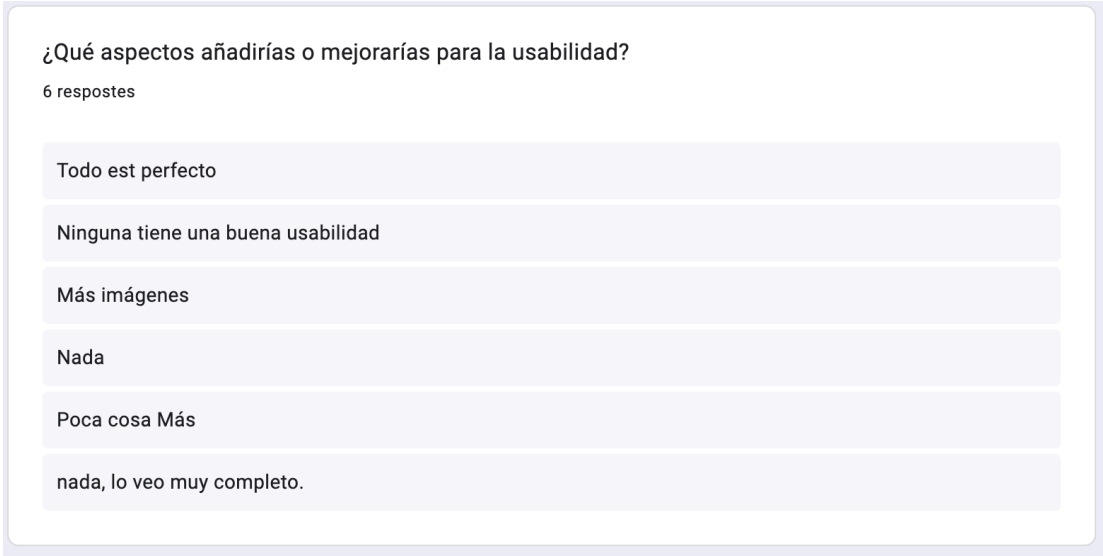
¿Qué aspectos añadirías o mejorarías para la estética?

6 respuestas

- Edta todo bien
- Edta genial!!
- Más color
- Nada, está todo muy bien
- Poca cosa más
- Me gusta mucho, pero quizás un poco más de color de fondo.

Figura 83: Resultados de la tercera pregunta de la encuesta. Elaborado con Google Forms.

Por último, la Figura 84 muestra las respuestas a la pregunta: ¿Qué aspectos añadirías o mejorarías para la usabilidad?.



¿Qué aspectos añadirías o mejorarías para la usabilidad?

6 respuestas

Todo est perfecto
Ninguna tiene una buena usabilidad
Más imágenes
Nada
Poca cosa Más
nada, lo veo muy completo.

Figura 84: Resultados de la cuarta pregunta de la encuesta. Elaborado con Google Forms.

Como se puede observar en los resultados de la encuesta, el requisito se cumple en ambas cuestiones. Esto se ha conseguido teniendo en cuenta la usabilidad de cada componente, y poniéndote en la piel de un usuario no familiarizado con la página web. Sin embargo, aún hay margen de mejora como se refleja en las sugerencias que han puesto los usuarios. Estas sugerencias se tendrán en cuenta para acciones futuras.

Rendimiento al hacer una búsqueda

Se han medido los tiempos que tardan las búsquedas en devolver los resultados, verificando que no superen el tiempo de 500 milisegundos. Para las pruebas se han hecho 5 búsquedas semánticas, tanto desde la página principal cómo en la página de la tienda.

En la Figura 85 se muestran los resultados de los tiempos obtenidos mediante la herramienta de desarrollo de Google Chrome. En las cinco mediciones el tiempo ha estado entre 126 y 180 milisegundos, lo que confirma el requisito de rendimiento. Este rendimiento se ha obtenido gracias a la funcionalidad de búsqueda vectorial de la base de datos.

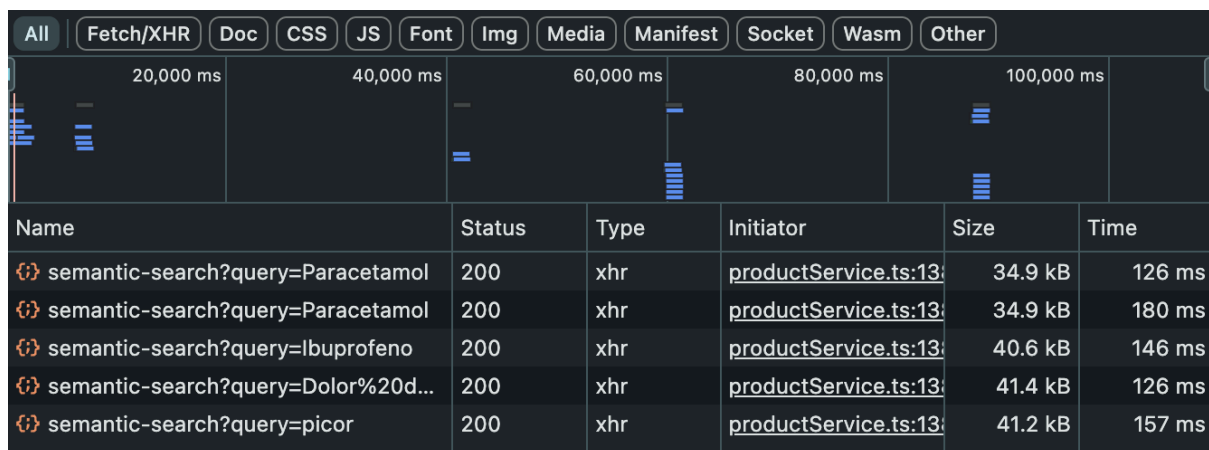


Figura 85: Tiempo de las búsquedas. Elaborado con herramienta de desarrollo de Chrome.

Adaptabilidad

Se hizo un código responsivo para la buena visualización tanto para ordenadores como para teléfonos móviles.

A continuación se muestra cómo se adapta el sistema en ambas pantallas, poniendo algunos ejemplos. En la Figura 86 se muestra la página principal en ambas pantallas.

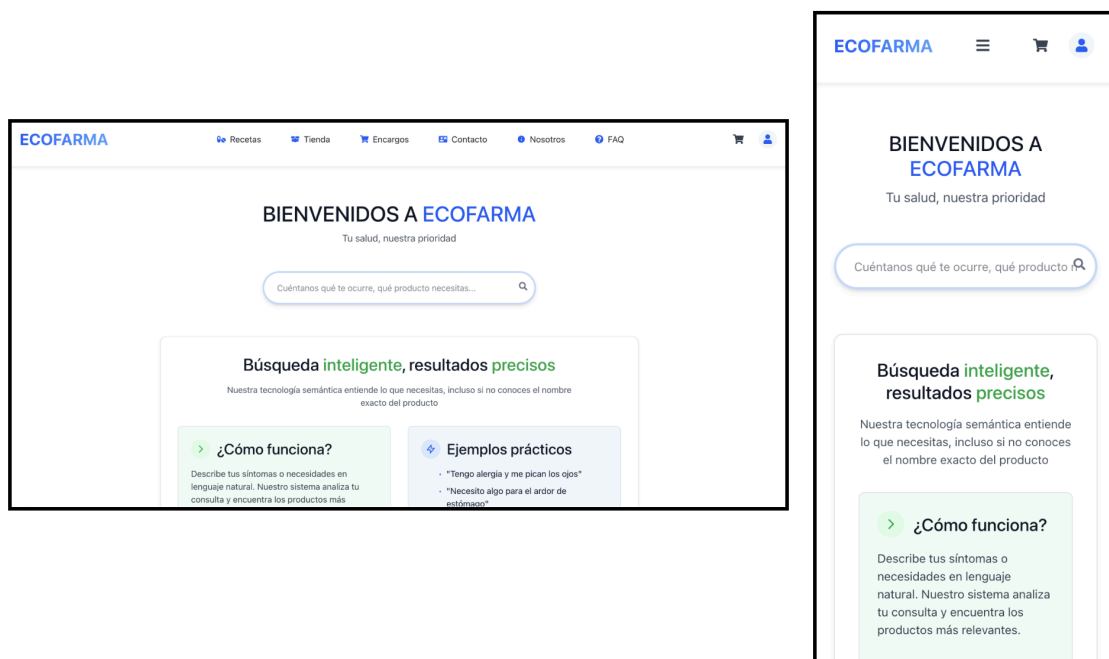


Figura 86: Página de inicio responsiva. Elaboración propia.

La siguiente Figura 87 muestra la página de contacto en ambas pantallas.

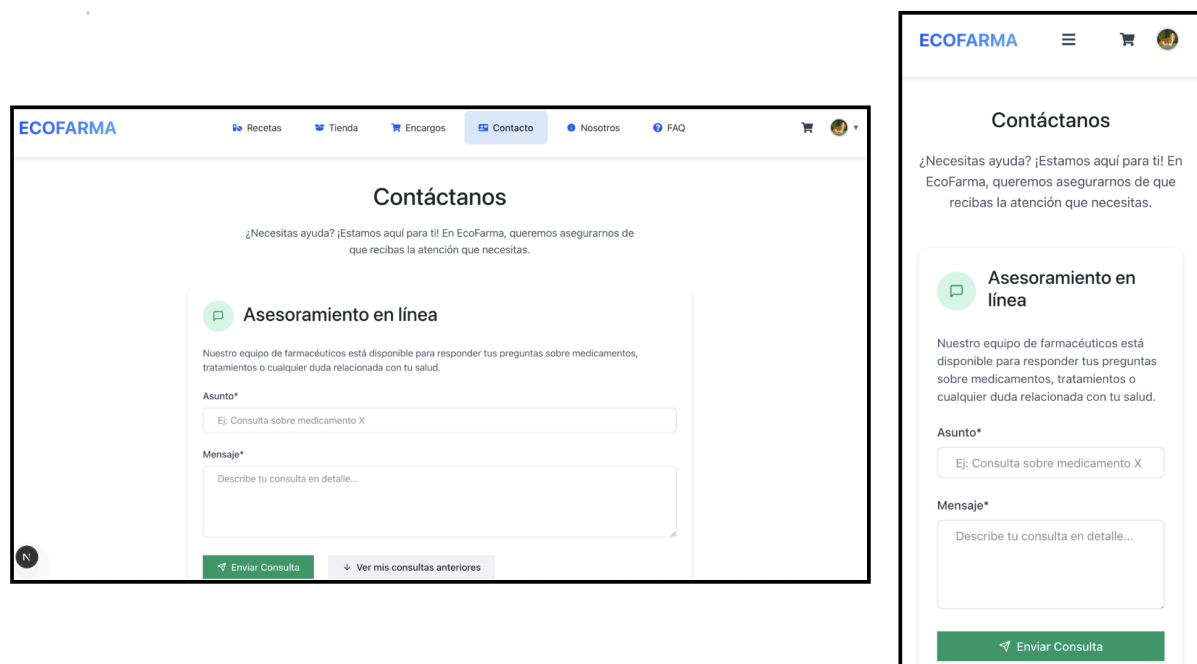


Figura 87: *Página de contacto responsiva. Elaboración propia.*

Cómo se puede observar en los ejemplos anteriores, los componentes se adaptan correctamente a los diferentes dispositivos, por lo que el requisito está validado. Esto se ha conseguido gracias al uso de Tailwind CSS.

Privacidad

Todos los datos sensibles han sido cifrados, se han aplicado políticas de acceso restringido y se ha aplicado el RGPD.

Se han aplicado cifrados en los datos sensibles como en las contraseñas y se han aplicado políticas de acceso restringido según el rol del usuario. Un usuario no logueado no puede acceder a páginas que requieren autenticación, así como un usuario con un rol determinado no puede acceder a páginas de otro rol.

13. Identificación de leyes y regulaciones

Es fundamental conocer el marco legal que regula este proyecto para garantizar su cumplimiento y evitar problemas en la gestión de recetas electrónicas, la protección de datos y la venta online de productos farmacéuticos. A continuación, se detallan las normativas más relevantes:

1. Real Decreto 1718/2010 sobre Receta Médica y Órdenes de Dispensación

Este decreto regula la prescripción y dispensación de medicamentos en España, estableciendo que la receta médica es un documento obligatorio para la adquisición de ciertos fármacos. Además, define los requisitos de validez de la receta electrónica y las obligaciones del farmacéutico en su dispensación [62] [63].

Aplicación al proyecto: Justifica la necesidad de que el usuario presente su Tarjeta Sanitaria Individual (TSI) al recoger los medicamentos reservados online. Aunque el usuario puede subir su receta a la web, la dispensación no puede completarse sin la validación presencial del farmacéutico, garantizando que se cumple la normativa vigente.

2. Normativa de CatSalut sobre la Receta Electrónica

Las regulaciones de CatSalut establecen que la dispensación de medicamentos mediante receta electrónica debe cumplir con protocolos de seguridad estrictos. En particular, solo el titular de la prescripción o una persona autorizada puede retirar los medicamentos en la farmacia [64].

Aplicación al proyecto: Esta normativa refuerza la necesidad de exigir la presentación de la TSI en la farmacia antes de la entrega de productos con receta. Además, garantiza que los datos de la receta no puedan ser utilizados por terceros sin la validación correspondiente.

3. Reglamento General de Protección de Datos (RGPD) - Reglamento (UE) 2016/679

El RGPD establece las normas sobre el tratamiento de datos personales en la Unión Europea, con especial énfasis en la protección de datos sensibles, como los datos de salud. Exige medidas de seguridad estrictas y el consentimiento explícito del usuario para el tratamiento de su información médica [65].

Aplicación al proyecto: La plataforma debe garantizar que los datos de los usuarios, como recetas médicas y datos de salud, se almacenen y procesen de forma segura. Además, el usuario debe otorgar su consentimiento explícito antes de subir su receta al sistema, asegurando su derecho a la privacidad.

4. Ley 34/2002 de Servicios de la Sociedad de la Información y de Comercio Electrónico (LSSI-CE)

Regula el comercio electrónico en España, estableciendo obligaciones para los operadores de plataformas digitales. Obliga a proporcionar información clara sobre los productos y servicios, garantizar la transparencia en la contratación y proteger los derechos de los consumidores [66].

Aplicación al proyecto: La farmacia online debe cumplir con la normativa de comercio electrónico, incluyendo información clara sobre precios, condiciones de venta y políticas de devolución.

5. Real Decreto Legislativo 1/2015 sobre Garantías y Uso Racional de los Medicamentos

Esta normativa regula la distribución y comercialización de medicamentos en España. Prohíbe la venta online de fármacos sujetos a prescripción médica, permitiendo únicamente su reserva a través de plataformas digitales [67].

Aplicación al proyecto: La plataforma debe diferenciar entre productos de venta libre y aquellos que requieren receta médica. Se podrá realizar la reserva online de medicamentos con receta, pero la recogida deberá hacerse presencialmente en la farmacia con validación de la TSI.

14. Resultados de la gestión del proyecto

Una vez finalizadas todas las fases del proyecto, es necesario exponer los resultados obtenidos en la gestión del proyecto. A continuación, se describirán los resultados derivados de la metodología utilizada, los cambios que han habido y las desviaciones respecto a la planificación inicial y al presupuesto previsto.

14.1 Resultados de la metodología

A lo largo del proyecto se ha seguido la metodología ágil definida inicialmente, lo que ha permitido un desarrollo flexible y dinámico. Esta forma de trabajo facilitó alternar entre la programación y la redacción de la memoria, lo cual evitó la saturación y permitió adaptarse rápidamente a imprevistos. El uso de *Scrum* y su organización en *sprints* semanales ofreció margen suficiente para afrontar dificultades sin afectar el progreso general. Aunque en algunos momentos fue necesario dedicar horas extra, todos los *sprints* se completaron con éxito. Se puede afirmar que la metodología elegida resultó clave para el buen desarrollo y gestión del proyecto.

Todas las herramientas de seguimiento explicadas inicialmente han facilitado y agilizado el desarrollo del proyecto. Por un lado, JIRA ha permitido organizar todas las tareas que se habían de hacer de manera clara y estructurada. Cada *sprint* tenía sus historias de usuario, lo cual te permitía enfocarte sólo en esas tareas. Por otro lado, GitHub ha facilitado centralizar todo el código del sistema en un entorno seguro en la nube, accesible desde cualquier lugar. También destaca por su sistema de control de versiones, que permite llevar un seguimiento preciso del desarrollo, gestionar los cambios realizados y, si es necesario, volver a versiones anteriores del proyecto.

Por último, la validación mediante tests ha sido útil durante el desarrollo del proyecto, ya que permitió detectar errores con antelación y solucionarlos progresivamente. Esto evitó que los fallos se acumularan hasta el final, facilitando su corrección a tiempo. Por otro lado, las reuniones periódicas y las correcciones de la tutora, Cristina Gómez, han sido fundamentales para mejorar la redacción de la memoria y orientar adecuadamente cada apartado.

14.2 Resultados de la planificación

La planificación realizada inicialmente en el apartado de Planificación no ha sido seguida exactamente debido a pequeñas desviaciones durante el proyecto.

La Tabla 16 compara las horas previstas con las realmente invertidas en cada fase. A pesar de estas variaciones en la carga horaria, las fechas de inicio y finalización establecidas en la Tabla 2 se respetaron sin retrasos. Esto fue posible gracias a la dedicación de tiempo adicional durante los fines de semana y al ahorro de tiempo en tareas que requirieron menos esfuerzo del esperado. A continuación de la tabla, se explican en detalle las causas de estas diferencias.

Id	Tarea	Horas Estimadas	Horas Reales	Diferencia
GP	GESTIÓN DEL PROYECTO	115	120	+5
GP1	Contextualización y alcance	20	22	+2
GP2	Planificación	20	20	0
GP3	Gestión económica y sostenibilidad	15	15	0
GP4	Integración documento final	5	5	0
GP5	Especificación del proyecto	25	27	+2
GP6	Diseño de la arquitectura	30	31	+1
D	DESARROLLO	337	341	+4
I	INCEPTION	25	25	0
S1	SPRINT 1	67	67	-2
S2	SPRINT 2	95	105	+10
S3	SPRINT 3	90	85	-5
S4	SPRINT 4	60	61	+1
	FINALIZACIÓN	88	91	+3
D17	Refinamiento, <i>testing</i> y <i>refactoring</i>	30	30	0
D18	Despliegue del sistema	3	6	+3
DC5	Acabar la memoria	30	30	0
GP7	Preparación de la defensa	25	25	0
TOTAL		540	552	+12

Tabla 16: Diferencia entre horas estimadas y reales. Elaboración propia.

Durante el desarrollo del proyecto se han realizado diversos ajustes, con el objetivo de reflejar con mayor precisión el estado actual del sistema y adaptar los requisitos a las necesidades reales del desarrollo.

Cambios en los requisitos del sistema

En la sección de contextualización y alcance se han ajustado los requisitos funcionales del sistema. En concreto, se han eliminado los siguientes filtros en las búsquedas de productos:

- Filtro por comercialización (Comercializados / No comercializados).
- Filtro por autorización (Autorizados / No autorizados).
- Filtro por estupefaciente.
- Filtro por psicotropo.

Se ha tomado esta decisión ya que se ha considerado que estos filtros no aportan valor a la búsqueda de productos.

Además, se ha decidido restringir el formato de las recetas que pueden ser cargadas por los usuarios, aceptando únicamente archivos en formato PDF. Esta decisión se ha tomado teniendo en cuenta varios factores como el tiempo, simplicidad y calidad del resultado.

Finalmente, también se ha eliminado la posibilidad de que los administradores puedan añadir nuevos usuarios, descartando esa funcionalidad del panel de administración.

En la sección de especificación se han realizado varias modificaciones para actualizar y simplificar la documentación funcional del sistema. En particular:

- Se han eliminado los casos de uso, así como sus descripciones y modelos de comportamiento de los requisitos quitados mencionados anteriormente.
- En el modelo conceptual, se ha eliminado el tipo de notificación “consulta respondida”, al no formar parte de ese tipo de notificación.
- Asimismo, se ha renombrado el tipo de notificación “renovación medicamento” como “Recordatorio”, con el objetivo de adoptar una denominación más genérica y adecuada a su función actual.

Por último, en la sección de diseño se ha actualizado el diagrama de clases (*back-end*):

- Eliminación de controladores no utilizados, como PharmController y CartController.
- Reorganización de las clases para mejorar la cohesión y claridad del diseño.
- Las funciones relacionadas con recordatorios y promociones han sido trasladadas al controlador de productos, lo cual se ha reflejado en el diagrama de clases del back-end.

Cambios en el desarrollo

Inicialmente, se generaron los embeddings con el modelo *nomic-embed-text* de Ollama dado que ofrecía buenos resultados y podía ejecutarse localmente. No obstante, su uso en producción iba a ser problemático debido a la complejidad de su integración y los requerimientos de infraestructura. Por este motivo, se optó por una alternativa más ligera y fácil de desplegar: el modelo *all-MiniLM-L6-v2* de Sentence Transformers. Se tuvo que volver a generar todos los embeddings usando este nuevo modelo.

No se han implementado los requisitos quitados mencionados anteriormente. En algunos casos, como los filtros, se ha reducido el tiempo requerido para hacer la tarea. En otros casos, como quitar la posibilidad de subir imágenes de recetas, han aumentado el tiempo ya que se intentó hacer la implementación.

Cambios en la finalización

En un principio, se valoró desplegar el proyecto en *Heroku* por su simplicidad y experiencias pasadas, pero se descartó debido a que su plan gratuito fue eliminado y las restricciones en recursos dificultaban el despliegue completo.

Para el *front-end* se optó por Vercel, ya que ofrece una integración directa con proyectos basados en *Next.js*, despliegue automático desde GitHub y gran rendimiento en producción. En cuanto al *back-end*, inicialmente se intentó desplegar en *Fly.io*, pero surgieron problemas debido al tamaño del proyecto, que superaba los límites permitidos. Finalmente, se decidió utilizar Railway, que ofrecía una solución más flexible y sencilla de configurar. Para poder desplegar correctamente, fue necesario contratar el plan de desarrollador por 5 €.

14.3 Resultados económicos

Las desviaciones han supuesto un aumento en los costes. Este sobrecoste se encuentra dentro de los márgenes de contingencia, calculada en el apartado 8.2.1.

Costes humanos

Id	Tarea	Coste Estimado (€)	Coste Real (€)	Diferencia (€)
GP	GESTIÓN DEL PROYECTO	3.330,30	3.474	+144
GP1	Contextualización y alcance	587,4	644,88	+57,48
GP2	Planificación	587,4	587,4	0
GP3	Gestión económica y sostenibilidad	431,1	431,1	0
GP4	Integración documento final	143,7	143,7	0
GP5	Especificación del proyecto	718,5	775,98	+57,48
GP6	Diseño de la arquitectura	862,2	890,94	+28,74
D	DESARROLLO	7.010,74	7.093,22	+82,48
I	INCEPTION	529,19	529,19	0
S1	SPRINT 1	1.390,32	1.349,08	-41,24
S2	SPRINT 2	1.967,81	2.174,01	+206,20
S3	SPRINT 3	1.863,02	1.759,92	-103,10
S4	SPRINT 4	1.260,40	1.281,02	+20,62
	FINALIZACIÓN	2.253,12	2.314,98	+61,86
D17	Refinamiento, testing y refactoring	610,56	610,56	0
D18	Despliegue del sistema	61,86	123,72	+61,86
DC5	Acabar la memoria	862,2	862,2	0
GP7	Preparación de la defensa	718,5	718,5	0
	TOTAL	12.594,16	12882,2	+288,04

Tabla 17: Resumen de las desviaciones presupuestarias de costes humanos. Elaboración propia.

15. Informe de sostenibilidad

La sostenibilidad es un aspecto fundamental en la realización de cualquier proyecto, especialmente en el contexto actual. Este apartado explica la integración de los principios de sostenibilidad en el desarrollo del proyecto, utilizando la matriz de sostenibilidad como método de evaluación. Finalmente, se incluye una reflexión personal sobre los aprendizajes adquiridos en relación con la sostenibilidad.

15.1 Sostenibilidad económica

El proyecto tiene como finalidad optimizar el funcionamiento de las farmacias mediante procesos automatizados. La digitalización y centralización de la gestión no solo reduce el tiempo empleado en las tareas, sino que también minimiza errores humanos. Esto se traduce en una mayor eficiencia del personal farmacéutico y, por tanto, en un incremento de las ventas. Así, se obtienen beneficios económicos tanto por la reducción de costes operativos como por el aumento de los ingresos.

Durante el desarrollo del proyecto, se ha llevado a cabo una planificación económica detallada, contemplando todos los recursos necesarios y anticipando posibles imprevistos que pudieran impactar el presupuesto. Este enfoque preventivo ha permitido evitar derroches y asegurar que el proyecto se completase sin desviaciones económicas, respetando en todo momento los límites presupuestarios establecidos.

Cabe destacar que, ante un crecimiento considerable de usuarios, sería necesario realizar inversiones adicionales. Estas podrían incluir la ampliación de la infraestructura técnica (como servidores y bases de datos) o la contratación de personal especializado en soporte técnico. Para hacer frente a estos posibles gastos, se podrían implementar estrategias de monetización, como la inclusión de publicidad en la plataforma.

15.2 Sostenibilidad social

Este proyecto aborda una necesidad social concreta en un entorno cada vez más digitalizado. La población está habituada a gestionar compras y servicios por internet, por lo que disponer de una plataforma eficiente para la administración de recetas, productos, pedidos y asesoramiento farmacéutico supone una gran mejora. Esta solución no sólo aporta comodidad y accesibilidad a los usuarios, sino que también aligera la carga de trabajo de los profesionales del sector, permitiéndoles enfocarse en tareas de mayor valor añadido y elevando la calidad del servicio que ofrecen.

A nivel personal, el desarrollo del proyecto me ha permitido adquirir conocimientos valiosos en distintas áreas. He fortalecido mis habilidades en gestión de proyectos, programación avanzada y entrenamiento de modelos de inteligencia artificial. Asimismo,

la implementación práctica de la metodología ágil Scrum me ha ayudado a comprender mejor el trabajo estructurado en equipo. Todas estas competencias representan un importante impulso para mi crecimiento profesional y mi futura incorporación al ámbito laboral.

15.3 Sostenibilidad ambiental

Al tratarse de una solución basada en software, el proyecto tiene un impacto ambiental relativamente bajo. Desde el inicio, se ha apostado por un enfoque responsable, aprovechando recursos ya disponibles como equipos informáticos, lo que ha evitado inversiones innecesarias en nuevo hardware.

Además, se ha hecho el trabajo en casa durante todo el desarrollo, lo cual ha eliminado la necesidad de desplazamientos y ha contribuido a reducir la huella ambiental. Se han aplicado buenas prácticas de diseño y desarrollo para que el sistema sea lo más eficiente y escalable posible, reduciendo así el consumo energético siempre que ha sido viable.

15.4 Reflexión personal

Hacer la autoevaluación sobre sostenibilidad me ha hecho ver cosas que antes pasaba por alto. Aunque ya intentaba aplicar acciones para reducir el impacto ambiental en mis proyectos, me he dado cuenta de que no basta con tener buenas intenciones: hace falta profundizar más, entender cómo medir ese impacto y aplicar estrategias concretas para reducirlo de verdad.

Al revisar los distintos aspectos que afectan a un proyecto, he empezado a ser más consciente de las consecuencias medioambientales y sociales que pueden tener nuestras decisiones como desarrolladores. Este ejercicio me ha motivado a seguir aprendiendo sobre sostenibilidad, porque estoy convencido de que, cuanto más sepa, mejores soluciones podré crear en el futuro: más responsables, más eficientes y con un impacto positivo real.

16. Conclusiones

Llegados a este punto, antes de dar por concluido el proyecto, es necesario realizar una última retrospectiva para evaluar el cumplimiento de los objetivos planteados, verificar la adquisición de las competencias técnicas definidas en la matrícula, y analizar posibles líneas de futuro para la aplicación.

16.1 Conclusiones del proyecto

En cuanto a los objetivos del proyecto, estos se han cumplido de forma satisfactoria, ya que se ha diseñado e implementado un sistema web alineado con los propósitos definidos al inicio. La solución integra un buscador semántico que facilita la localización eficiente de productos, unas recomendaciones personalizadas basadas en el historial de búsqueda, y funcionalidades para la gestión de recetas y encargos. También incorpora un sistema de asesoramiento farmacéutico para mejorar la atención al usuario. Todo ello se ha desarrollado dentro de una plataforma intuitiva, accesible y centrada en ofrecer una experiencia de navegación fluida y eficaz.

A lo largo de este proyecto he tenido la oportunidad de poner en práctica muchos de los conocimientos que he ido adquiriendo durante la carrera. Desde el uso de metodologías ágiles para organizar el trabajo, hasta la aplicación de patrones de diseño y la definición de requisitos tal y como aprendimos en clase. Todo ha contribuido a que el desarrollo avanzara de forma ordenada y constante. Cuando surgieron problemas (que, como en cualquier proyecto, no faltaron), pude resolverlos a tiempo, lo que me permitió completar cada sprint dentro del plazo y evitar agobios de última hora.

Eso no significa que todo haya sido perfecto. Hay detalles que sé que podría mejorar, como el diseño visual de la web o ciertas optimizaciones en el código que harían el sistema más eficiente. Aun así, estoy contento con el resultado. Siento que el proyecto cumple con los objetivos planteados y me ha servido no solo para demostrar lo que he aprendido, sino también para identificar en qué áreas puedo seguir creciendo como futuro ingeniero.

16.2 Justificación de las competencias

CES1.1: Desarrollar, mantener y evaluar sistemas y servicios software complejos y/o críticos. [En profundidad]

A lo largo del proyecto se ha construido un sistema en el que se conectan de forma eficiente la interfaz de usuario, la lógica de servidor, la base de datos y distintos servicios externos. El *front-end* del sistema ha sido diseñado para ofrecer una experiencia fluida y adaptativa, mientras que el *back-end* se encarga de gestionar las interacciones con la base de datos y coordinar la comunicación externa. Durante todo el proceso de desarrollo se han seguido principios sólidos de diseño y se han aplicado buenas prácticas de programación, lo que ha permitido asegurar la escalabilidad y el correcto funcionamiento del sistema. En consecuencia, puede afirmarse que esta competencia ha sido plenamente adquirida y aplicada.

CES1.2: Dar solución a problemas de integración en función de las estrategias, estándares y tecnologías disponibles. [En profundidad]

El sistema no se compone de una única tecnología, sino que es el resultado de la integración de varios componentes que, al estar interconectados, funcionan como un todo. Por este motivo, el proyecto ha implicado una integración eficaz entre el Next.js del *front-end*, Python del *back-end* y MongoDB para la base de datos. También, ha sido necesario integrar servicios externos y gestionar adecuadamente los datos. Para ello, se han utilizado servicios como Amazon Web Services para almacenar las fotos de perfil, la API de Resend para enviar correos electrónicos, y la API de CIMA para obtener información sobre productos. Dada la dedicación invertida en esta tarea, se considera que esta competencia se ha alcanzado con creces.

CES1.7: Controlar la calidad y diseñar pruebas en la producción de software. [Un poco]

A lo largo del desarrollo, se han incorporado diversas pruebas unitarias destinadas a validar que cada módulo funcione de manera correcta e independiente. Estas pruebas han sido clave para mejorar la solidez del código y detectar posibles errores. Considerando la cantidad de pruebas y el alto nivel de cobertura, puede afirmarse que esta competencia ha sido superada satisfactoriamente.

CES2.1: Definir y gestionar los requisitos de un sistema software. [Bastante]

Se ha realizado un análisis riguroso de las necesidades del sistema, ajustando las especificaciones conforme surgían nuevos requerimientos durante el desarrollo. Los requisitos no funcionales incorporan criterios claros de evaluación, mientras que los funcionales se describen mediante su escenario principal de éxito y posibles variantes. La atención y esfuerzo dedicados a esta fase permiten afirmar que la competencia ha sido superada con éxito.

16.3 Integración del conocimiento

A lo largo del desarrollo del proyecto, se han aplicado diferentes conocimientos adquiridos durante el grado. A continuación, se detallarán las principales asignaturas del grado cuyos contenidos se han utilizado en la realización de este trabajo.

Introducción al Diseño de Interfaces (IDI)

Esta asignatura ha sido clave para aplicar principios de diseño centrado en el usuario y mejorar la experiencia de usuario del sistema. Gracias a ella, se ha podido crear una interfaz clara, intuitiva y accesible.

Bases de Datos (BD) y Diseño de Bases de Datos (DBD)

Ambas asignaturas han proporcionado los conocimientos necesarios para diseñar e implementar una base de datos estructurada y eficiente, adecuada para las necesidades del proyecto.

Arquitectura del Software (AS)

Gracias a esta asignatura, se han podido aplicar patrones de diseño y estructurar el proyecto siguiendo una arquitectura clara y mantenible. Me ha ayudado a separar responsabilidades y facilitar la escalabilidad del sistema.

Aplicaciones y Servicios Web (ASW)

Los contenidos de esta asignatura han sido esenciales para implementar y consumir APIs REST dentro del proyecto. Además, ha ayudado a estructurar correctamente endpoints, gestionar peticiones HTTP y tratar respuestas del servidor.

Gestión de Proyectos de Software (GPS)

Se han aplicado técnicas aprendidas como la definición de historias de usuario, planificación iterativa y seguimiento de tareas. También ha ayudado a comprender mejor el ciclo de vida de un proyecto y sus fases.

Ingeniería de Requisitos (ER)

Esta asignatura ha sido clave para identificar, redactar y clasificar requisitos funcionales y no funcionales. Gracias a ello, se pudo definir correctamente el alcance del sistema y las expectativas del usuario.

Proyecto de Ingeniería de Software (PES)

Esta asignatura fue fundamental como preparación directa para el proyecto, al simular un proyecto real de desarrollo. Ayudó a integrar conocimientos de diferentes áreas y a trabajar con metodologías ágiles.

16.4 Valoración personal

Como retrospectiva personal, este proyecto lo he disfrutado de principio a fin. El hecho de poder aplicar todo lo que he aprendido durante la carrera en un caso real es muy enriquecedor. Sin lugar a dudas, este proyecto me ha permitido demostrar que todos estos años de estudio han valido la pena, y aprender muchas cosas nuevas.

El hecho de que la idea del proyecto surgiera a partir del trabajo de mi madre ha hecho que me implicara de forma cuidadosa en cada detalle. Mantener una comunicación constante con ella a lo largo del desarrollo me ha permitido comprender en profundidad las necesidades reales tanto de una farmacia como de sus clientes. Esta conexión directa con el entorno farmacéutico me ha ayudado a tomar decisiones más acertadas durante el diseño del sistema. Además, me ha permitido descubrir un sector que desconocía y que ahora me resulta interesante.

Un aspecto del que estoy orgulloso es la manera en la que he afrontado las dificultades encontradas. La realización de este proyecto no era tarea sencilla ya que había muchas tecnologías nuevas de las que no tenía experiencia. Sin embargo, en lugar de frustrarme por las dificultades, buscaba la documentación necesaria para solucionar el problema y desarrollaba la mejor opción posible. Sin mi constancia y organización del proyecto, hubiese sido mucho más difícil la elaboración del proyecto.

16.5 Acciones futuras

El proyecto se considera finalizado tras verificar el cumplimiento de todos los objetivos y requisitos establecidos. No obstante, se identifican posibles mejoras y funcionalidades adicionales que podrían implementarse en caso de disponer de más tiempo.

Pedidos a domicilio

Implementar un sistema de entrega a domicilio para medicamentos sin receta, ampliando así la accesibilidad del servicio. Esto facilitaría el acceso a productos farmacéuticos para personas con movilidad reducida o poco tiempo.

Integrar pasarela de pagos

Permitir a los usuarios realizar pagos directamente desde la plataforma. Esto agiliza el proceso de encargo y mejora la experiencia de usuario.

Mejorar el sistema de analítica

A parte de las valoraciones de búsquedas y recomendaciones, añadir estadísticas como productos más encargados, tiempos de gestión y comportamiento del usuario. Esta información ayudará a tomar decisiones estratégicas.

Asistente virtual (chatbot)

Incorporar un chatbot con inteligencia artificial que ayude a resolver dudas frecuentes, recomendar productos o guiar al usuario durante un encargo. Esto mejoraría la experiencia del usuario al ofrecer atención inmediata.

Propuestas de usuarios y farmacéuticos:

Mayor precisión en la configuración horaria

Permitir que los usuarios seleccionen franjas horarias exactas para realizar encargos o recibir recordatorios. Esto da más control y flexibilidad a los clientes. También facilita una mejor organización del personal farmacéutico.

Más imágenes y mejor diseño visual

Incluir más imágenes en la página web, así como más color en el fondo. Esto hace que la navegación sea más agradable y genera mayor satisfacción al cliente final.

Gestión multiusuario para familias o cuidadores

Permitir que una misma cuenta gestione varios perfiles (por ejemplo, para familiares o pacientes a cargo de un cuidador). Así, un solo usuario podría hacer encargos para diferentes personas.

Bibliografía

- [1] Redacción Médica. El comercio online farmacéutico en España crece un 40% en cinco años. <https://www.redaccionmedica.com/secciones/industria/el-comercio-online-farmaceutico-en-espana-crece-un-40-en-cinco-anos-8715> (visitado 10-02-2025)
- [2] Statista. Encuestados que han comprado medicamentos en una farmacia en los últimos 12 meses. <https://es.statista.com/grafico/30197/encuestados-que-han-comprado-medicamentos--en-un-a-farmacia-en-los-ultimos-12-meses/> (visitado 10-02-2025)
- [3] Elastic. What is vector embedding? <https://www.elastic.co/what-is/vector-embedding> (visitado 22-01-2025)
- [4] Elastic. What is semantic search? <https://www.elastic.co/es/what-is/semantic-search> (visitado 22-01-2025)
- [5] Amazon Web Services. The difference between frontend and backend. <https://aws.amazon.com/es/compare/the-difference-between-frontend-and-backend/> (visitado 22-01-2025)
- [6] La Razón. Digitalización: oportunidad en el sector farmacéutico. https://www.larazon.es/sociedad/digitalizacion-oportunidad-sector-farmaceutico_202501296799e013e95c0600017e730b.html (visitado 11-02-2025)
- [7] ISMP España. Errores de medicación y seguridad del paciente. <https://www.ismp-espana.org/documentos/view/63> (visitado 12-02-2025)
- [8] OpenWebinars. Embeddings: Qué son y cómo transforman datos en información. <https://openwebinars.net/blog/embeddings/> (visitado 13-02-2025)
- [9] Disruptivo Oficial. Online Pharmacy Spain. <https://disruptivooficial.com/blog/online-pharmacy-spain> (visitado 10-02-2025)
- [10] Atida. Farmacia online. <https://www.atida.com/es-es> (visitado 14-02-2025)
- [11] PromoFarma. Compra online de medicamentos. <https://www.promofarma.com/> (visitado 14-02-2025)
- [12] Dosfarma. Farmacia online en España. <https://www.dosfarma.com/> (visitado 14-02-2025)
- [13] Gluo. Requisitos no funcionales: ¿Por qué son importantes? <https://www.gluo.mx/blog/requisitos-no-funcionales-por-que-son-importantes> (visitado 21-01-2025)

- [14] Robertson, S., & Robertson, J. Volere Requirements Specification Template. <https://www.volere.org/templates/volere-requirements-specification-template/> (visitado 20-01-2025)
- [15] Unión Europea. Reglamento general de protección de datos (RGPD). EUR-Lex. <https://eur-lex.europa.eu/ES/legal-content/summary/general-data-protection-regulation-gdpr.html> (visitado 20-01-2025)
- [16] Atlassian. Scrum y metodologías ágiles. <https://www.atlassian.com/agile/scrum> (visitado 17-02-2025)
- [17] InterQuality. Scrum methodology. <https://interqualitybg.com/en/resources/scrum-and-agile-resources/scrum-methodology> (visitado 17-02-2025)
- [18] Atlassian. Software Jira. <https://www.atlassian.com/es/software/jira> (visitado 17-02-2025)
- [19] GitHub. Plataforma de desarrollo basada en Git. <https://github.com/> (visitado 18-02-2025)
- [20] Humberto, Sergio. Conociendo GitFlow. <https://medium.com/@sergiohumberto27/conociendo-gitflow-a588716fbc28> (visitado 19-02-2025)
- [21] Facultat d'Informàtica de Barcelona. Normativa TFG GEI - Document final. https://www.fib.upc.edu/sites/fib/files/documents/actes/normativatfg-gei_document_final.pdf (visitado 19-02-2025).
- [22] Agencia Española de Medicamentos y Productos Sanitarios (AEMPS). CIMA REST API. https://www.aemps.gob.es/apps/cima/docs/CIMA_REST_API.pdf (visitado 20-02-2025).
- [23] Vercel. Your complete platform for the web. <https://vercel.com/> (visitado 21-04-2025).
- [24] Railway. Shipping great products is hard. Scaling infrastructure is easy. <https://railway.com/> (visitado 26-04-2025).
- [25] Apple. MacBook Air 13" Chip M3. <https://www.apple.com/es/shop/buy-mac/macbook-air/13-pulgadas-medianoche-chip-m3-de-apple-con-cpu-de-8-n%C3%BAcleos-y-gpu-de-8-n%C3%BAcleos-16-gb-memoria-256gb> (visitado 22-02-2025).
- [26] Trust. Fyda Ratón inalámbrico. <https://www.trust.com/es/product/23804-fyda-rechargeable-wireless-comfort-mouse> (visitado 22-02-2025).
- [27] Microsoft. Visual Studio Code. <https://code.visualstudio.com/> (visitado 21-02-2025).

- [28] Postman. Herramienta Postman. <https://www.postman.com/> (visitado 22-02-2025).
- [29] MongoDB. Base de datos MongoDB. <https://www.mongodb.com/> (visitado 22-02-2025).
- [30] Amazon Web Services (AWS). Servicios en la nube de AWS. <https://aws.amazon.com/es/> (visitado 22-02-2025).
- [31] Lucid. Lucidchart - Diagramación y colaboración. <https://lucid.app/> (visitado 22-02-2025).
- [32] Wix. Website builder. <https://es.wix.com/> (visitado 23-02-2025)
- [33] Resend. Email for developers. <https://resend.com> (visitado 24-03-2025)
- [34] MediPIM. Documentación de la API de MediPIM.
<https://platform.medipim.be/docs/api/v4/using-the-api.html> (visitado 24-02-2025).
- [35] Glassdoor. Comparación de sueldos. <https://www.glassdoor.es/Sueldos/index.htm> (visitado 26-02-2025).
- [36] Agencia Tributaria. Tabla de amortización simplificada.
https://sede.agenciatributaria.gob.es/Sede/ayuda/manuales-videos-folletos/manuales-practicos/folleto-actividades-economicas/3-impuesto-sobre-renta-personas-fisicas/3_5-estimacion-directa-simplificada/3_5_4-tabla-amortizacion-simplificada.html (visitado 27-02-2025).
- [37] Días Laborables. Calendario de días laborables y feriados 2025.
https://www.dias-laborables.es/dias_laborables_feriados_2025.htm (visitado 27-02-2025).
- [38] Coworking Spain. CREC Coworking Eixample Gran.
<https://coworkingspain.es/espacios/coworking/barcelona/crec-coworking-eixample-gran> (visitado 29-02-2025).
- [39] Next.js. Framework de React para aplicaciones web. <https://nextjs.org/> (visitado 03-03-2025).
- [40] UNIR. Frameworks: definición y características.
<https://unirfp.unir.net/revista/ingenieria-y-tecnologia/framework/> (visitado 03-03-2025).
- [41] AWS. ¿Qué es una API RESTful? <https://aws.amazon.com/es/what-is/restful-api/> (visitado 03-03-2025).
- [42] Next.js. Estructura de proyecto en Next.js.
<https://nextjs.org/docs/app/getting-started/project-structure> (visitado 04-03-2025).
- [43] Medium. React: Container Component Pattern.
<https://statefull.medium.com/react-container-component-eee01e721694> (visitado 05-03-2025).

- [44] Refactoring Guru. Patrón de diseño Observer.
<https://refactoring.guru/es/design-patterns/observer> (visitado 05-03-2025).
- [45] Dev.to. Patrón de diseño de Módulo.
<https://dev.to/thelordofth3cloud/patron-de-diseno-de-modulo-eso-como-se-come-5g13>
(visitado 05-03-2025).
- [46] Refactoring Guru. Patrón de diseño Facade.
<https://refactoring.guru/es/design-patterns/facade> (visitado 05-03-2025).
- [47] Refactoring Guru. Patrón de diseño Singleton.
<https://refactoring.guru/es/design-patterns/singleton> (visitado 05-03-2025).
- [48] Medium. Controller-Service-Repository Pattern.
<https://tom-collings.medium.com/controller-service-repository-16e29a4684e5> (visitado 06-03-2025).
- [49] Refactoring Guru. Patrón de diseño State.
<https://refactoring.guru/es/design-patterns/state> (visitado 06-03-2025).
- [50] Platzi. Patrón Repository. <https://platzi.com/blog/patron-repository/> (visitado 06-03-2025).
- [51] KeepCoding. Patrón de Inyección de Dependencias.
<https://keepcoding.io/blog/patron-de-inyeccion-de-dependencias/> (visitado 06-03-2025).
- [52] OpenWebinars. ¿Qué es JSON Web Token y cómo funciona?
<https://openwebinars.net/blog/que-es-json-web-token-y-como-funciona/> (visitado 14-03-2025).
- [53] Zustand. Getting Started – Introduction.
<https://zustand.docs.pmnd.rs/getting-started/introduction> (visitado 23-04-2025).
- [54] Ollama. nomic-embed-text. <https://ollama.com/library/nomic-embed-text> (visitado 20-04-2025).
- [55] Hugging Face. all-MiniLM-L6-v2 – Sentence Transformers.
<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2> (visitado 20-04-2025).
- [56] Medium. Better Article Recommendations with Triplet Fine-Tuning.
<https://medium.com/s%C3%BCddeutsche-zeitung-digitale-medien/better-article-recommendations-with-triplet-fine-tuning-6b52a587b85f> (visitado 24-04-2025).
- [57] Springer. Better Article Recommendations with Triplet Fine-Tuning.
<https://link.springer.com/article/10.1007/s10994-024-06529-8> (visitado 24-04-2025).

- [58] SaludSegur. Ejemplo receta electrónica Adeslas.
<https://saludsegur.es/adeslas-receta-electronica-privada/ejemplo-receta-electronica-adeslas/>
(visitado 25-04-2025).
- [59] ICOEV. Receta electrónica para dentistas.
<https://www.icoev.es/servicios-para-colegiados/recetadentistas/> (visitado 25-04-2025).
- [60] PyPI. pdfplumber 0.11.6. <https://pypi.org/project/pdfplumber/0.11.6/> (visitado 25-04-2025).
- [61] APScheduler. Background Scheduler.
<https://apscheduler.readthedocs.io/en/3.x/modules/schedulers/background.html> (visitado 12-05-2025).
- [62] BOE. Real Decreto sobre receta médica y órdenes de dispensación.
<https://www.boe.es/buscar/act.php?id=BOE-A-2011-1013> (visitado 15-02-2025).
- [63] Ministerio de Sanidad. Preguntas Frecuentes sobre la Receta Electrónica.
<https://www.sanidad.gob.es/areas/saludDigital/recetaElectronicaSNS/faq/faq.htm> (visitado 13-02-2025).
- [64] CatSalut. Tríptico informativo sobre la receta electrónica.
https://catsalut.gencat.cat/web/.content/minisite/catsalut/ciutadania/serveis_atencio_salut/_quins_serveis_ofereix_catsalut/atencio_farmaceutica/documents/triptico_receta.pdf
(visitado 13-02-2025).
- [65] EUR-Lex. Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo.
<https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32016R0679> (visitado 14-02-2025).
- [66] BOE. Ley Orgánica de Protección de Datos de Carácter Personal.
<https://www.boe.es/buscar/act.php?id=BOE-A-2002-13758> (visitado 14-02-2025).
- [67] BOE. Ley de servicios de la sociedad de la información y de comercio electrónico.
<https://www.boe.es/buscar/act.php?id=BOE-A-2015-8343> (visitado 14-02-2025).

Anexo 1: Descripción de casos de uso

Usuario, farmacéutico y administrador: Gestión de cuentas

Caso de uso	#1 Registrar	Actor principal	Usuario
Precondiciones	-		
Disparador	El usuario quiere crear una cuenta en el sistema.		
Escenario principal de uso			
1. El usuario indica que quiere crear una cuenta y rellena el formulario de registro (nombre, apellido, correo y contraseña). 2. El sistema valida los datos. 3. El sistema guarda la información en la base de datos. 4. El sistema indica al usuario que la cuenta se ha creado correctamente. 5. El sistema redirige al usuario a la pantalla principal.			
Extensiones			
4a. El sistema determina que el correo ya está en uso. 4a1. El sistema avisa al usuario que el correo ya está en uso y permite al usuario volver a intentarlo.			

Caso de uso	#2 Iniciar sesión	Actor principal	Usuario, Farmacéutico y Administrador
Precondiciones	-		
Disparador	El actor quiere acceder a su cuenta del sistema.		
Escenario principal de uso			
1. El actor introduce el correo y la contraseña e indica al sistema que valide las credenciales. 2. El sistema valida las credenciales. 3. El sistema redirige al actor a la pantalla principal.			
Extensiones			
3a. El sistema determina que las credenciales son incorrectas. 3a1. El sistema avisa al actor que las credenciales son incorrectas y permite al actor volver a intentarlo.			

Caso de uso	#3 Cerrar sesión	Actor principal	Usuario, Farmacéutico y Administrador
Precondiciones	El actor está dado de alta en el sistema.		
Disparador	El actor quiere cerrar la sesión de su cuenta del sistema.		
Escenario principal de uso			
1. El actor indica al sistema que quiere cerrar sesión. 2. El sistema redirige al actor a la pantalla principal.			

Caso de uso	#5 Eliminar cuenta	Actor principal	Usuario
Precondiciones	El usuario está dado de alta en el sistema.		
Disparador	El usuario quiere eliminar su cuenta del sistema.		
Escenario principal de uso			
1. El usuario indica que quiere eliminar su cuenta e introduce la contraseña. 2. El sistema valida la contraseña. 3. El sistema elimina los datos correspondientes a la cuenta en la base de datos. 4. El sistema indica al usuario que la cuenta ha sido eliminada correctamente y redirige al usuario a la pantalla principal.			
Extensiones			
3a. El sistema determina que la contraseña introducida no es correcta. 3a1. El sistema avisa al usuario que la contraseña introducida no es correcta y permite al usuario volver a intentarlo.			

Caso de uso	#7 Eliminar usuario	Actor principal	Administrador
Precondiciones	El administrador ha iniciado sesión. El usuario está dado de alta en el sistema.		
Disparador	El administrador quiere eliminar un usuario del sistema.		
Escenario principal de uso			
1. El administrador indica que quiere eliminar un usuario. 2. El sistema elimina los datos correspondientes a la cuenta en la base de datos. 3. El sistema indica al administrador que el usuario se ha eliminado			

correctamente y redirige al administrador a la pantalla principal.

Caso de uso	#8 Añadir farmacéutico	Actor principal	Administrador
Precondiciones	El administrador ha iniciado sesión.		
Disparador	El administrador quiere añadir un farmacéutico al sistema.		
Escenario principal de uso			
1. El administrador indica que quiere añadir un nuevo farmacéutico y rellena el formulario de registro. 2. El sistema valida los datos. 3. El sistema guarda la información en la base de datos. 4. El sistema indica al administrador que el farmacéutico se ha añadido correctamente y redirige al administrador a la pantalla principal.			
Extensiones			
4a. El sistema determina que el correo ya está en uso. 4a1. El sistema avisa al administrador que el correo ya está en uso y permite al administrador volver a intentarlo.			

Caso de uso	#9 Eliminar farmacéutico	Actor principal	Administrador
Precondiciones	El administrador ha iniciado sesión. El farmacéutico está dado de alta en el sistema.		
Disparador	El administrador quiere eliminar un farmacéutico del sistema.		
Escenario principal de uso			
1. El administrador indica que quiere eliminar un usuario. 2. El sistema elimina los datos correspondientes a la cuenta en la base de datos. 3. El sistema indica al administrador que el usuario se ha eliminado correctamente y redirige al administrador a la pantalla principal.			

Caso de uso	#10 Consultar usuarios	Actor principal	Farmacéutico y Administrador
Precondiciones	El actor ha iniciado sesión.		
Disparador	El actor quiere ver los usuarios registrados en el sistema.		

Escenario principal de uso	
1. El actor indica que quiere ver los usuarios registrados en el sistema. 2. El sistema muestra los usuarios registrados al actor.	

Caso de uso	#11 Consultar farmacéuticos	Actor principal	Administrador
Precondiciones	El administrador ha iniciado sesión.		
Disparador	El administrador quiere ver los farmacéuticos registrados en el sistema.		
Escenario principal de uso			
1. El administrador indica que quiere ver los farmacéuticos registrados en el sistema. 2. El sistema muestra los farmacéuticos registrados al administrador.			

Usuario: Búsqueda y recomendaciones

Caso de uso	#12 Consultar catálogo de productos	Actor principal	Usuario
Precondiciones	-		
Disparador	El usuario quiere consultar el catálogo de productos.		
Escenario principal de uso			
1. El usuario indica que quiere consultar el catálogo de productos disponibles en el sistema. 2. El sistema muestra al usuario los productos disponibles.			

Caso de uso	#15 Filtrar por categoría	Actor principal	Usuario
Precondiciones	El usuario tiene acceso al catálogo de productos en el sistema.		
Disparador	El usuario quiere filtrar los productos del catálogo por categoría.		
Escenario principal de uso			

3. El usuario hace una búsqueda semántica de productos seleccionando categorías.
4. El sistema muestra los productos relacionados con la indicación del usuario.

Caso de uso	#16 Filtrar por prescripción	Actor principal	Usuario
Precondiciones	El usuario tiene acceso al catálogo de productos en el sistema.		
Disparador	El usuario quiere filtrar los productos del catálogo por prescripción.		
Escenario principal de uso			
1. El usuario hace una búsqueda semántica de productos seleccionando si los quiere con receta o sin receta. 2. El sistema muestra los productos relacionados con la indicación del usuario.			

Caso de uso	#17 Filtrar por precio	Actor principal	Usuario
Precondiciones	El usuario tiene acceso al catálogo de productos en el sistema.		
Disparador	El usuario quiere filtrar los productos del catálogo por precio.		
Escenario principal de uso			
1. El usuario hace una búsqueda semántica de productos seleccionando precios. 2. El sistema muestra los productos relacionados con la indicación del usuario.			

Caso de uso	#19 Consultar historial de búsquedas de productos	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión.		
Disparador	El usuario quiere consultar su historial de búsqueda de productos.		
Escenario principal de uso			
1. El usuario indica que quiere ver su historial de búsqueda de productos. 2. El sistema recupera el historial de búsqueda asociado al usuario. 3. El sistema muestra el historial del usuario en orden cronológico.			

Caso de uso	#20 Consultar FAQ	Actor principal	Usuario
Precondiciones	-		
Disparador	El usuario quiere consultar las preguntas y respuestas más frecuentes.		
Escenario principal de uso			
1. El usuario indica que quiere ver las preguntas y respuestas más frecuentes. 2. El sistema muestra las preguntas y respuestas más frecuentes al usuario.			

Caso de uso	#22 Valorar resultados de recomendaciones	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión. El sistema ha mostrado recomendaciones.		
Disparador	El usuario quiere valorar la relevancia de los resultados de las recomendaciones.		
Escenario principal de uso			
1. El usuario indica su valoración después de realizar una búsqueda. 2. El sistema muestra al usuario la valoración realizada.			

Usuario: Gestión de recetas

Caso de uso	#24 Consultar recetas electrónicas	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión.		
Disparador	El usuario quiere consultar sus recetas electrónicas.		
Escenario principal de uso			
1. El usuario indica que quiere ver sus recetas electrónicas. 2. El sistema muestra las recetas electrónicas asociadas al usuario.			

Caso de uso	#25 Eliminar receta electrónica	Actor principal	Usuario
--------------------	---------------------------------	------------------------	---------

Precondiciones	El usuario ha iniciado sesión.
Disparador	El usuario quiere eliminar una receta electrónica suya.
Escenario principal de uso	
1. El usuario indica que quiere eliminar su receta. 2. El sistema elimina los datos correspondientes a la receta en la base de datos. 3. El sistema indica al usuario que la receta ha sido eliminada correctamente.	

Caso de uso	#27 Consultar recetas privadas	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión.		
Disparador	El usuario quiere consultar sus recetas privadas.		
Escenario principal de uso			
1. El usuario indica que quiere ver sus recetas privadas. 2. El sistema muestra las recetas privadas asociadas al usuario.			

Caso de uso	#28 Eliminar receta privada	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión.		
Disparador	El usuario quiere eliminar una receta privada suya.		
Escenario principal de uso			
1. El usuario indica que quiere eliminar su receta. 2. El sistema elimina los datos correspondientes a la receta en la base de datos. 3. El sistema indica al usuario que la receta ha sido eliminada correctamente.			

Usuario y farmacéutico: Asesoramiento farmacéutico

Caso de uso	#33 Consultar consultas del usuario	Actor principal	Farmacéutico
Precondiciones	El farmacéutico ha iniciado sesión. El usuario existe en el sistema. Existe al menos una consulta del usuario en el sistema.		
Disparador	El farmacéutico quiere consultar las consultas del usuario.		
Escenario principal de uso			
1. El farmacéutico indica que quiere ver las consultas del usuario. 2. El sistema muestra al farmacéutico las consultas del usuario.			

Usuario y farmacéutico: Gestión de Encargos

Caso de uso	#37 Consultar carrito	Actor principal	Usuario
Precondiciones	-		
Disparador	El usuario quiere consultar su carrito.		
Escenario principal de uso			
1. El usuario indica que quiere consultar su carrito de productos. 2. El sistema muestra al usuario la lista de productos que el usuario ha añadido al carrito.			

Caso de uso	#39 Consultar encargos	Actor principal	Usuario
Precondiciones	El usuario ha iniciado sesión. El usuario ha realizado al menos un encargo.		
Disparador	El usuario quiere consultar sus encargos.		
Escenario principal de uso			
1. El usuario indica que quiere consultar sus encargos. 2. El sistema muestra al usuario los encargos del usuario.			

Caso de uso	#40 Consultar encargos del usuario	Actor principal	Farmacéutico
Precondiciones	El farmacéutico ha iniciado sesión.		
Disparador	El farmacéutico quiere consultar los encargos del usuario.		
Escenario principal de uso			
1. El farmacéutico indica que quiere ver los encargos del usuario. 2. El sistema muestra los encargos del usuario.			

Administrador: Gestión de productos

Caso de uso	#43 Añadir nuevo producto	Actor principal	Administrador
Precondiciones	El administrador ha iniciado sesión.		
Disparador	El administrador quiere añadir un nuevo producto al sistema.		
Escenario principal de uso			
1. El administrador indica que quiere añadir un nuevo producto e introduce los datos del producto. 2. El sistema valida los datos. 3. El sistema guarda la información en la base de datos. 4. El sistema indica al administrador que el producto se ha añadido correctamente y redirige al administrador a la pantalla inicial.			
Extensiones			
4a. El sistema indica que la clave primaria ya está en uso. 4a1. El sistema avisa al administrador que la clave primaria ya está en uso y permite al administrador volver a intentarlo.			

Caso de uso	#45 Eliminar producto	Actor principal	Administrador
Precondiciones	El administrador ha iniciado sesión. Existe al menos 1 producto en el sistema.		
Disparador	El administrador quiere eliminar un producto.		
Escenario principal de uso			

1. El administrador indica que quiere eliminar un producto.
2. El sistema elimina los datos correspondientes al producto en la base de datos.
3. El sistema indica al administrador que el producto se ha eliminado correctamente y redirige al administrador a la pantalla principal.

Anexo 2: Modelo de comportamiento

#1 Registro

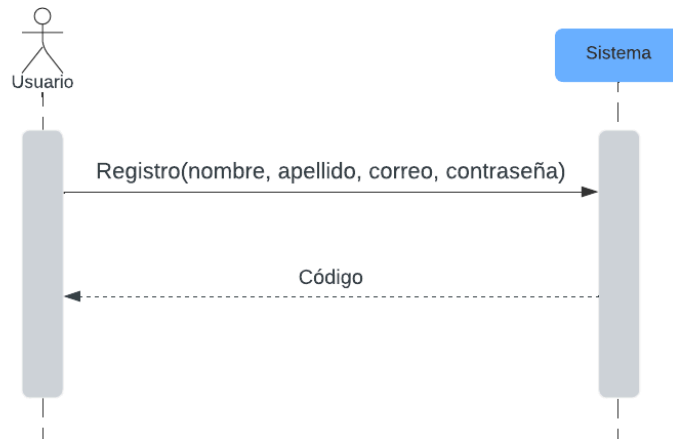


Figura 88: Diagrama de registro. Elaborado con Lucidchart.

Registro(nombre: String, apellido: String, correo: String, contraseña: String)	
Precondición	-
Postcondición	Se crea un usuario con los datos introducidos.
Body	Código indicando si la operación ha sido un éxito o no.

#2 Iniciar Sesión

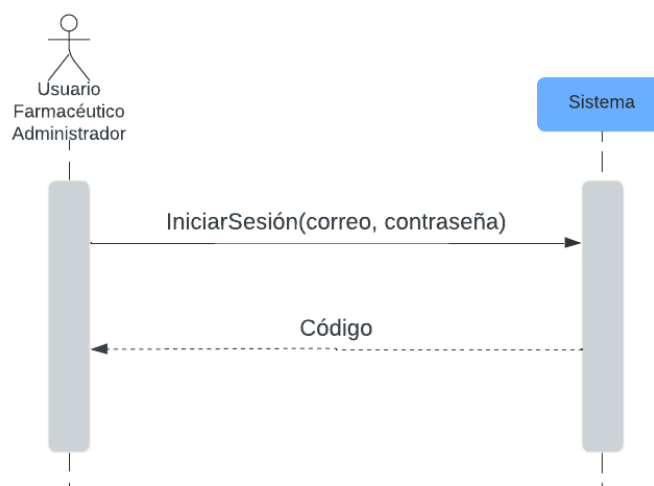


Figura 89: Diagrama de inicio de sesión. Elaborado con Lucidchart.

IniciarSesión(correo: String, contraseña: String)	
Precondición	-
Postcondición	-
Body	Código de acceso indicando si la operación ha sido un éxito.

#3 Cerrar Sesión

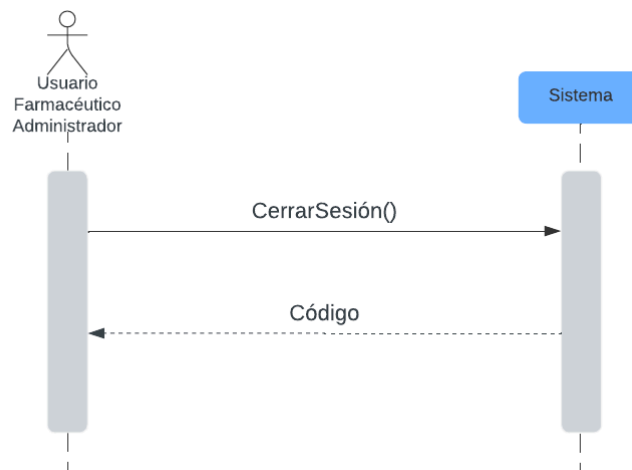


Figura 90: Diagrama de cerrar sesión. Elaborado con Lucidchart.

CerrarSesión()	
Precondición	-
Postcondición	El actor ya no tiene la sesión abierta en el sistema.
Body	Código de acceso indicando si la operación ha sido un éxito.

#5 Eliminar cuenta

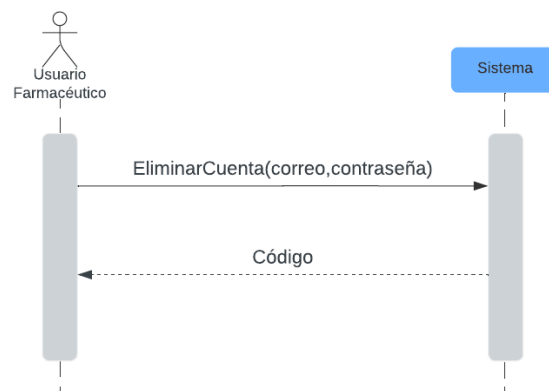


Figura 91: Diagrama de eliminar cuenta. Elaborado con Lucidchart.

EliminarCuenta(correo:String, contraseña: String)	
Precondición	correo existe en el sistema.
Postcondición	Se elimina el actor del sistema.
Body	Se devuelve un código de aceptación si se ha eliminado la cuenta o un código de error si la contraseña introducida no es la contraseña del actor.

Para los casos de uso **#7 Eliminar usuario** y **#9 Eliminar farmacéutico** pasa lo mismo, el modelo de comportamiento es el mismo que el caso de uso **#5 Eliminar cuenta**. Por lo tanto, no hace falta definir los diagramas de secuencia de estos dos casos.

Para el caso de uso **#8 Añadir farmacéutico**, el modelo de comportamiento es el mismo que el de caso de uso **#1 Registro**. Lo único que cambia es que el actor es el administrador.

#10 Consultar usuarios

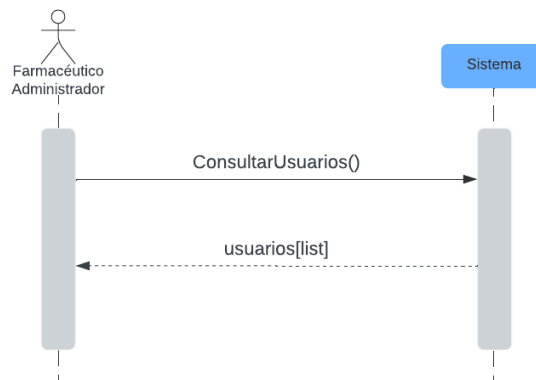


Figura 92: Diagrama de consultar usuarios. Elaborado con Lucidchart.

ConsultarUsuarios()	
Precondición	-
Postcondición	-
Body	Se devuelve la lista de usuarios registrados en el sistema.

Para el caso de uso **#11 Consultar farmacéuticos** el modelo de comportamiento es el mismo que el **#10 Consulta usuarios**. Lo único que cambia es que el actor es el administrador y se devuelve una lista de farmacéuticos.

#12 Consultar catálogo de productos

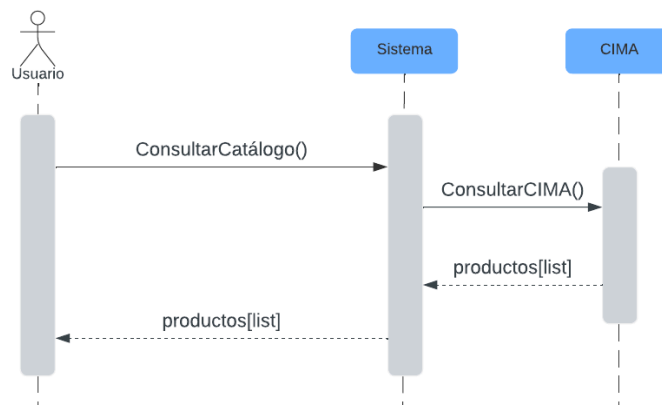


Figura 93: Diagrama de consultar catálogo. Elaborado con Lucidchart.

ConsultarCatálogo()	
Precondición	-
Postcondición	-
Body	Se devuelve la lista de productos del catálogo.

#19 Consultar historial de búsqueda de productos

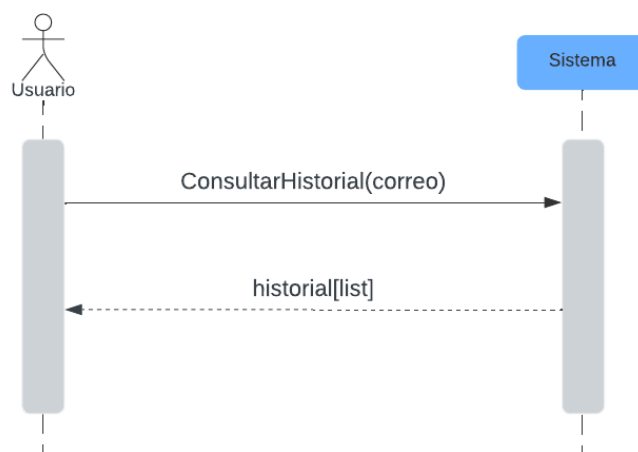


Figura 94: Diagrama de consultar historial. Elaborado con Lucidchart.

ConsultarHistorial(correo: String)	
Precondición	correo existe en el sistema.
Postcondición	-

Body	Se devuelve la lista de productos del historial que ha buscado el usuario.
-------------	--

#20 Consultar FAQ

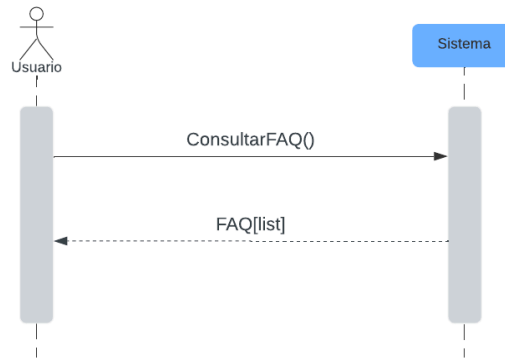


Figura 95: Diagrama de consultar FAQ. Elaborado con Lucidchart.

ConsultarFAQ()	
Precondición	-
Postcondición	-
Body	Se devuelve la lista de preguntas y respuestas más frecuentes.

#24 Consultar recetas electrónicas

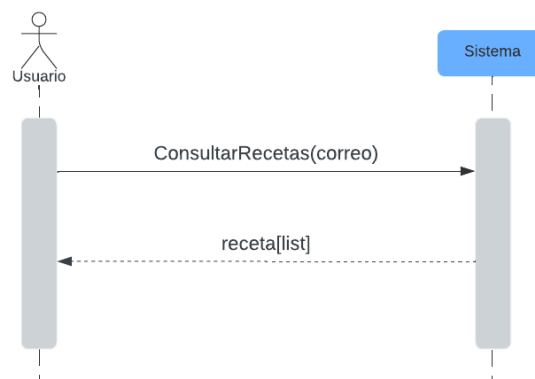


Figura 96: Diagrama de consultar recetas. Elaborado con Lucidchart.

ConsultarRecetas(correo: String)	
Precondición	correo existe en el sistema.
Postcondición	-
Body	Se devuelve la lista de recetas que ha subido el usuario.

#25 Eliminar receta electrónica

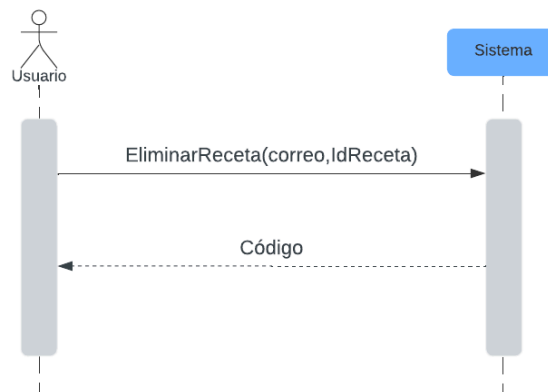


Figura 97: Diagrama de eliminar receta. Elaborado con Lucidchart.

EliminarReceta(IdReceta: String)	
Precondición	correo existe en el sistema. IdReceta existe en el sistema.
Postcondición	La receta IdReceta se elimina del sistema.
Body	Se devuelve un código de aceptación si se ha eliminado la receta, sino se devuelve código de error.

Los modelos de comportamiento de los casos de uso **#26 Cargar receta privada**, **#27 Consultar recetas privadas** y **#28 Eliminar receta privada** son iguales a **#23 Cargar receta electrónica**, **#24 Consultar recetas electrónicas** y **#25 Eliminar receta electrónica**, respectivamente.

El modelo de comportamiento del caso de uso **#37 Consultar carrito** es igual al **#24 Consultar recetas electrónicas**, lo que cambia es que devuelve un productos[list].

#39 Consultar encargos

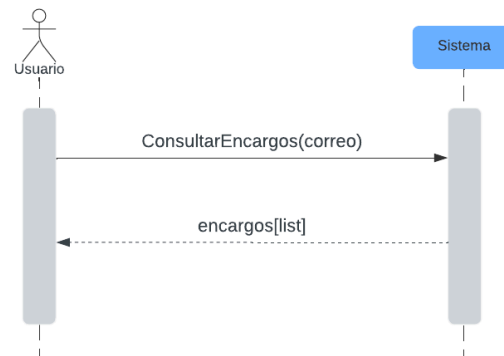


Figura 98: Diagrama de consultar encargos. Elaborado con Lucidchart.

ConsultarEncargos(correo: String)	
Precondición	correo existe en el sistema.
Postcondición	-
Body	Se devuelve los encargos asociados al usuario.

El modelo de comportamiento del caso de uso **#40 Consultar encargos del usuario** es igual que el #39 Consultar encargos. Lo único que cambia es que el actor que ejecuta la acción es el **farmacéutico**.

#43 Añadir nuevo producto

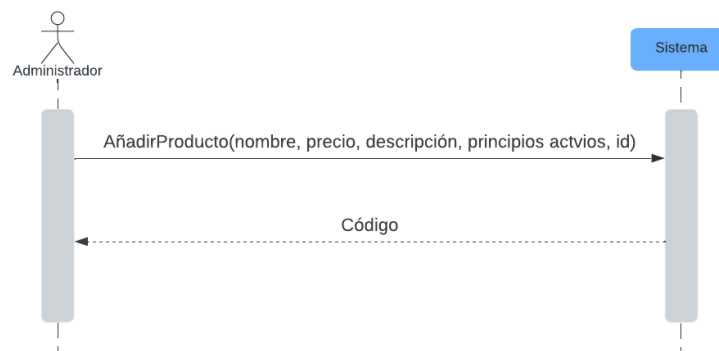


Figura 99: Diagrama de añadir producto. Elaborado con Lucidchart.

AñadirProducto(nombre: String, precio: Int, descripción: String, principios activos: Array[String], id: String)	
Precondición	-
Postcondición	Se crea un producto con los datos introducidos.

Body	Código indicando si la operación ha sido un éxito o no.
-------------	---

#45 Eliminar producto



Figura 100: Diagrama de eliminar producto. Elaborado con Lucidchart.

EliminarProducto(id: String)	
Precondición	El <i>id</i> del producto existe.
Postcondición	Se elimina el producto del sistema.
Body	Se devuelve un código de aceptación si se ha eliminado el producto o un código de error de lo contrario.

Anexo 3: Diseño externo

Inicio (usuarios):

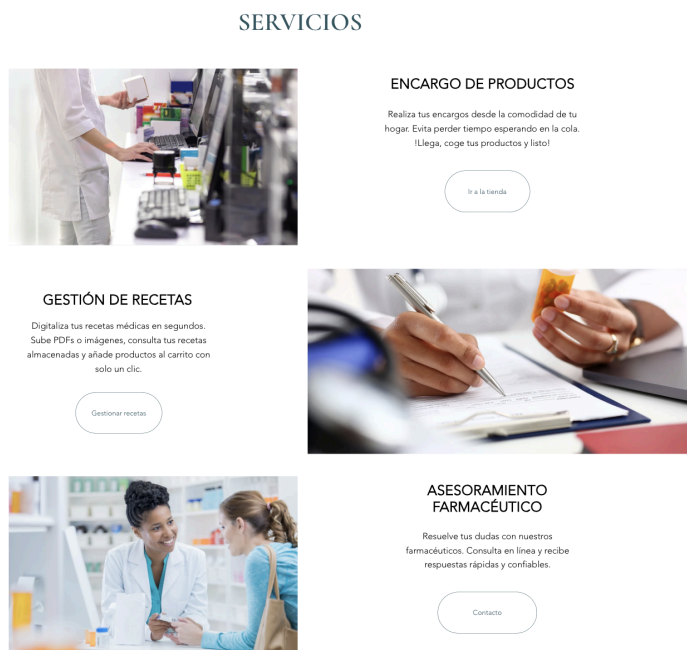


Figura 101: Diseño de inicio de usuarios II. Elaboración propia.

Más abajo, se presentan los principales servicios que ofrece la farmacia, incluyendo la gestión de recetas electrónicas, la posibilidad de realizar encargos de productos y un servicio de asesoramiento farmacéutico personalizado. Estos servicios están organizados de forma clara y accesible para facilitar su uso por parte de los clientes.

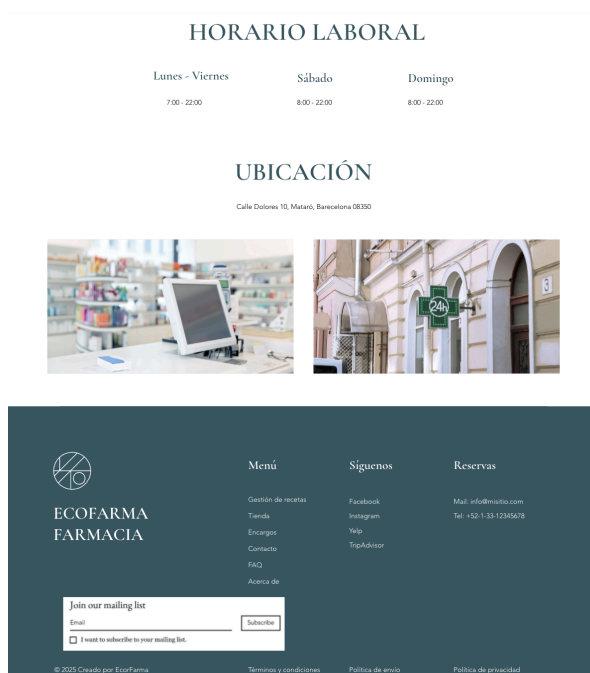


Figura 102: Diseño de inicio de usuarios III. Elaboración propia.

En la parte inferior de la pantalla, se muestra el horario y la ubicación de la farmacia. El pie de página incluye opciones para suscribirse a notificaciones sobre ofertas y novedades, así como enlaces a la política de privacidad y términos y condiciones.

Inicio de Sesión (usuarios, farmacéuticos y administradores):



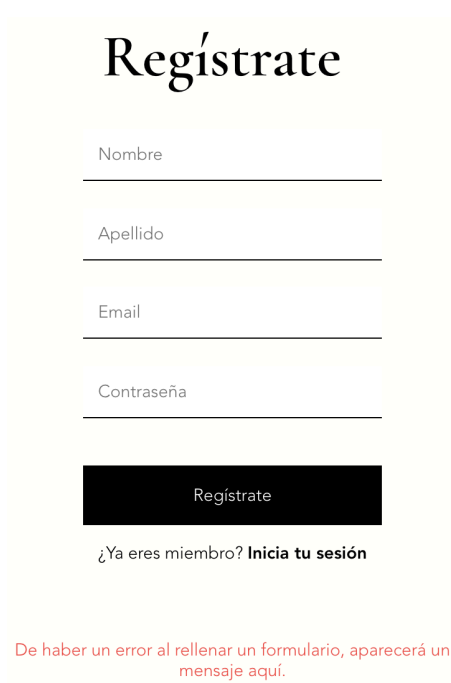
The login page design features a light gray background. At the top, the heading 'Iniciar sesión' is displayed in a large, bold, black font. Below it, a link '¿Es tu primera vez? [Regístrate](#)' is shown in a smaller black font. The form consists of two input fields: 'Email *' and 'Contraseña *', both with light gray borders and placeholder text. Below the password field, there is a link '¿Olvidaste la contraseña?' and a red error message 'Un mensaje aparecerá aquí de haber un error.' At the bottom, a black button with the text 'Iniciar sesión' in white is centered.

La página de inicio de sesión permite a los usuarios acceder a sus cuentas mediante un formulario donde deben ingresar su correo electrónico y contraseña. Su diseño es sencillo y claro, facilitando una experiencia rápida y segura.

Si el usuario no tiene una cuenta, se proporciona un enlace para ir a la página de registro. Si el usuario olvida su contraseña, puede utilizar el enlace "Recuperar contraseña", que le permitirá restablecerla a través de su correo electrónico. Si el inicio de sesión es correcto, se redirige al usuario a la página de inicio.

Figura 103: *Diseño de inicio de sesión.*
Elaboración propia.

Registro (usuarios):



The registration page design has a light gray background. The heading 'Regístrate' is at the top in a large, bold, black font. Below it are four input fields: 'Nombre', 'Apellido', 'Email', and 'Contraseña', each with a light gray border and placeholder text. At the bottom, there is a black button with the text 'Regístrate' in white. Below the button, a link '¿Ya eres miembro? [Inicia tu sesión](#)' is shown. At the very bottom, a red error message 'De haber un error al rellenar un formulario, aparecerá un mensaje aquí.' is displayed.

La página de registro permite a los nuevos usuarios crear una cuenta completando un formulario con su nombre, apellido, correo electrónico y contraseña. El diseño del formulario es claro e intuitivo, validando los datos ingresados para garantizar una experiencia segura y sin errores.

Debajo del formulario, se incluye un enlace a la página de "Iniciar sesión" para aquellos que ya tienen una cuenta. Esto facilita la navegación y evita confusiones, asegurando que los usuarios puedan acceder rápidamente al sistema. Si el registro es correcto, se redirige al usuario a la página de inicio.

Figura 104: *Diseño de registro.*
Elaboración propia.

Recuperación de Contraseña (usuarios, farmacéuticos y administradores):

Recuperar contraseña

Introduce tu Email para enviar el código de verificación

Email *

Revise su bandeja de entrada para introducir el código recibido

Código *

Nueva Contraseña *

Un mensaje aparecerá aquí de haber un error.

La página de recuperación de contraseña permite a los usuarios restablecer su acceso en caso de olvido. Primero, se solicita el correo electrónico asociado a la cuenta para enviar un código de verificación. Este código asegura que solo el propietario de la cuenta pueda cambiar la contraseña.

Luego, el usuario debe ingresar el código recibido y establecer una nueva contraseña segura. Al confirmar los cambios, si todo es correcto, el sistema redirige automáticamente a la página de "Iniciar sesión", donde el usuario podrá acceder con su nueva contraseña.

Figura 105: Diseño de recuperar contraseña.
Elaboración propia.

Perfil de Usuario (usuario, farmacéutico, administrador):

ECOFARMA Farmacia

Gestión de recetas Tienda Encargos Contacto Acerca de

Albert Comas Pacheco

Mis recetas Mis encargos Mis recordatorios Historial de búsqueda Mi cuenta

Cuenta

Mira y edita tu información personal.

Información personal

Actualiza tu información personal.

Nombre

Apellido

Correo

Información de inicio de sesión

Revisa y actualiza tu email de inicio de sesión y contraseña.

Email de inicio de sesión:
albert.comas.pacheco@estudiantat.upc.edu
Los propietarios del sitio solo pueden cambiar su email de inicio de sesión en los ajustes de la cuenta de Wix.

Contraseña:
•••••
Los propietarios del sitio solo pueden cambiar su contraseña en los ajustes de la cuenta de Wix.

La página de perfil de usuario permite a los usuarios registrados, farmacéuticos y administradores gestionar su información personal. Incluye un formulario donde pueden modificar datos como nombre, apellido y correo, asegurando que la información esté siempre actualizada.

Además, los usuarios tienen la opción de eliminar su cuenta de forma permanente. Sin embargo, esta opción no estará disponible para farmacéuticos y administradores.

Figura 106: Diseño de perfil de usuario.
Elaboración propia.

Historial de Búsquedas (usuarios):

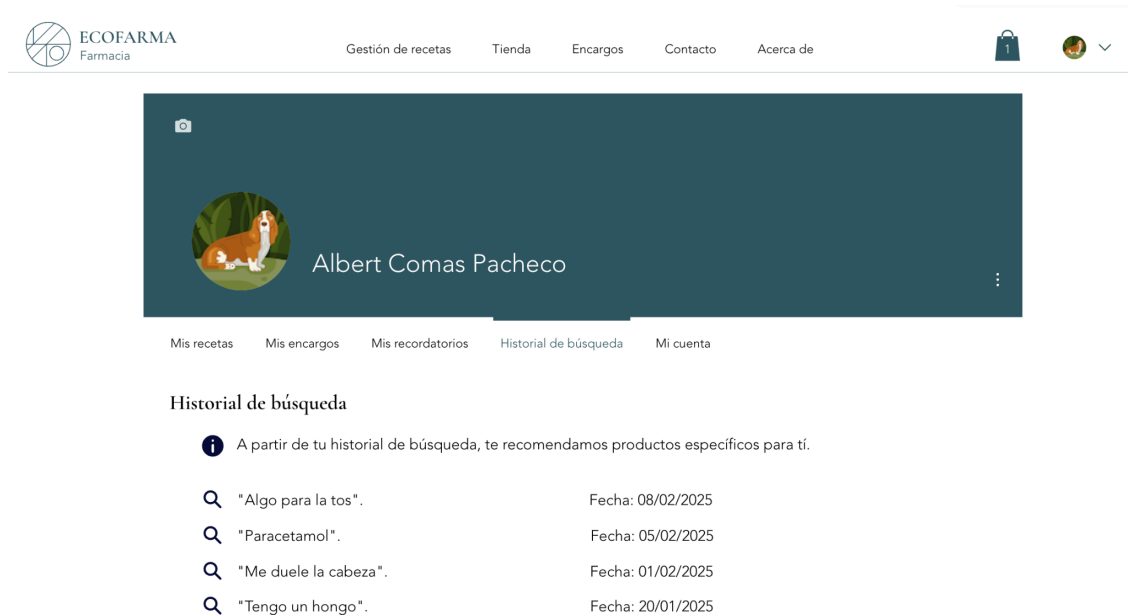


Figura 107: Diseño de historial de búsqueda. Elaboración propia.

La página de historial de búsquedas permite a los usuarios consultar un registro de las búsquedas realizadas en el catálogo de productos. Cada entrada muestra el término buscado y la fecha en que se realizó la consulta, proporcionando un acceso rápido a búsquedas anteriores. Esta funcionalidad facilita la reutilización de consultas frecuentes, mejorando la eficiencia del usuario.

Recordatorios (usuarios):

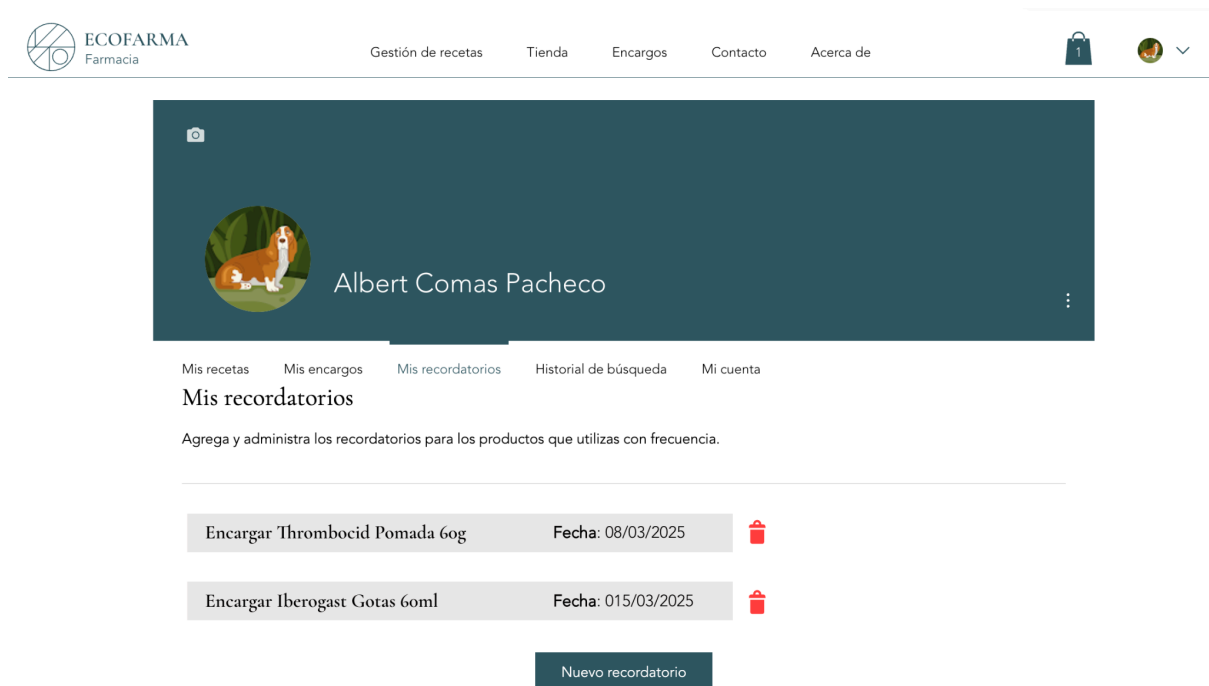


Figura 108: Diseño de recordatorios. Elaboración propia.

La página de recordatorios permite a los usuarios gestionar avisos para encargar medicamentos de uso frecuente. Muestra una lista de los medicamentos a recordar, incluyendo el nombre del producto y la fecha en la que debe ser encargado. Esta funcionalidad ayuda a los usuarios a no olvidar la renovación de sus tratamientos.

En la parte inferior de la página, un botón de "Nuevo Recordatorio" permite agregar nuevos medicamentos a la lista, configurando la fecha del próximo encargo. El diseño es intuitivo y centrado en la comodidad del usuario, facilitando la planificación y organización de sus necesidades médicas.

FAQ (usuarios):

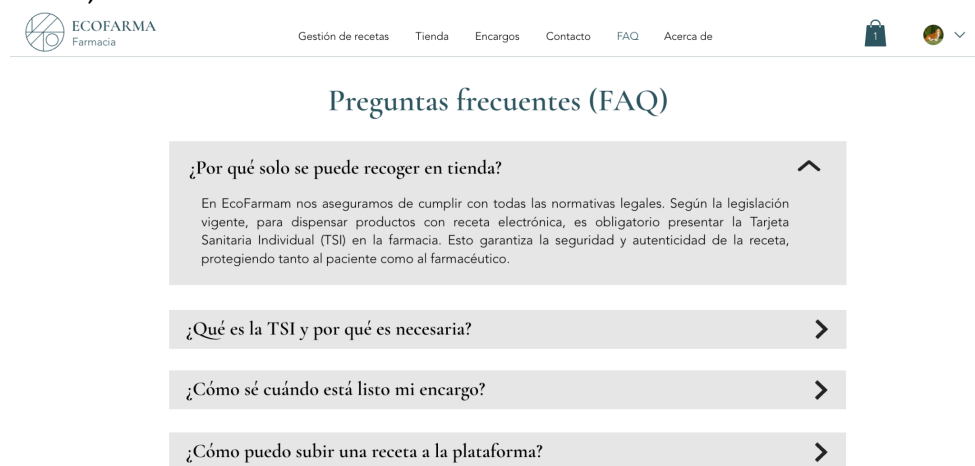


Figura 109: Diseño de FAQ. Elaboración propia.

La página de FAQ ofrece respuestas a las dudas más comunes sobre el uso de la plataforma, como la gestión de recetas, encargos, asesoramiento farmacéutico y políticas de privacidad. Su diseño claro y conciso permite a los usuarios resolver sus inquietudes de manera rápida y eficiente.

Gestión de Encargos (farmacéuticos):



Figura 110: Diseño de encargos para farmacéuticos. Elaboración propia.

Esta página permite a los farmacéuticos visualizar todos los encargos realizados por los usuarios. Muestra detalles como el número de encargo, el nombre del usuario, la fecha de recogida, la lista de productos y el precio total. Además, incluye una opción para cambiar el estado del encargo (pendiente, listo para recoger, entregado).

Esta herramienta facilita la organización y el seguimiento de los pedidos, asegurando una gestión eficiente y un servicio ágil para los usuarios. El diseño es claro y funcional, optimizado para una rápida toma de decisiones.

Consulta/Gestión de Usuarios (farmacéuticos y administradores):



Figura 111: *Diseño de consulta de usuarios.* Elaboración propia.

La página de Consulta/Gestión de Usuarios permite a farmacéuticos y administradores visualizar la información de los usuarios del sistema. Para los farmacéuticos, la funcionalidad es solo de consulta, mostrando un listado con el nombre, apellido, correo, encargos activos y consultas activas de cada usuario. Los botones de eliminar y añadir usuario no serán visibles.

Para los administradores, además de la consulta, la página permite la gestión de usuarios. Se añaden botones para eliminar usuarios y añadir nuevos usuarios, abriendo un formulario similar al de la página de registro. También se incluye un buscador para filtrar rápidamente a los usuarios.

Gestión de Farmacéuticos (administradores):

GESTIÓN DE FARMACÉUTICOS

Buscar farmacéuticos... 

Añadir farmacéutico

Nombre: farma	Apellido: Test		
Farmacéutico: farma@gmail.com			
Nombre: María	Apellido: Díaz		

Figura 112: Diseño de gestión de farmacéuticos. Elaboración propia.

La página de Gestión de Farmacéuticos permite al administrador visualizar y administrar a los farmacéuticos del sistema. Se muestra un listado con nombre, apellido y correo, junto con botones para eliminar o añadir nuevos farmacéuticos. Además, incluye un buscador para localizar rápidamente a un farmacéutico específico.

Gestión del Catálogo de Productos (administradores):

Añadir nuevo producto

Nombre *	Precio *
Ingresa el nombre	Ingresa el precio
Identificador *	
Ingresa el identificador	
Ha de ser un número de 5 dígitos.	
Principios activos *	
Ingresa los principios activos del producto	
Descripción *	
Describe el producto	
Confirmar	

Figura 113: Diseño de añadir productos. Elaboración propia.

El formulario de añadir producto permite registrar un nuevo artículo en el catálogo completando los campos de nombre, precio, identificador, principios activos y descripción. Este proceso garantiza que cada producto tenga la información necesaria para su correcta gestión y visualización en la tienda.

Editar producto: 11245

Nombre	Precio
Paracetamol 500mg	€5.99

Descripción

El paracetamol es eficaz para reducir el dolor y la fiebre. Está indicado para el alivio sintomático

Confirmar

Figura 114: Diseño de editar producto. Elaboración propia.

El formulario de edición de producto facilita la modificación de productos existentes, permitiendo actualizar su nombre, precio y descripción. Esto permite mantener la información siempre precisa y actualizada, asegurando que los usuarios dispongan de datos fiables al consultar el catálogo.

Estadísticas y Reportes (administradores):



Figura 115: Diseño de estadísticas. Elaboración propia.

La página de estadísticas proporciona una visión general del rendimiento del sistema a través de dos gráficos clave. El primero muestra un informe sobre la valoración de las búsquedas, y el segundo un informe sobre la valoración de las recomendaciones. Estos datos ayudan a evaluar y mejorar la experiencia del usuario en la plataforma.